

MCU User Guide for Myshkin

Revised February 17th, 2026

University of Nebraska–Lincoln
Department of Electrical & Computer Engineering

Contents

1	System Configuration	7
2	Packaging and Pins Configuration	8
3	Address Space	11
4	Startup and Reset Behavior	12
5	Interrupts	13
6	CPU Instructions and Assembly	15
6.1	CPU Registers	15
6.2	Load/Store Memory Instructions	16
6.3	Shift Instructions	16
6.4	Arithmetic Instructions	17
6.5	Bitwise Instructions	17
6.6	Comparison Instructions	18
6.7	Branch Instructions	18
6.8	Jump and Link Instructions	19
6.9	Pseudo-Instructions	19
7	Forth Interpreter	21
7.1	Memory Access Functions	21
7.2	Print and Input Functions	22
7.3	Arithmetic Functions	23
7.4	Bitwise Logic Functions	25
7.5	Comparison Functions	27
7.6	Boolean Functions	27
7.7	Stack Manipulation Functions	28
7.8	SPI Flash R/W and Programming Functions	30
7.9	Miscellaneous Functions	30
7.10	Creating New Forth Functions	32
7.11	Conditionals	32
7.12	Loops	32
8	Software Development and Build Process	33
8.1	The RISC-V Toolchain and its Extensions	33
8.2	Getting the RISC-V Toolchain on Linux	33
8.3	Getting the RISC-V Toolchain on Windows	34
8.4	Compilation	34
8.5	Linking	35
8.6	Intel Hex File Generation	36
8.7	A Note on Using C++	36
9	Uploading Program to SPI Flash	37
9.1	SPI Flash Boot Process	37
9.2	Upload Process	37

10 GPIOx Peripheral	39
10.1 Primary/GPIO Mode Functionality	39
10.2 Secondary/Peripheral Mode Functionality	39
10.3 Pin Interrupts	40
10.4 Register Description	41
10.4.1 PIN Register	41
10.4.2 POUT Register	41
10.4.3 POUTS Register	41
10.4.4 POUTC Register	41
10.4.5 POUTT Register	42
10.4.6 PDIR Register	42
10.4.7 PIFG Register	42
10.4.8 PIES Register	42
10.4.9 PIE Register	43
10.4.10 PSEL Register	43
10.4.11 PREN Register	43
11 SPIx Peripheral	44
11.1 SPI Pins	44
11.2 Generalized SPI Transfer Process	45
11.3 Bit and Byte Ordering	46
11.4 SPI Baud Clock	47
11.5 Transmit Buffer Register Queue	47
11.6 Flags and Interrupts	47
11.7 Master Mode Transfer Procedure	47
11.8 Slave Mode Transfer Procedure	49
11.9 External SPI Flash Native Memory Read Access	49
11.10 SPI0 Boot Behavior	50
11.11 Register Description	51
11.11.1 SPICR Register	51
11.11.2 SPISR Register	52
11.11.3 SPITX Register	53
11.11.4 SPIRX Register	53
11.11.5 SPIFOS Register	54
12 UARTx Peripheral	55
12.1 UART Pins	55
12.2 Generalized UART Transmission Process	55
12.3 UART Baud Generation	56
12.4 Parity Bit	56
12.5 Transmit Buffer Register Queue	57
12.6 Flags and Interrupts	57
12.7 Transmit Procedure	58
12.8 Receive Procedure	58
12.9 UART0 Boot Behavior	58
12.10 Register Description	60
12.10.1 UARTCR Register	60
12.10.2 UARTSR Register	60

12.10.3	UARTBR Register	61
12.10.4	UARTRX Register	61
12.10.5	UARTTX Register	62
13	TIMERx Peripheral	63
13.1	Timer Pins	63
13.2	Clock Sources and Divider	63
13.3	Starting, Halting, Setting, and Reading the Timer	64
13.4	Overflow and Rollover Behavior	64
13.5	Output Compare Units and Pulse Width Modulation	64
13.6	Input Capture Units	65
13.7	Flags and Interrupts	65
13.8	Register Description	67
13.8.1	TIMCR Register	67
13.8.2	TIMSR Register	68
13.8.3	TIMVAL Register	69
13.8.4	TIMCMP0 Register	70
13.8.5	TIMCMP1 Register	70
13.8.6	TIMCMP2 Register	71
13.8.7	TIMCAP0 Register	71
13.8.8	TIMCAP1 Register	72
14	SYSTEM Peripheral	73
14.1	System Clocks and Clock Management	73
14.2	Power Saving Features	73
14.3	CPU Sleep Mode	74
14.4	CRC Module	75
14.5	Global Analog Bias Generator	76
14.6	Global Bandgap Voltage Generator	76
14.7	Register Description	78
14.7.1	SYSCLKCR Register	78
14.7.2	CLKDIVCR Register	79
14.7.3	MEMPWRCR Register	79
14.7.4	CRCDATA Register	81
14.7.5	CRCSTATE Register	82
14.7.6	IRQEN Register	82
14.7.7	IRQPRI Register	82
14.7.8	WDTPASS Register	83
14.7.9	WDTCR Register	83
14.7.10	WDTSR Register	85
14.7.11	WDTVAL Register	85
14.7.12	DC00FREQ Register	86
14.7.13	DC01FREQ Register	86
14.7.14	TPMR Register	87
14.7.15	BIASCR Register	87
14.7.16	BIASDBP Register	88
14.7.17	BIASDBPC Register	88
14.7.18	BIASDBNC Register	88

14.7.19	BIASDBN Register	89
15	NNx Peripheral	90
15.1	Register Description	90
15.1.1	NNCR Register	90
15.1.2	NNIVA Register	91
15.1.3	NNOVA Register	92
15.1.4	NNWMA Register	92
15.1.5	NNLSI Register	93
15.1.6	NNLSO Register	93
16	SARADCx Peripheral	95
16.1	Register Description	95
16.1.1	SARADCCR Register	95
16.1.2	SARADCCDIV Register	95
16.1.3	SARADCSR Register	96
16.1.4	SARADCDATA Register	96
16.1.5	SARADCTPR Register	97
17	AFEx Peripheral	98
17.1	Register Description	98
17.1.1	AFECR0 Register	98
17.1.2	AFECR1 Register	99
17.1.3	AFECFB Register	99
17.1.4	AFERFB Register	100
17.1.5	AFETHR Register	100
17.1.6	AFETPR Register	100
17.1.7	AFESPT Register	101
17.1.8	AFEPIT Register	101
17.1.9	AFEEIT Register	102
17.1.10	AFELIT Register	102
17.1.11	AFERJT Register	102
17.1.12	AFERST Register	103
17.1.13	AFEAOFST Register	103
17.1.14	AFEBLLT Register	103
17.1.15	AFECSAREF Register	104
17.1.16	AFECSABP Register	104
17.1.17	AFECSABPC Register	104
17.1.18	AFECSABNC Register	104
17.1.19	AFECSABN Register	105
17.1.20	AFECMSHR Register	105
17.1.21	AFECLPF Register	105
17.1.22	AFESR Register	106
17.1.23	AFEADCVAL Register	106
17.1.24	AFEVPC Register	106
17.1.25	AFETPC Register	107

18 I2Cx Peripheral	108
18.1 Register Description	109
18.1.1 I2CCR Register	109
18.1.2 I2CFR Register	111
18.1.3 I2CSR Register	112
18.1.4 I2CMTX Register	115
18.1.5 I2CMRX Register	115
18.1.6 I2CSTX Register	115
18.1.7 I2CSRX Register	115
18.1.8 I2CAR Register	116
18.1.9 I2CAMR Register	116
19 Register and Peripheral Memory Map	117
19.1 GPIO0 Peripheral	117
19.2 GPIO1 Peripheral	117
19.3 SPI0 Peripheral	117
19.4 SPI1 Peripheral	117
19.5 UART0 Peripheral	118
19.6 UART1 Peripheral	118
19.7 TIMER0 Peripheral	118
19.8 TIMER1 Peripheral	118
19.9 GPIO2 Peripheral	119
19.10 SYSTEM Peripheral	119
19.11 NN0 Peripheral	120
19.12 SARADC0 Peripheral	120
19.13 AFE0 Peripheral	120
19.14 GPIO3 Peripheral	121
19.15 I2C0 Peripheral	121
19.16 I2C1 Peripheral	121
20 Errata	123

1 System Configuration

The microcontroller (MCU) described in this document utilizes the picorv32 CPU core, a 32-bit RISC-V processor. The microcontroller is composed of the RISC-V CPU, a ROM that contains boot software and a forth interpreter, RAM memory cell(s), and several peripherals. The MCU hardware is configurable using a custom automatic code generation tool, which also automatically generates this user guide document to reflect its configuration.

2 Packaging and Pins Configuration

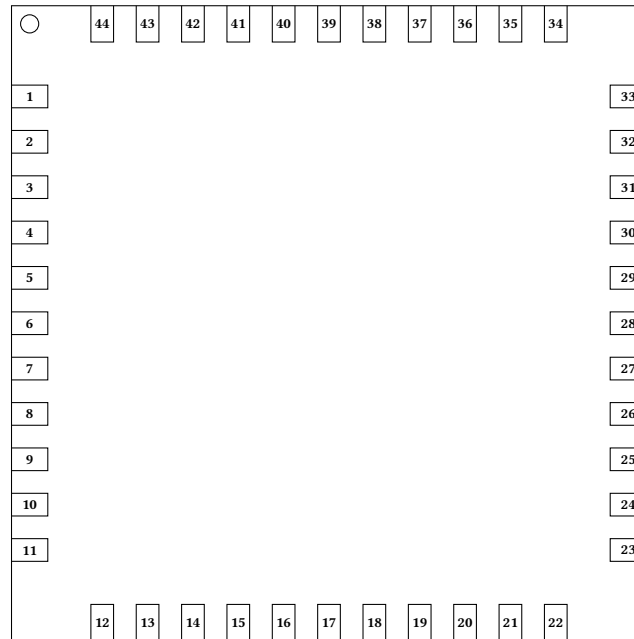


Figure 1: QFN-44 (7 × 7 mm, 0.5 mm pitch)

The chip package is a QFN-44 with dimensions 7 × 7 mm and pitch 0.5 mm. The package footprint (as seen from the top down) is shown in Figure 1. The pins on the package are listed in Table 1.

Table 1: Package Pins

#	Pin	I/O	Power Domain	Power Pins
1	P3.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
2	P2.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
3	P2.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
4	P2.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
5	P2.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
6	P2.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
7	P2.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
8	P2.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
9	P2.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
10	VDD	Power Input	Digital Core (1.0 V)	VDD/VSS
11	RESETN	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
12	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
13	P1.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
14	P1.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
15	P1.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
16	P1.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
17	P1.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
18	P1.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST

Table 1 continued on next page

Table 1 continued from previous page

#	Pin	I/O	Power Domain	Power Pins
19	P1.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
20	P1.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
21	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
22	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
23	NC	–	–	–
24	P0.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
25	P0.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
26	P0.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
27	P0.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
28	P0.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
29	P0.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
30	P0.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
31	P0.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
32	AVSS	Power Input	Analog (3.3 V)	AVDD/AVSS
33	ECS0.RE	Output	Analog (3.3 V)	AVDD/AVSS
34	ECS0.CE	Output	Analog (3.3 V)	AVDD/AVSS
35	ATP-IN	Input	Analog (3.3 V)	AVDD/AVSS
36	ATP-OUT	Output	Analog (3.3 V)	AVDD/AVSS
37	AVDD	Power Input	Analog (3.3 V)	AVDD/AVSS
38	P3.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
39	P3.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
40	P3.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
41	P3.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
42	P3.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
43	P3.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
44	P3.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST

Many of the digital I/O pins are GPIO pins. These pins can be in either primary/GPIO mode, or secondary/peripheral mode (see Section 10 on the GPIO peripherals for more details). A list of all the GPIO pins, along with their primary and secondary functions and reset values, is shown in Table 2.

Table 2: GPIO Pins

Pin	Primary Function	Secondary Function	Reset Values			
			SEL	OUT	DIR	REN
P0.0	–	CS_FLASH	0 (Primary)	1 (High)	1 (Output)	0 (Disabled)
P0.1	–	MISO0	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P0.2	–	MOSI0	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P0.3	–	SCK0	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P0.4	–	LFXT	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P0.5	–	HFXT	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P0.6	–	TRAP	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P0.7	BOOT	–	0 (Primary)	0 (Low)	0 (Input)	1 (Enabled)
P1.0	–	CS1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P1.1	–	MISO1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)

Table 2 continued on next page

Table 2 continued from previous page

Pin	Primary Function	Secondary Function	Reset Values			
			SEL	OUT	DIR	REN
P1.2	–	MOSI1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P1.3	–	SCK1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P1.4	–	TX0	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P1.5	–	RX0	1 (Secondary)	0 (Low)	0 (Input)	0 (Disabled)
P1.6	–	TX1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P1.7	–	RX1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.0	–	T0CMP0	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.1	–	T0CMP1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.2	–	T0CAP0	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.3	–	T0CAP1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.4	–	T1CMP0	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.5	–	T1CMP1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.6	–	T1CAP0	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P2.7	–	T1CAP1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.0	–	SDA0	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.1	–	SCL0	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.2	–	SDA1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.3	–	SCL1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.4	–	DTP0	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.5	–	DTP1	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.6	–	DTP2	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)
P3.7	–	DTP3	0 (Primary)	0 (Low)	0 (Input)	0 (Disabled)

3 Address Space

0x00000	ROM Size = 16 KiB	} RAM (Total size = 32 KiB)
0x03FFF		
0x04000	Peripherals Size = 4 KiB	
0x04FFF		
0x05000	<i>Unused</i>	
0x07FFF		
0x0C000	SRAM03 Size = 16 KiB	
0x0FFFF		
0x10000	SRAM04 Size = 16 KiB	
0x13FFF		
0x0014000	<i>Unused</i>	
0x0FFFFFF		
0x1000000	SPI Flash Native Memory Access (read only)	
0x1FFFFFF		

The 32-bit memory bus on the MCU is in the von Neumann architecture. All addressable memory elements (i.e. the ROM, RAM, and memory mapped registers) are all located on a single bus. A pointer can access any of these locations in memory.

The ROM cell contains boot code, which is read-only. Each SRAM cell can be read or written to, and may contain both program and variable information. The chip does not contain programmable non-volatile memory (e.g. flash memory), so the program must be entirely stored in RAM along with all runtime memory. Each SRAM and ROM cell can be individually powered off or on using the SYSTEM peripheral. When powered off, the cells use less power and cannot be read or written to. SRAM cells that are powered off do not retain their previous data, and the data becomes undefined.

The master interrupt handler begins at address 0x9000, which is called when any interrupt occurs. The master interrupt handler then determines which interrupt service routine (ISR) it needs to call and looks up the particular ISR address in the interrupt vector table (IVT), which begins at address 0x8000 and contains thirty-two 4-byte pointers to the ISRs.

4 Startup and Reset Behavior

To place the MCU in reset mode, drive the `resetsn` pin LOW (asserted). To take the MCU out of reset mode, let the `resetsn` pin go high (deasserted). The `resetsn` pin has an internal 47 k Ω pullup resistor which constantly attempts to pull the `resetsn` pin HIGH. If the user does not wish to use the `resetsn` pin, it is safe to leave it unconnected.

When the MCU is first powered up, a power-on reset (POR) circuit resets the MCU. As the core voltage, beginning at 0 V, begins to climb, the POR circuit does not assert the internal reset. Once the core voltage reaches a particular value (which is temperature and process dependent, but typically around 0.7 V), the POR circuit asserts the internal reset signal for about 20 ns and then deasserts it. If either the `resetsn` pin or the POR reset signal is asserted, the MCU will be in reset mode. In reset, the memory mapped registers and CPU state variables are all reset to their powerup values.

When the MCU comes out of reset, it immediately begins to execute the bootloader stored in the ROM, setting the program counter to address 0x0000. If the `BOOT` pin is LOW, the bootloader launches a Forth interpreter which communicates over UART0 at 115,200 baud (see Section 7 for details concerning the Forth interpreter). If the `BOOT` pin is HIGH, the bootloader reads and executes the program stored on an external SPI flash on SPI0 (see Section 9 for details on how to program the MCU). When the bootloader reads the program stored in the SPI flash, it dumps the contents of the SPI flash directly into RAM, completely filling the RAM from beginning to end using an address-to-address copy. After the bootloader has copied the program stored in the SPI flash into RAM, it jumps (moves the program counter) to address 0x8100 and begins executing the code.

5 Interrupts

Table 3: Interrupt Vectors

Priority	Interrupt Source
0	CPU Internal Timer
1	EBREAK, ECALL, or Illegal Instruction
2	Bus Error (unaligned memory access)
0	SYSTEM
1	GPIO0
9	SPI0
11	SPI1
13	UART0
16	TIMER0
22	TIMER1
28	GPIO1
36	GPIO2
44	GPIO3
52	UART1
55	AFE0
56	SARADC0
57	I2C0
70	I2C1

The CPU has 32 available interrupt slots, which may or may not be populated with an actual interrupt signal. Each interrupt slot can be individually masked to disable it using the custom `maskirq` assembly instruction (any bit that is set to 1 masks, or disables, the interrupt). Furthermore, the CPU can be instructed to wait until an interrupt occurs using the custom `waitirq` assembly instruction (note that the `waitirq` instruction does not cause the CPU to go to sleep; it merely causes the CPU to jump back to the `retirq` instruction repeatedly until an interrupt occurs).

In general, a peripheral will get into a state that asserts its interrupt signal, which immediately causes the CPU to jump (move the program counter) to address 0x9000, which must be the start address of the master interrupt handler. The master interrupt handler is responsible for determining which interrupt has occurred, choosing the correct interrupt service routine (ISR) to call, and calling the ISR by looking up its address in the interrupt vector table (IVT). The user is responsible for placing the master interrupt handler at the correct address (0x9000), and must program the master interrupt handler to do the following: save the 32 CPU registers to RAM, determine which interrupts are pending, call the appropriate ISR(s), load the CPU registers back from RAM, and return execution to its prior place by moving the program counter back to where it was before the interrupt occurred. Furthermore, the user is also responsible for placing the peripherals into a state such that their interrupt signals are no longer asserted while executing the ISRs. If the user fails to do so and the peripheral continues to assert its interrupt signal, then the CPU will jump back to the master interrupt handler immediately after the prior call to the master interrupt handler returns. This will most likely cause the MCU to get stuck executing the master interrupt handler endlessly without executing any of the main code.

If an interrupt occurs while the CPU is off (in sleep mode), then the CPU will turn on (exit sleep mode) until the custom `retirq` assembly instruction is executed, whereupon the CPU will turn back off (re-enter sleep mode) unless instructed otherwise. The CPU clock is disabled while the CPU is off (in sleep mode),

and the CPU registers and program counter retain their values. If the clock that sources MCLK (and by extension, the CPU clock) is powered down or disabled while the CPU is in sleep mode, then any interrupt will automatically power up and enable the source clock for the duration of the interrupt. The clocks and CPU on/off states can be controlled in the SYSTEM peripheral. Note that if the clock that sources SMCLK is disabled, peripherals may not be able to change the state of their interrupt signals.

Note that all interrupts are masked (disabled) when the MCU boots. In order to use/enable interrupts, you must unmask them with the custom `mask_irq` assembly instruction. Furthermore, the peripherals that generate the interrupts must also be configured to enable interrupts in order for their interrupts to occur.

Though it is not strictly required, it is recommended to create an interrupt vector table (IVT) that stores the 32-bit pointer (or memory address) to each ISR in order of priority. As there are 32 available interrupt slots, the IVT occupies 128 bytes of RAM. The automatic code generation tool generates the IVT by default at the very beginning of the RAM, though the user is not required to use it or populate it. If the IVT is used, the master interrupt handler should use it to call the appropriate ISR(s). The auto-generated linker script defines the locations for each element of the IVT, and the memory map C/C++ header file provides a macro to place a pointer to the ISR in the appropriate IVT element.

6 CPU Instructions and Assembly

The MCU has an on-chip 32-bit RISC-V CPU with the Integer, Multiplication, and Compressed [IMC] extensions. The RISC-V instruction set architecture (ISA) determines how the CPU deciphers binary machine code into instructions and what to do with each instruction. It also determines which CPU registers are used for what purpose.

6.1 CPU Registers

There are 32 CPU registers defined in the RISC-V ISA. Though the user can technically use any register for any purpose (except for `x0/zero`, which is read-only and is always equal to `0x00000000`), it is generally advisable to use each register for its intended purpose and follow the convention that the RISC-V ISA describes. The convention was created to create a standard use for each register so that assembly functions can be written that will be compatible with any RISC-V assembly program. The RISC-V compiler always follows this convention. Therefore, if the user also follows the convention, then the user's assembly code will be drop-in compatible with other compiled code (such as C code). If the user does not follow the convention, then it may cause many runtime errors.

Each register has a specific purpose and a designated “saver” that is allowed to write to it. The saver is either the caller (the code that calls a function) or the callee (the function that was called). This allows for communication between caller and callee such that arguments can be passed from caller to callee and returned data can be sent back from callee to caller. For example, the registers `a0-a7` are intended for function arguments and are written to by the caller. Registers `s0-s11` are preserved across function calls and are written to by the callee. Furthermore, registers `t0-t6` are temporary register that can be used for any purpose and are written to by the caller.

In assembly, registers can be referred to interchangeably by their register number or name (e.g. you can either type `x1` or `ra` to refer to the same register). Generally, one should only use the register names since they give hints as to the purpose of the register.

A CPU instruction can have up to two source register `rs1` and `rs2`, and up to one destination register `rd`. `rs1`, `rs2`, and `rd` can be any of the 32 CPU registers (e.g. `sp`, `t6`, `zero`, etc.). The source registers are used as “arguments” to the instruction and are not modified by the instruction, and the destination register is modified by the instruction as its output.

A list of all 32 CPU registers is provided in Table 4.

Table 4: CPU Registers

Register	Name	Description	Saver
<code>x0</code>	<code>zero</code>	Hard-wired zero (read-only)	N/A
<code>x1</code>	<code>ra</code>	Return Address	Caller
<code>x2</code>	<code>sp</code>	Stack Pointer	Callee
<code>x3</code>	<code>gp</code>	Global Pointer	–
<code>x4</code>	<code>tp</code>	Thread Pointer	–
<code>x5</code>	<code>t0</code>	Temporary/Alternate Link Register	Caller
<code>x6-x7</code>	<code>t1-t2</code>	Temporary Registers	Caller
<code>x8</code>	<code>s0/fp</code>	Saved Register/Frame Pointer	Callee
<code>x9</code>	<code>s1</code>	Saved Register	Callee
<code>x10-x11</code>	<code>a0-a1</code>	Function Arguments/Return Values	Caller
<code>x12-x17</code>	<code>a2-a7</code>	Function Arguments	Caller
<code>x18-x27</code>	<code>s2-s11</code>	Saved Registers	Callee

Table 4: CPU Registers

Register	Name	Description	Saver
x28-x31	t3-t6	Temporary Registers	Caller

6.2 Load/Store Memory Instructions

These instructions interface between the memory bus (containing RAM, ROM, and memory-mapped registers) and the CPU registers. They are used to load data from memory into a CPU register or store data from a CPU register into memory. The immediate parameter (i) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is added to the value of one of the registers. All load/store instructions take 5 MCLK cycles to execute.

- Load Signed Byte: `lb rd, I(rs1)` : loads the signed byte (8 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Unsigned Byte: `lbu rd, I(rs1)` : loads the unsigned byte (8 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Signed Half Word: `lh rd, I(rs1)` : loads the signed half word (16 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Unsigned Half Word: `lhu rd, I(rs1)` : loads the unsigned half word (16 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Load Word: `lw rd, I(rs1)` : loads the signed or unsigned word (32 bits) stored at memory location pointed to by (rs1 + I) into the destination CPU register rd
- Store byte: `sb rs1, I(rs2)` : stores the signed or unsigned byte (8 bits) from source CPU register 1 rs1 into the memory location pointed to by (rs2 + I)
- Store half word: `sh rs1, I(rs2)` : stores the signed or unsigned half word (16 bits) from source CPU register 1 rs1 into the memory location pointed to by (rs2 + I)
- Store word: `sh rs1, I(rs2)` : stores the signed or unsigned word (32 bits) from source CPU register 1 rs1 into the memory location pointed to by (rs2 + I)

6.3 Shift Instructions

These instructions perform bit shift operations (e.g. <<). The immediate parameter (I) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is used as the number of bits to shift. Each shift instruction takes 3 MCLK cycles to execute.

- Shift Left: `sll rd, rs1, rs2` : Logical shifts the value in rs1 left by rs2 bits and stores the result in rd.
- Shift Left Immediate: `slli rd, rs1, I` : Logical shifts the value in rs1 left by I bits and stores the result in rd.
- Shift Right: `sll rd, rs1, rs2` : Logical shifts the value in rs1 right by rs2 bits and stores the result in rd.

- Shift Right Immediate: `slli rd, rs1, I` : Logical shifts the value in `rs1` right by `I` bits and stores the result in `rd`.
- Shift Right Arithmetic: `sll rd, rs1, rs2` : Arithmetic shifts the value in `rs1` right by `rs2` bits and stores the result in `rd`.
- Shift Right Arithmetic Immediate: `slli rd, rs1, I` : Arithmetic shifts the value in `rs1` right by `I` bits and stores the result in `rd`.

6.4 Arithmetic Instructions

These instructions perform arithmetic operations. The immediate parameter (I) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the operation (except for in the `auipc` instruction). These instructions take 3 MCLK cycles.

- Addition: `add rd, rs1, rs2` : Adds the signed/unsigned integer values in `rs1` and `rs2` together and stores the sum in `rd`
- Addition Immediate: `addi rd, rs1, I` : Adds the signed/unsigned integer value in `rs1` with the immediate 12-bit signed integer `I` together and stores the sum in `rd`
- Subtraction: `sub rd, rs1, rs2` : Subtracts the signed/unsigned integer values in `rs1` from `rs2` and stores the result in `rd` (i.e. $rd \leftarrow rs1 - rs2$)
- Load Upper Immediate: `lui rd, I` : Loads the 20-bit immediate value `I` into the upper 20 bits of the destination CPU register `rd` (the lower 12 bits are all cleared to 0s)
- Add Upper Immediate to PC: `auipc rd, I` : Adds the 12-bit signed integer immediate `I` to the program counter to jump the program's execution to a location `I` away from the current program counter. Note that `I` can be negative.

6.5 Bitwise Instructions

These instructions perform bitwise operations. The immediate parameter (I) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 3 MCLK cycles.

- Bitwise exclusive OR: `xor rd, rs1, rs2` : Performs the bitwise XOR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise exclusive OR Immediate: `xori rd, rs1, I` : Performs the bitwise XOR of `rs1` and the immediate `I` and stores the result in `rd`.
- Bitwise OR: `or rd, rs1, rs2` : Performs the bitwise OR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise OR Immediate: `ori rd, rs1, I` : Performs the bitwise OR of `rs1` and the immediate `I` and stores the result in `rd`.
- Bitwise AND: `and rd, rs1, rs2` : Performs the bitwise AND of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise AND Immediate: `andi rd, rs1, I` : Performs the bitwise AND of `rs1` and the immediate `I` and stores the result in `rd`.

6.6 Comparison Instructions

These instructions perform comparison operations. The results of these instructions are either true (0x00000001) or false (0x00000000). The immediate parameter (I) refers to a hard-coded 12-bit signed or unsigned integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 3 MCLK cycles.

- Set when Less Than: `slt rd, rs1, rs2`: If $rs1 < rs2$ (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate: `slti rd, rs1, I`: If $rs1 < I$ (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Unsigned: `sltu rd, rs1, rs2`: If $rs1 < rs2$ (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate Unsigned: `sltiu rd, rs1, I`: If $rs1 < I$ (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.

6.7 Branch Instructions

These instructions are conditional instructions that branch (jump) the program execution to a new location if the condition is met, or proceed on to the next instruction like normal if the condition is not met. The immediate parameter (I) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the program counter if the condition is met. Usually, in assembly, the program writer can simply provide the *label* to the desired branch location/function in the program (e.g. `labelToFunction`;) rather than its actual memory location, which is usually not known before being built. The number of MCLK cycles it takes to execute these instructions is dependent on if the condition is met and the branch is taken or if it is not. If the condition is met and the branch is taken, they take 5 MCLK cycles. If not, they take 3 MCLK cycles.

- Branch if Equal: `beq rs1, rs2, I`: Branches to $PC + I$ if $rs1 = rs2$, proceeds to the next instruction like normal otherwise.
- Branch if Not Equal: `bne rs1, rs2, I`: Branches to $PC + I$ if $rs1 \neq rs2$, proceeds to the next instruction like normal otherwise.
- Branch if Less Than: `blt rs1, rs2, I`: Branches to $PC + I$ if $rs1 < rs2$ (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal: `bge rs1, rs2, I`: Branches to $PC + I$ if $rs1 \geq rs2$ (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Less Than Unsigned: `bltu rs1, rs2, I`: Branches to $PC + I$ if $rs1 < rs2$ (unsigned integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal Unsigned: `bgeu rs1, rs2, I`: Branches to $PC + I$ if $rs1 \geq rs2$ (unsigned integers), proceeds to the next instruction like normal otherwise.

6.8 Jump and Link Instructions

The jump and link instructions write the address of the next consecutive instruction in the program to the destination register, but then jump the program counter to another location in memory. These instructions are used to call new functions while preserving the “return address”, or the memory location of the instruction to go back to after the function returns. The immediate parameter (I) refers to a hard-coded 20-bit signed integer that is stored inside the CPU instruction itself and is used to modify the program counter. Usually, in assembly, the program writer can simply provide the *label* to the desired location/function in the program (e.g. `labelToFunction:`) rather than its actual memory location, which is usually not known before being built.

- Jump and Link: `jal rd, I` Stores the return address to `rd` (which is usually the return address register `ra`) and then adds `I` to the program counter, effectively jumping to `PC + I`. This takes 3 MCLK cycles to execute.
- Jump and Link Register: `jalr rd, rs1, I` Stores the return address to `rd` (which is usually the return address register `ra`) and then sets the program counter to `rs1 + I`, effectively jumping to `rd1 + I`. This takes 6 MCLK cycles to execute.

6.9 Pseudo-Instructions

The RISC-V assembler also includes several pseudo-instructions for convenience. These pseudo-instructions are actually special cases of regular instructions, and can all be expressed as regular instructions.

- No operation: `nop` → `addi zero, zero, 0`
- Move: `mv rd, rs1` → `addi rd, rs1, 0`
- Bitwise Complement/Inversion: `not rd, rs1` → `xor rd, rs1, -1`
- Set if Equal to Zero: `seqz rd, rs1` → `sltiu rd, rs1, 1`
- Set if Not Equal to Zero: `snez rd, rs1` → `sltu rd, zero, rs1`
- Set if Less Than Zero: `sltz rd, rs1` → `slt rd, rs1, zero`
- Set if Greater Than Zero: `sgtz rd, rs1` → `slt rd, zero, rs1`
- Jump (without linking): `j I` → `jal zero, I`
- Return: `ret` → `jalr zero, ra, 0`
- Call function: `call I` → `auipc t1 I; jalr ra t1 I`
- Load immediate: `li rd, I` (*this could compile to several instructions*)
- Branch if Greater Than: `bgt rs1, rs2, I` → `blt rs2, rs1, I`
- Branch if Less Than or Equal To: `ble rs1, rs2, I` → `bge rs2, rs1, I`
- Branch if Greater Than Unsigned: `bgtu rs1, rs2, I` → `bltu rs2, rs1, I`

- Branch if Less Than or Equal To Unsigned: `bleu rs1, rs2, I` → `bgeu rs2, rs1, I`
- Branch if Equal to Zero: `beqz rs1, I` → `beq rs1, zero, I`
- Branch if Not Equal to Zero: `bnez rs1, I` → `bne rs1, zero, I`
- Branch if Less Than or Equal To Zero: `blez rs1, I` → `bge zero, rs1, I`
- Branch if Greater Than or Equal To Zero: `bgez, rs1, I` → `bge rs1, zero, I`
- Branch if Less Than Zero: `bltz rs1, I` → `blt rs1, zero, I`
- Branch if Greater Than Zero: `bgtz rs1, I` → `blt zero, rs1, I`

7 Forth Interpreter

When the MCU comes out of reset and the BOOT pin is LOW, the MCU boots to an interactive Forth interpreter based on the Forth language. Forth is a stack-based language where variables are pushed to a stack in LIFO (last in, first out) order. When a variable is set, it is pushed to the top of the stack (TOS). When a variable is used, it is popped (or consumed) from the TOS. Functions that take arguments pop them from the TOS. Functions that have return values push them to the TOS (after first popping its arguments). Variables are not assigned names, and the only way of distinguishing them from one another is by their order on the stack. All variables are treated as 32-bit signed integers. Variables may be pushed to the stack through the command line by simply typing the variable value into the interpreter as a plain text integer value or as a plain text hexadecimal value. Hexadecimal values must be preceded by the string `0x` without spaces in between (e.g. `0xA4`), but the value may be upper or lower case and does not need leading zeros. Variables are not pushed or popped and functions are not executed until a newline (or line feed) `\n` character is received, at which point the entire received line will be executed in order. New functions may be declared, and may use the stack and any already defined functions to perform their procedures. If the Forth interpreter does not recognize an input as a variable or a function, it prints the `?` char over the UART.

The Forth interpreter contained in the ROM of the MCU communicates over UART0 at 2,048 baud. The baud may be changed by reconfiguring SMCLK and the UART0 peripheral. When the Forth interpreter first starts up, it prints the string `rv4th!\n>`, and is then ready to receive characters over the UART. The Forth interpreter does not perform error handling on stack underflows. In the case of a stack underflow, it simply uses the value 0 as the TOS. By default, the echo is enabled, which repeats all characters received by the Forth interpreter back to the host. This is useful for typing into a terminal emulator manually, but may be switched off with the command `0 echo`.

Several useful built-in Forth functions are listed below. The syntax is read as follows: The Forth function keyword is surrounded by quotations (though you do not actually type the quotes into the Forth interpreter), and is followed by the list of arguments and return values in parentheses. Left of the double dash is the list of arguments in the order that they would be typed into the Forth interpreter (reverse stack order), and right of the double dash is the list of return values in reverse stack order. Reverse stack order means that the rightmost item is at the TOS, and items to the left of it are below it on the stack. Arguments are always popped from the stack before the function executes, and return values are always pushed to the stack after the function finishes.

7.1 Memory Access Functions

- `@ ( addr -- )`

Read directly from the memory address and push the value to the TOS

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to read from (must be aligned to a 4-byte (32-bit) boundary)

Return values: none

- `! ( x addr -- )`

Write directly to memory address

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)

2. x: The value to write to the memory address

Return values: none

- **+**! (x addr --)

Add to value, direct memory access (*addr += x)

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. x: The value to add to the value at the memory address

Return values: none

- **sbi** (bitnum addr --)

Set bit, direct memory access (*addr |= (1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. bitnum: The bit number

Return values: none

- **cbi** (bitnum addr --)

Clear bit, direct memory access (*addr &= ~(1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. bitnum: The bit number

Return values: none

- **mask** (bitmask x addr --)

Change only masked bits, direct memory access (*addr = (*addr & ~bitmask) | (x & bitmask))

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. x: The value to write (only masked bits will be written)
3. bitmask: The bit mask. Only bits with 1s can be changed

Return values: none

7.2 Print and Input Functions

- **.** (x --)

Prints the TOS as an integer

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The value to print

Return values: none

- **h.** (x --)

Prints the TOS as a hex string

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The value to print

Return values: none

- **echo** (bool --)

Change the echo state, which controls whether the Forth interpreter repeats all of its received characters

Arguments: (in order from TOS down, reverse text order):

1. bool: (TOS) If 0, does not echo. Otherwise, it does

Return values: none

- **emit** (x --)

Prints the char at the TOS (integers are mapped to their ASCII counterparts)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The ASCII value of the char to print

Return values: none

- **key** (-- char)

Waits for a char over UART and pushes the ASCII value of the char to the TOS

Arguments: none

Return values: (in order from TOS down, reverse text order):

1. char: (TOS) The char that was received over UART

- **list** (--)

Prints all currently available functions

Arguments: none

Return values: none

7.3 Arithmetic Functions

- **+** (y x -- ret)

Adds two numbers (y + x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **-** (y x -- ret)

Subtracts two numbers (y - x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- ***** (y x -- retprodhi retprodlo)

Multiplies two numbers (y * x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. retprodlo: (TOS) Lower 32 bits of the return product
2. retprodhi: Upper 32 bits of the return product

- **/%** (y x -- retdiv retrem)

Divides two numbers with remainder (y / x and y % x)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. retrem: (TOS) The remainder (or modulo) result
2. retdiv: The integer division result

- **neg** (x -- ret)

Returns the negative (twos complement) of x

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **abs** (x -- ret)

Absolute value ($|x|$)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

7.4 Bitwise Logic Functions

- **~** (x -- ret)

Bitwise complement/inversion ($\sim x$)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **|** (y x -- ret)

Bitwise OR ($y | x$)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **&** (y x -- ret)

Bitwise AND ($y \& x$)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **^** (y x -- ret)

Bitwise XOR ($y \wedge x$)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `*2 ( x -- ret )`

Shift left 1 bit ($x \ll 1$)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `/2 ( x -- ret )`

Shift right 1 bit ($x \gg 1$)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `sll ( x bits -- ret )`

Shift left logical ($x \ll \text{bits}$)

Arguments: (in order from TOS down, reverse text order):

1. bits: (TOS) Signed integer
2. x: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- `srl ( x bits -- ret )`

Shift right logical ($x \gg \text{bits}$), no sign extension

Arguments: (in order from TOS down, reverse text order):

1. bits: (TOS) Signed integer
2. x: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

7.5 Comparison Functions

- **>** (y x -- ret)

Greater than. Returns 1 if $y > x$, returns 0 otherwise.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) 1 if true, 0 if false

- **<** (y x -- ret)

Less than. Returns 1 if $y < x$, returns 0 otherwise.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) 1 if true, 0 if false

- **==** (y x -- ret)

Equals. Returns 1 if $y == x$, returns 0 otherwise.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) 1 if true, 0 if false

7.6 Boolean Functions

- **not** (x -- ret)

Logical not (!x). Returns 1 if x is equal to 0, returns 0 otherwise

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **and** (y x -- ret)

Logical and (y && x). Returns 1 if y and x are both true (nonzero), returns 0 otherwise

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **or** (y x -- ret)

Logical or ($y \parallel x$). Returns 1 if y or x are true (nonzero), returns 0 otherwise

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer
2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

7.7 Stack Manipulation Functions

- **dup** (x -- x x)

Duplicates the TOS (pops the TOS and then pushes the value to the TOS twice)

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Value to duplicate

Return values: (in order from TOS down, reverse text order):

1. x: (TOS) The duplicated value
2. x: The original TOS

- **drop** (x --)

Pops the TOS and sets it as the internal Forth i value. Usually used to delete the TOS.

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) The value that is dropped

Return values: none

- **swap** (y x -- x y)

Swaps the TOS with the value below the TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Old TOS
2. y: Old value below the TOS

Return values: (in order from TOS down, reverse text order):

1. y: (TOS) Old value below the TOS
2. x: Old TOS

- **over** (y x -- y x y)

Copies the value below the TOS and pushes it to the TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed Integer
2. y: Signed Integer

Return values: (in order from TOS down, reverse text order):

1. y: (TOS) Signed Integer
2. x: Signed Integer
3. y: Signed Integer

- **tuck** (y x -- x y x)

Copies the TOS and inserts it two after the TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed Integer
2. y: Signed Integer

Return values: (in order from TOS down, reverse text order):

1. x: (TOS) Signed Integer
2. y: Signed Integer
3. x: Signed Integer

- **roll** (n --)

Removes value at index n of the stack and pushes it to the TOS (zero indexed)

Arguments: (in order from TOS down, reverse text order):

1. n: (TOS) Stack index (zero indexed)

Return values: none

- **pick** (n --)

Copies value at index n of the stack and pushes it to the TOS (zero indexed)

Arguments: (in order from TOS down, reverse text order):

1. n: (TOS) Stack index (zero indexed)

Return values: none

- **depth** (-- ret)

Gets size of stack and pushes it to the top of the stack (making the new stack size the value of TOS + 1)

Arguments: none

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

7.8 SPI Flash R/W and Programming Functions

- **fr** (bin length addr --)

Reads data from the SPI flash.

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address in the SPI flash to begin reading from
2. length: The number of bytes to read
3. bin: If true (nonzero), transmits binary data, otherwise transmits ASCII hex string

Return values: none

- **fe** (conf addr -- eraseSuccessful)

Erases a 256-byte page from the SPI flash memory. The start address must be 256-byte aligned.

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The start address of the page to erase in the SPI flash. Must be 256-byte aligned.
2. conf: If equal to 123, erases the page and returns 1, otherwise does not erase and returns 0

Return values: (in order from TOS down, reverse text order):

1. eraseSuccessful: (TOS) Nonzero if successful, zero if failed

- **fw** (bin addr --)

Writes data to the SPI flash. Must write exactly 256 bytes at a time. The start address must be 256-byte aligned.

Arguments: (in order from TOS down, reverse text order):

1. addr: (TOS) The address in the SPI flash to begin writing to. Must be 256-byte aligned.
2. bin: If true (nonzero), expects to receive binary data, otherwise expects to receive an ASCII hex string

Return values: none

7.9 Miscellaneous Functions

- **rst** (--)

Performs a soft reset (turns on all memory cells and jumps to address 0 in the ROM, does not reset clocks or hardware)

Arguments: none

Return values: none

- **clk** (meastime clkssel -- freq)

Measures selected clock frequency

Arguments: (in order from TOS down, reverse text order):

1. clkssel: (TOS) Clock select. 0 = SMCLK; 1 = MCLK; 2 = LFXT; 3 = HFXT

2. meastime: Measurement time. 0 = 1 s; 1 = 0.5 s; 2 = 0.25 s; 3 = 0.125 s

Return values: (in order from TOS down, reverse text order):

1. freq: (TOS) Selected clock frequency in Hz

- **min** (y x -- ret)

Returns minimum value of y and x

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **max** (y x -- ret)

Returns maximum value of y and x

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

2. y: Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **swpb** (x -- ret)

Swap bits 15 downto 8 with bits 7 downto 0 of TOS, also causes bits 31 downto 16 to all be 0

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

- **swphw** (x -- ret)

Swap upper 16 bits with lower 16 bits of TOS

Arguments: (in order from TOS down, reverse text order):

1. x: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. ret: (TOS) Return value

7.10 Creating New Forth Functions

To create a new Forth function on-the-fly, simply use the syntax `: funcname definition ;` where `funcname` is the name of the new function and `definition` is the body of the function. The new function may be called at any time after the semicolon (even before a new line, though it will not actually be executed until a new line) by typing the function name, just like all other Forth functions. New Forth functions can only add or remove variables from the stack and call other defined Forth functions. The arguments to custom Forth functions are the initial values on the stack that the custom function pops off the TOS. Likewise, the return values are any new values the custom function pushes to the TOS.

Listed below is an example program that increments the TOS by one when it is called:

```
: inc 1 + ;
```

7.11 Conditionals

The `if then` functions implement a conditional statement which may only be used inside a user defined function. The conditional is used with the syntax:

```
if procedureIfTrue then
```

inside a function (do not forget the space and semicolon which must come after `forthinlinethen`). The `if` function pops the TOS and then executes whatever is defined within `procedureIfTrue` if and only if the TOS is true (nonzero). The `then` function marks the end of the conditional procedure. For example, a function to print the text cool if the TOS is equal to 5 might look like this:

```
: cool? 5 == if 108 111 111 99 emit emit emit emit then ;
```

The `else` keyword may be inserted after `procedureIfTrue` if you want to execute something if the conditional is false. The syntax then becomes:

```
if procedureIfTrue else procedureIfFalse then
```

7.12 Loops

Forth has the equivalent of C-like while loops (or, more accurately, `do while` loops) with its `begin until` loops, which may only be executed inside a user defined function. The syntax is:

```
begin loopProcedure until
```

which first executes the loop procedure. When it gets to the `until` statement, it then checks if the TOS is true (nonzero) or false (zero). If the TOS is false, it loops back to the beginning of the loop. If true, it exits the loop and continues on.

Forth also has the equivalent of C-like for loops with its `do loop` functions. Again, this may only be used inside a user defined function. It uses the syntax:

```
stop start do loopProcedure loop
```

where `stop` is the index the loop stops on (it does NOT execute the loop procedure when the index is equal to `stop`), `start` is the first loop index, and `loopProcedure` is any functions or variables that you want to execute inside the loop. There is a special function `i` that may only be called from within a loop which pushes the loop index to the TOS. The functions `j` and `k` behave similarly, except they return the indices of the second and third nested loops respectively when using nested loops. The `i` function always returns the index of the deepest (or most frequent) loop.

Below is an example Forth program that prints the numbers from 0 to 9 using a `do loop` procedure:

```
: looptest 10 0 do i . loop ;
```


8 Software Development and Build Process

8.1 The RISC-V Toolchain and its Extensions

To compile software for the picorv32 RISC-V CPU, you will need the RISC-V GNU toolchain, which is an open source compiler suite that supports a variety of languages. The base toolchain, called RV32I, contains the fewest instructions and is in general the simplest. The “I” suffix stands for “integer”, meaning that the toolchain supports instructions that deal with basic integer arithmetic, excluding multiplication, division, and modulo instructions. All instructions in RV32I are 32 bits long, without exception. Two extensions for the compiler exist that may be supported by this picorv32 implementation: integer multiplication extension [M], and the compressed instruction set extension [C] (see Section 1 to see if this MCU supports the extensions). The integer multiplication extension adds multiply, divide, and modulo instructions to the instruction set, provided that the CPU includes hardware to perform such operations (again, see Section 1). If the CPU does not contain such hardware, then the [M] extension cannot be used. In that case, the compiler would infer software emulation of the instructions whenever multiplication, division, or modulo is needed, at the cost of computation time. The compressed instruction set extension adds duplicates of many of the instructions in RV32I, except the duplicates use only 16 bits rather than 32. According to the RISC-V ISA documentation, using the [C] extension will likely reduce program sizes by about 20% to 30%. Not all instructions can be compressed, however, and if those that are not compressed do not fall on a 4-byte aligned memory address, they will take an additional two clock cycles to execute.

8.2 Getting the RISC-V Toolchain on Linux

You can find the source code for the RISC-V toolchain at <https://github.com/riscv/riscv-gnu-toolchain>, which also contains instructions for building the toolchain. However, the website instructions do not give directions for including the compressed instruction set extension [C]. Therefore, it is suggested to follow the procedure below to download and build the RV32IMC toolchain. Once you have the toolchain with both extensions, you can switch off the [M] and/or [C] extensions with the `-march` compiler flag (more on that later). It is suggested that you use Ubuntu or Linux Mint, for which the software dependencies needed to build the toolchain can be fulfilled with:

```
sudo apt install autoconf automake autotools-dev curl libmpc-dev
libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf
libtool patchutils bc zlib1g-dev libexpat-dev
```

The instructions are as follows:

1. Switch to your home directory with

```
cd ~
```

2. Make a new directory with

```
mkdir riscv-download
```

and go into it with

```
cd riscv-download
```

3. Download the RV32IMC toolchain with

```
git clone --recursive https://github.com/riscv/riscv-gnu-toolchain riscv-gnu-toolchain
-rv32imc
```

4. Enter into the new directory with

```
cd riscv-gnu-toolchain-rv32imc
```

5. Create a build directory with

```
mkdir build
```

Enter into the new build directory with `cd build`

6. Configure the build with

```
../configure --with-arch=rv32imc --prefix=/opt/riscv/rv32imc
```

7. Begin the build process with

```
sudo make
```

This will take quite a while. If you wish to speed up the process, you can allocate more CPU cores to the build process by using `make -jn` instead of `make` where `n` can be replaced by the number of CPU cores you want to use to build the toolchain. Once the build process has finished successfully, the toolchain will be located at `/opt/riscv/rv32imc`

8. Add a permanent environment variable to your system that defines the location of the `riscv` directory (perhaps in your `.bashrc` script) with something like

```
export RISCVC=/opt/riscv
```

You may also wish to add the `/opt/riscv/rv32imc/bin` directory to your `PATH`, though this is optional.

9. You may now safely delete the `riscv-download` directory with

```
sudo rm -rf ~/riscv-download
```

8.3 Getting the RISC-V Toolchain on Windows

The RISC-V toolchain has no official support for Windows, but several options exist to get it working. For Windows 10 users, it is recommended to install the Windows Subsystem for Linux (WSL) in Ubuntu mode, which can be found in the Microsoft Store app. Then, in WSL, follow the installation instructions in Section 8.2. You must then use WSL to compile all software for the RISC-V MCU. If you have an older version of Windows, it is suggested that you use an Ubuntu or Linux Mint virtual machine to build the RISC-V toolchain and compile your MCU software.

8.4 Compilation

The RISC-V toolchain includes a GCC compiler. Thus, the compilation process is similar to other toolchains that use GCC. To compile a source file into an object file, use:

`riscv32-unknown-elf-gcc -c $CFLAGS srcFile -o objectFile.o` where `$CFLAGS` should be replaced by any compilation flags, `srcFile` is the source code file (C, C++, assembly, or other GCC-supported languages), and `objectFile.o` is the output object file. The compiler determines which language is used in a source file by the file extension (`.c` is for C, `.cpp` is for C++, `.S` is for assembly code which must be preprocessed, and `.s` is for assembly code which is not preprocessed).

There are several compilation flags that are suggested:

- `-march=rv32imc` Architecture flag. This specifies which extensions are enabled in the compilation process. You may only include “m” and/or “c” if your MCU implementation supports it.

- `-Os` Optimize for program size flag. Attempts to shrink the program size.
- `-O<number>` Optimization level. Lower numbers perform less optimization, higher include more.
- `-I <include_directory>` Include directory flag. Adds the specified directory to the list of directories that have header files in them. Any files in the include directory may be included with the `#include <filename.h>` preprocessor directive.
- `-g` GDB debugger enable flag. While the picorv32 core does not support hardware debugging, adding the `-g` flag to both the compiler and linker commands will allow the `.lst` file to show the assembly commands your code compiles into.

You must compile every source file that you wish to use in your project. If your project is in C or C++, it is suggested that you use the provided `start.S` assembly file to call your main function. This assembly file needs to have its first instruction at address `0x8100`, so it must be the first file to be given to the linker. You must have one and only one `int main()` function throughout all of your source files in a single project. For C++ projects, you need to start the main function with `extern "C" int main()` to prevent the C++ compiler from name-mangling the main function.

8.5 Linking

Once all source files have been compiled, they must be linked to produce the output program binary file, the `.elf` file. The linker assigns memory addresses to every variable and function. The MCU contains no writable non-volatile memory, so the program and the variables all share the RAM (with a notable exception for the bootloader and Forth interpreter stored on the ROM). The linker must be aware of the RAM and ROM start address and size, the interrupt vector table (IVT) start address and size (along with the address of each ISR pointer within the IVT), the first program instruction address that the bootloader jumps to after loading the program into RAM, and the address of the master interrupt handler.

Defining the aforementioned memory segments and addresses is done using a linker script. It is highly suggested to use the provided linker script `MCU.ld` and its include files and tweak it to suit your need.

To execute the linker, use:

```
riscv32-unknown-elf-gcc $LFLAGS $OBJECTS -o program.elf
```

where `$LFLAGS` is a list of linker flags, `$OBJECTS` is a list of all the compiled object files needed for the project, and `program.elf` is the output ELF binary containing the program.

There are several linker flags that are suggested:

- `-march=rv32imc` Architecture flag. This specifies which extensions are enabled in the compilation process. You may only include `m` and/or `c` if your MCU implementation supports it.
- `-flto` Link-time optimization flag. This instructs the linker to perform optimizations to the final binary.
- `-nostdlib` Disables the use of the standard library for linking, decreasing software bloat. However, using the flag precludes using standard library functions such as software emulation of integer multiplication and division, floating point numbers, initialization routines, memory allocation, etc. It is suggested that you only use this flag if you have a specific need to reduce the software size.
- `-T linker_script.ld` Path to the linker script
- `-Map mapfile.map` Path to the output map file, which contains the symbol table for your project with their addresses

- `-L linkerfiles_dir` Path to the linker scripts directory that contains any files that are included in the main linker script
- `-g` GDB debugger enable flag. While the picorv32 core does not support hardware debugging, adding the `-g` flag to both the compiler and linker commands will allow the `.lst` file to show the assembly commands your code compiles into.

8.6 Intel Hex File Generation

Once the final ELF file has been created from the linker, an Intel hex file can be created, which converts the binary program data into an easy-to-read ASCII file. Furthermore, the Python package `IntelHex` can read the file, allowing you to use Python to upload a new program to the SPI flash. A Python program that utilizes `IntelHex` to upload new programs, called `forthprog.py`, is provided.

8.7 A Note on Using C++

C++ generates some extra data sections that C and assembly do not use. In particular, it generates the `.eh_frame` section, which is used for exception handling. You may not need exception handling, in which case you can pass in the compiler flags:

```
-fno-exceptions -fno-unwind-tables -fno-asynchronous-unwind-tables
```

which will significantly reduce the size of `.eh_frame`. If you wish to eliminate it entirely, you can create a phony section in the linker script at an unused memory address and point the `.eh_frame` section to it.

9 Uploading Program to SPI Flash

9.1 SPI Flash Boot Process

In order to boot in SPI flash mode, there must first be a valid program stored in the SPI flash memory in the proper location. In SPI flash mode, the bootloader reads the data in the SPI flash memory starting at address 0x0000 and copies every 32-bit word into RAM, also starting at address 0x8000. This particular address is the beginning of the RAM on the MCU. The bootloader stops copying from the SPI flash once completely fills the RAM.

Note the distinction between copying 32-bit words as opposed to 8-bit bytes. Though bytes and words are both MSB-first, they are also transmitted starting with the lowest address to the greatest address from the SPI flash. If the bootloader copied every 8-bit byte, the transmission sequence it would expect would be:

byte0, byte1, byte2, byte3, byte4, byte5, byte6, byte7 ...

However, because the bootloader expects MSB-first 32-bit words, the sequence must be:

byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...

Therefore, the program data on the SPI flash must follow the 32-bit word sequence.

9.2 Upload Process

The Forth boot mode has a mechanism for writing data to the SPI flash, which may be used to upload a new program for use in the SPI flash boot mode. When in Forth boot mode, the flash write `fw` function writes a page of data (which is exactly 256 bytes long and begins on an address that is a multiple of 256) to the SPI flash. The Forth function is used with the syntax:

```
bin addr fw
```

If `bin` is true (nonzero), the command expects 256 bytes of binary data to be transmitted over the UART. If false, it expects a 512-character long ASCII hex string representing the data to be transmitted over the UART. The `addr` argument must be the address of the beginning of the 256-byte page, and must be aligned to a 256-byte boundary.

To execute the command, a newline (or line feed) character must be transmitted to the MCU over the UART. Once the command begins executing, it initiates some handshaking to ensure a proper transaction. The MCU prints a `$` character to indicate it is ready for the page data to be transmitted. After that, you must transmit the page data in the format specified by the `bin` argument. Remember to transmit the data in the proper order (byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...). Once the page data has been transmitted, the MCU will compute a CRC value upon the data using the CRC16_CDMA2000 algorithm (polynomial = 0xC857, initial CRC value = 0xFFFF, no final XOR, no data reflection, no output reflection) and then transmit the CRC value as a 4-character long hex string. If this value matches your own CRC computation (or if you do not care and simply want it to write the data), transmit a single `Y` character without anything else to tell the MCU to write the data to the SPI flash. Any other character will abort the process, leaving the SPI flash memory unchanged. You are technically supposed to wait an average of 10 ms (25 ms at the greatest) before writing more data to the SPI flash. However, the substantial handshaking and transmission times likely fulfill this requirement, and it is likely that no delay is needed.

If the entire page is blank, you may instead use the flash erase `fe` function to erase the page, making every byte in the page equal to 0xFF. This is much quicker than writing data to the SPI flash, and can speed up programming times substantially for smaller programs. The syntax to use it is:

```
conf addr fe
```

If the `conf` parameter is equal to 123, the function will erase the 256-byte page at address `addr` (which must be 256-byte aligned) and then return 1. If not equal to 123, it will not erase the page and will return 0. You are technically supposed to wait an average of 6 ms (25 ms at the greatest) before another erase

or write operation. Because the erase function is much quicker than the write function, it may become necessary to delay between erases at higher bauds.

You may read back the data stored on the SPI flash with the `fr` function. See Section 7.8 for details.

In order to write an entire program to the SPI flash, you must write/erase a total of 128 pages (each with a length of 256 bytes) beginning with page address 0x8000. After the data is written, you may execute the newly uploaded program by putting the MCU in reset, setting the BOOT pin to SPI flash mode, and taking the chip out of reset. Or, you can simply set the BOOT pin and move the program counter to address 0x0000, which may be done with the `rst` Forth function.

It is highly suggested to use the provided Python program `forthprog.py` to upload new programs to the SPI flash, which takes an Intel hex file containing the new program and sends it to the MCU and the SPI flash using the process described above.

10 GPIOx Peripheral

The MCU has several GPIOx peripherals, each of which control up to 8 general purpose input/output (GPIO) pins. Each GPIO peripheral (typically called a port) constitutes a parallel port, allowing all 8 pins in the port to be updated simultaneously. Each GPIO pin may also be configured and updated individually, allowing for design flexibility. Each GPIO pin is referred to as Px.y, where x refers to the port number and y refers to the corresponding bit in the GPIO registers that controls the pin. For example, P2.5 refers to port 2, bit 5.

Every GPIO pin on the MCU can be in one of two modes: primary/GPIO mode, or secondary/peripheral mode. Section 2 details the secondary function of each GPIO pin.

10.1 Primary/GPIO Mode Functionality

In primary/GPIO mode, the GPIO pin state is controlled by the GPIO registers PxDIR, PxOUT, and PxREN. The PxDIR register determines if the pin is in input or output mode, the PxOUT register determines if the pin outputs a logic high or logic low level when the pin is in output mode, and the PxREN register enables or disables the internal pullup resistor when in input mode. Table 5 shows the various states available to every GPIO pin.

PxDIR[y]	PxOUT[y]	PxREN[y]	Px.y State
0	-	0	High impedance input, pullup resistor disabled
0	-	1	Pullup resistor enabled, weakly pulled high
1	0	-	Output, strongly driven low
1	1	-	Output, strongly driven high

Table 5: GPIO Configurations

The PxIN register always reads the digital state of the pin, regardless of if it is in input or output mode, and regardless of if it is in primary/GPIO mode or secondary/peripheral mode. The pin Px.y is a logic low level if PxIN[y] is a 0, and is a logic high level if it is a 1.

Figure 2 depicts the interaction between the registers and the GPIO pin.

10.2 Secondary/Peripheral Mode Functionality

Most GPIO pins have a secondary/peripheral function which is controlled by one of the peripherals in the MCU. The mode of the GPIO is determined by the PxSEL register. If PxSEL[y] is 0, Px.y is in primary/GPIO mode, and if it is 1, Px.y is in secondary/peripheral mode.

In most cases, when in peripheral mode, the peripheral seizes control over the pin's DIR, OUT, and REN controls, overriding PxDIR[y], PxOUT[y], and PxREN[y]. For example, when the SCK0 pin is in peripheral mode, the SPI0 peripheral controls the state of the pin, and the corresponding PxDIR[y], PxOUT[y], and PxREN[y] registers have no effect (until PxSEL[y] becomes 0, of course). However, some pins with secondary/peripheral functions are never seized by the corresponding peripheral, even when in secondary/peripheral mode. In yet others, several but not all of the controls are seized by the peripheral. Considering the T0CAP0 pin, which is only used by the TIMER0 peripheral as an input, the peripheral does not control the state of the pin at all, and thus it must be manually set as a high impedance input before using it for timing purposes. However, for the SDA0 pin, the I2C0 peripheral only seizes control over the DIR and OUT controls, and allows the REN control to be manually configurable by PxREN[y] at all times.

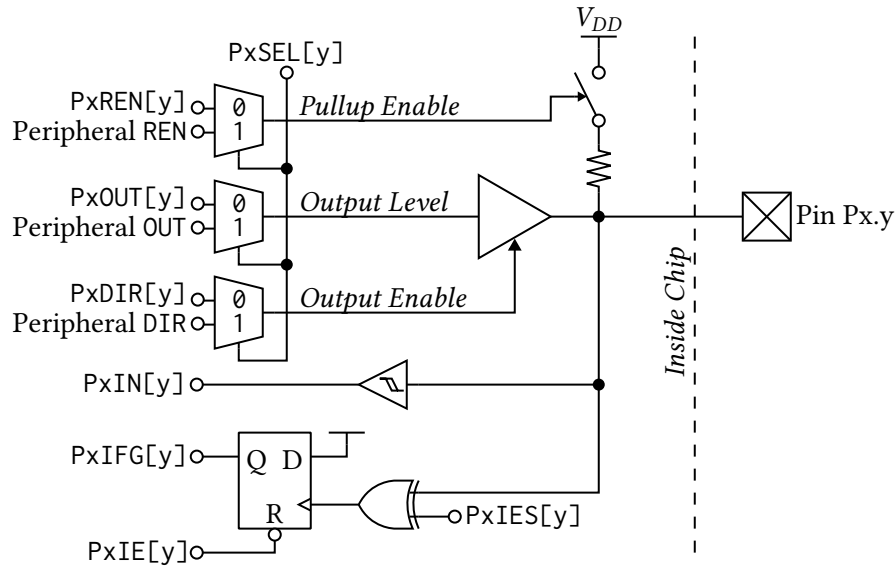


Figure 2: GPIO Pin Diagram

10.3 Pin Interrupts

Each GPIO port has a hardware interrupt that can be triggered by any combination of the pins in the port. To enable the interrupt, the GPIOx interrupt must be unmasked and the desired pin(s) that are allowed to trigger the interrupt must be set as 1s in the PxIE register.

The interrupt edge select register PxIES determines which edge will trigger the interrupt on the corresponding pin. The interrupt triggers on a rising edge if PxIES[y] is 0, and triggers on a falling edge if 1. Note that modifying the value of PxIES[y] while PxIE[y] is 1 could immediately generate an interrupt, so the interrupt should be disabled before changing PxIES[y].

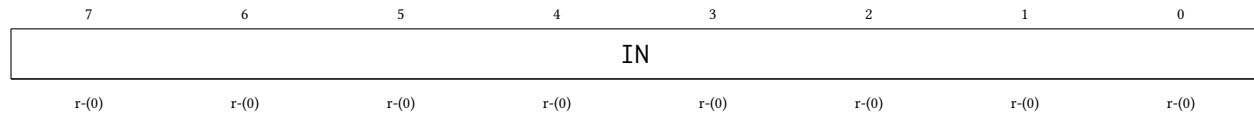
When the pin Px.y has its interrupt enabled and experiences the edge selected by PxIES[y], then the interrupt flag PxIFG[y] is immediately set to 1 and the GPIOx ISR is executed. There is one ISR for all of the pins contained in GPIOx, but the PxIFG[y] can be used to determine which pin(s) generated the interrupt. Note that PxIFG[y] must be cleared before the ISR returns, else the ISR will mistakenly re-execute immediately upon the prior return.

10.4 Register Description

10.4.1 PIN Register

Register memory slot: 0, offset: 0 bytes, size: 1 byte

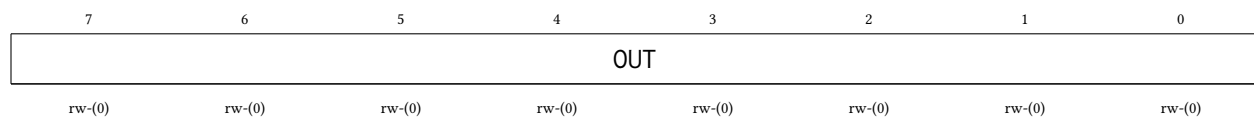
GPIO read pin register. Each bit corresponds to the input logic state of the GPIO pin of the same number at the time of the memory access reading this register (not synchronized to any clock). Reading a 0 in a bit indicates a logic low pin state; reading a 1 indicates a logic high state.



10.4.2 POUT Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

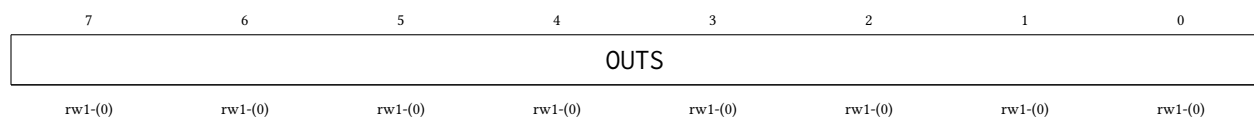
GPIO output drive register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is set to GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to make the corresponding pin output a logic low value; write a 1 to output a logic high value.



10.4.3 POUTS Register

Register memory slot: 2, offset: 8 bytes, size: 1 byte

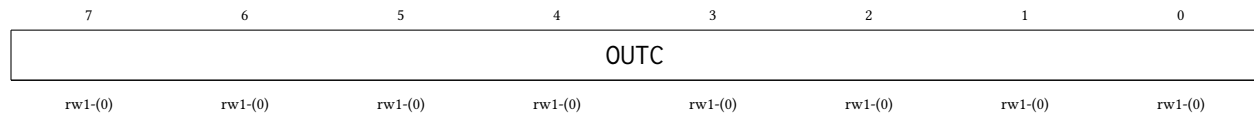
GPIO output drive set register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to set the corresponding pin (make the pin output a logic high value). Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.



10.4.4 POUTC Register

Register memory slot: 3, offset: 12 bytes, size: 1 byte

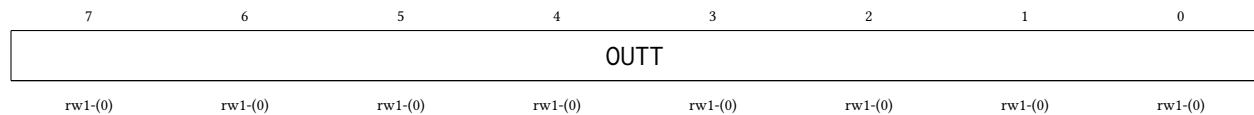
GPIO output drive clear register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to clear the corresponding pin (make the pin output a logic low value). Writing a 0 has no effect. Reading this register yields the inversion of the output drive register PxOUT.



10.4.5 POUTT Register

Register memory slot: 4, offset: 16 bytes, size: 1 byte

GPIO output drive toggle register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to toggle the corresponding pin (make the pin output a logic low value). Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.



10.4.6 PDIR Register

Register memory slot: 5, offset: 20 bytes, size: 1 byte

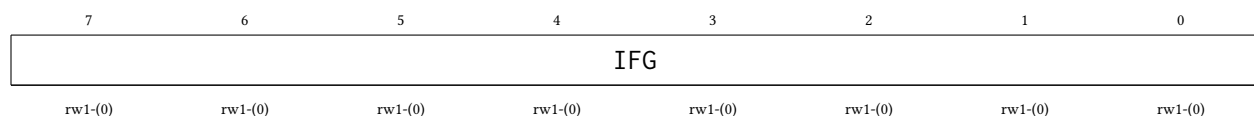
GPIO pin direction register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to set the corresponding pin to input mode; write a 1 to set to output mode.



10.4.7 PIFG Register

Register memory slot: 6, offset: 24 bytes, size: 1 byte

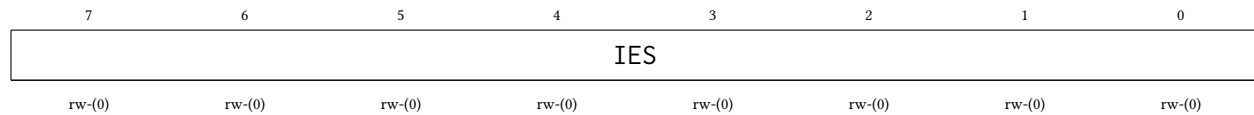
GPIO interrupt flag register. Each bit corresponds to the GPIO pin of the same number. Reading a 0 in a bit indicates there is no pending interrupt for the corresponding pin; reading a 1 indicates there is a new interrupt pending for the corresponding pin. Write a 1 to each bit for which you wish to clear the interrupt flag.



10.4.8 PIES Register

Register memory slot: 7, offset: 28 bytes, size: 1 byte

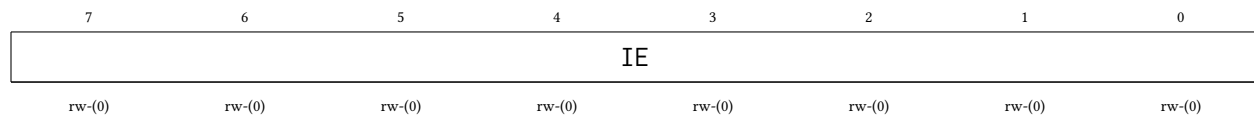
GPIO interrupt edge select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin interrupt rising edge triggering; write a 1 to set to falling edge triggering.



10.4.9 PIE Register

Register memory slot: 8, offset: 32 bytes, size: 1 byte

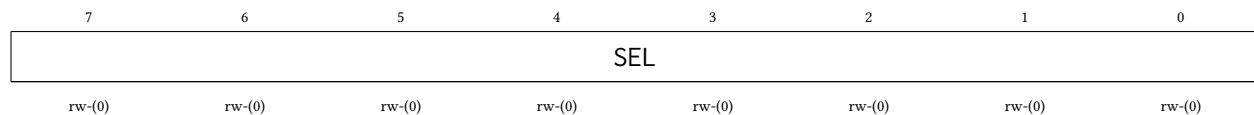
GPIO interrupt enable register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to disable the pin interrupt; write a 1 to enable the pin interrupt.



10.4.10 PSEL Register

Register memory slot: 9, offset: 36 bytes, size: 1 byte

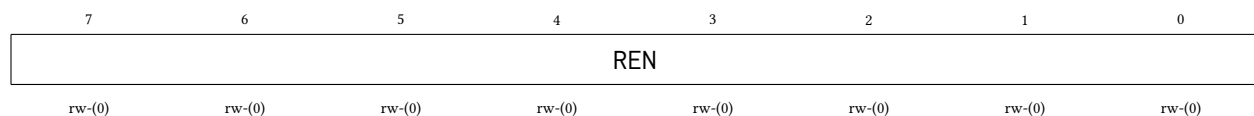
GPIO peripheral select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin to GPIO (primary) mode; write a 1 to set the pin to secondary function (peripheral) mode. When a pin is in secondary function (peripheral) mode, the governing peripheral takes control of the pin output, direction, resistor enable, and open-collector enable states, and the PxOUT, PxDIR, and PxREN have no effect on the pin. Note that pin interrupts are still available when in secondary function (peripheral) mode in addition to any interrupts the secondary function/peripheral may generate. Also note that not all pins necessarily have a secondary function; in this case, if the corresponding bit in PxSEL is set to 1, the pin will be configured as an input with the pullup resistor disabled and the open-collector mode disabled.



10.4.11 PREN Register

Register memory slot: 10, offset: 40 bytes, size: 1 byte

GPIO resistor enable register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to disable the pin pullup resistor; write a 1 to enable the pin pullup resistor.



11 SPIx Peripheral

The Serial Peripheral Interface (SPI) peripheral is a high speed, full-duplex, synchronous, multipoint serial interface. It transfers one bit at a time. The SPI bus requires three wires (SCK, MOSI, and MISO), along with one CS wire for every slave. The main features of the SPI peripheral are listed below:

- Master or slave mode (except SPI0, which is master only)
- LSB-first or MSB-first transfers
- 8-, 16-, or 32-bit transfer lengths
- Master mode supports SPI modes 0-3
- Slave mode supports SPI modes 1 and 3
- Optional byte swap for multi-byte transfers with systems of different endiannesses
- Programmable transfer rate
- Continuous transfer operation to reduce dead time
- Interrupts for transfer complete and TX register empty
- Native memory read access for SPI flash chips (on SPI0 only)

11.1 SPI Pins

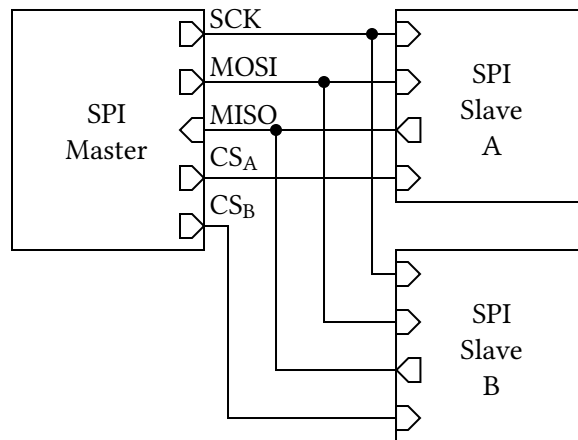


Figure 3: SPI Connection Diagram

The SPI bus has four pins: SCKx, MOSIx, MISOx, and CSx (not present on SPI0). A connection diagram for the SPI bus is shown in Figure 3.

SCKx is the serial clock pin, which governs when each data bit is both latched and updated. It must be attached to the SCK pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of SCKx.

MOSIx is the master out slave in pin, which transfers each data bit from the master to the slave. It is an output when the SPI port is enabled and in master mode, otherwise it is a high impedance input. It

must be attached to the MOSI pin on every other device that shares the SPI bus. When the corresponding $PxSEL[y]$ bit is a 1, the $SPIx$ peripheral seizes the DIR, OUT, and REN controls of $MOSIx$.

MISOx is the master in slave out pin, which transfers each data bit from the slave to the master. It is an output when the SPI port is enabled, in slave mode, and the CSx pin is asserted (low), otherwise it is a high impedance input. It must be attached to the MISO pin on every other device that shares the SPI bus. When the corresponding $PxSEL[y]$ bit is a 1, the $SPIx$ peripheral seizes the DIR, OUT, and REN controls of MISOx.

CSx is the chip select pin for the $SPIx$ peripheral on this chip. It is a high impedance input that waits to be asserted (pulled low) by an external master if this $SPIx$ peripheral is in slave mode, which begins the SPI transfer. One CS wire needs to be connected between a GPIO pin of the master and the CS pin of the slave for every slave on the bus. Each slave CS wire must be attached to a dedicated GPIO pin on the master in a one-to-one fashion. The $SPIx$ peripheral never seizes control of the CSx pin (regardless of the state of the corresponding $PxSEL[y]$ bit), and as such it must be configured as a high impedance input manually.

11.2 Generalized SPI Transfer Process

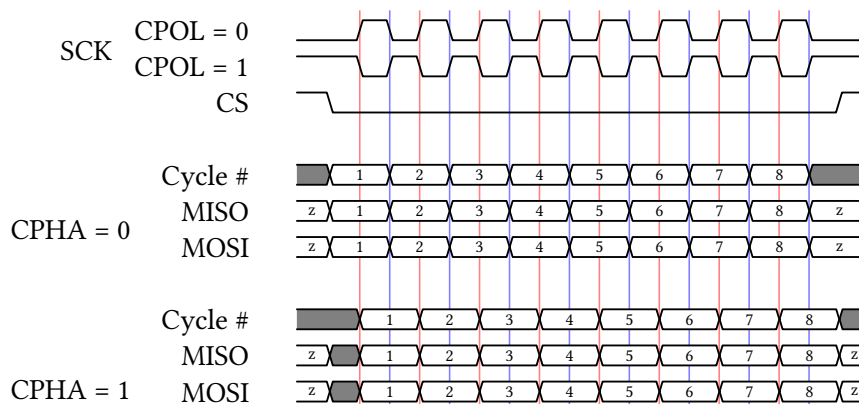


Figure 4: SPI Timing Diagram

When no transfer is occurring, the SPI bus is idle. The transfer begins when the master asserts (pulls low) the CS wire attached to the desired slave. Immediately, the master seizes control of the MOSI wire by making its MOSI pin an output, and the slave does the same for the MISO wire. Note that the master always has control of the SCK and CS wires, which are configured as outputs. Then, the master issues SCK clock pulses in increments of 8, 16, or 32 (one frame of data). On every transition of SCK, both the master and slave alternate between updating and latching the values of MOSI and MISO, the master updating MOSI or latching MISO, and the slave updating MISO or latching MOSI. When the wire is updated, the next bit of data is pushed out of the TX shift register and written to the wire. When the wire is latched, that bit is read from the wire and pushed into the RX shift register. As the SPI port is full-duplex, the master and slave both send data to the other simultaneously. After each frame of data has been transferred, both the master and slave RX shift registers contain their respective received words (the 8-, 16-, or 32-bit data transmitted in the frame). At that point, the master may either continue the process by transferring another frame, or cease the transfer by deasserting (pulling high) the CS wire. The frames that the master issues while the slave's CS wire is low is called a packet, and the master may issue as many frames in a packet as it wants before deasserting CS.

The order of the update/latch process, as well as the polarity of the SCK wire, are determined by two 1-bit parameters: clock polarity (CPOL) and clock phase (CPHA). CPOL determines the idle state and

transition direction of SCK, while CPHA determines when the data in MOSI and MISO is updated and latched. These parameters together are referred to as the SPI mode, or SPIMODE, shown in Table 6. Note that this SPI peripheral supports all four SPI modes while in master mode, but only supports SPI modes 1 and 3 while in slave mode (CPHA = 1). Figure 4 shows a SPI timing diagram for all four SPI modes, with a single-frame packet with 8-bit frames.

SPIMODE	CPOL	CPHA	SCK Idle	Data Update Edge	Data Latch Edge
0	0	0	Low	Trailing/Falling	Leading/Rising
1	0	1	Low	Leading/Rising	Trailing/Falling
2	1	0	High	Trailing/Rising	Leading/Falling
3	1	1	High	Leading/Falling	Trailing/Rising

Table 6: SPIMODE Key

Note that during the very first frame of a transfer, the slave is required to send data to the master on the MISO wire, regardless of if it has any valid data to send. If it has no data to send, a slave will issue a “dummy” word, which the master must ignore.

11.3 Bit and Byte Ordering

The SPI peripheral has the ability to transfer the most significant bit (MSB) first or the least significant bit (LSB) first using SPIMSB. When SPIMSB is 0, the LSB is transferred first, and when SPIMSB is 1, the MSB is transferred first.

For 16- and 32-bit transfers, the byte ordering may optionally be swapped using SPITXSB and SPIRXSB, which swap the bytes being transmitted and received, respectively. For example, during a 16-bit transfer, if SPITXSB is enabled, then byte 1 of SPITX will be transmitted followed by byte 0. If SPIRXSB is enabled, then the first byte received will be placed in byte 1 of SPIRX, and the next 8 bits received will be placed in byte 0. In a 32-bit transfer, the byte order would thus be 3, 2, 1, 0. Figure 5 illustrates this ordering.

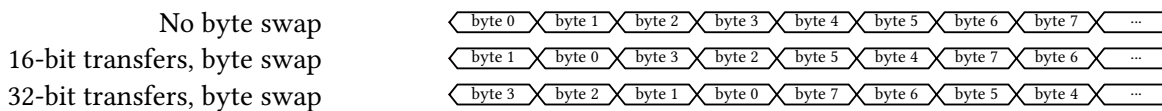


Figure 5: SPI Byte Ordering

Note that the bits in the frame are ordered MSB-first or LSB-first *before* the byte swap operation takes place. As such, a 16-bit, MSB-first transfer with byte swapping activated would transmit the bits 7 down to 0, and then 15 down to 8. Figure 6 shows the bit ordering for a 16-bit transfer.

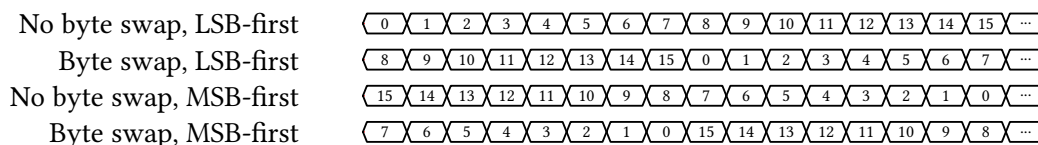


Figure 6: SPI Bit Ordering (16-bit transfer)

11.4 SPI Baud Clock

When in master mode, the frequency of the SCKx clock pin is determined by the frequency of SMCLK and the SPI baud control SPIBR, given by the expression:

$$f_{\text{SCK}} = \frac{f_{\text{SMCLK}}}{2 \times (1 + \text{SPIBR})}$$

Note that the maximum frequency of SCKx is half that of SMCLK.

11.5 Transmit Buffer Register Queue

Writing to the SPI transmit buffer register SPIxTX queues up the word written to it as the next frame to transmit. If in master mode and a transfer is not currently ongoing (i.e., SCKx is not clocking), the SPI peripheral immediately begins transmitting the word written to SPIxTX, and SPITEIF is set immediately. If a transfer is currently ongoing, the SPI peripheral waits for the current frame to finish transferring. As soon as it finishes, the SPI peripheral begins transmitting the new frame written to SPIxTX and sets SPITEIF. If in slave mode, the SPI peripheral latches and begins transferring the last word written to SPIxTX and sets SPITEIF whenever the master starts the first clock cycle of SCKx.

The TX queue allows the SPI peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the SPIxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the SPITEIF flag to be 1 or SPIBUSY to be 0 before writing to SPIxTX.

11.6 Flags and Interrupts

The SPI peripheral has two interrupt flags (SPITCIF and SPITEIF) and one indicator (SPIBUSY). If the corresponding interrupts are enabled in the SPIxCR register and the SPIx ISR is not masked, then the two interrupt flags will generate an interrupt whenever they are equal to 1. Both interrupt flags can be cleared by writing a 0 to them, which must be done before the SPIx ISR returns, else the ISR will mistakenly re-execute immediately upon the prior return.

The SPITCIF interrupt flag is set the moment a SPI transfer is completed, and can only be cleared by writing a 0 to it.

The SPITEIF interrupt flag is set the moment the SPI TX transmit buffer register SPIxTX becomes empty (i.e., is ready to queue up the next word to be transmitted). In master mode, if a frame is not currently being transferred, it is set immediately when a new word is written to SPIxTX. If a frame is currently being transferred, it is set as soon as the current transfer is completed, whereupon the value in SPIxTX begins to be transmitted. In slave mode, SPITEIF is set when the master starts the first clock cycle of SCKx.

The SPIBUSY indicator simply indicates if the SPI is busy or not. In master mode, it is 1 if a frame is currently being transferred (i.e. SCKx is actively clocking), and 0 if not. Note that SPIBUSY can therefore be 0 in between frame transfers if a new word to transmit has not been queued before the prior frame finishes transferring. In slave mode, SPIBUSY is 1 if the CSx pin is asserted (driven low), and is 0 if not.

11.7 Master Mode Transfer Procedure

To use the SPI peripheral in master mode, the user must first initialize the SPIx peripheral by configuring the SPIxCR register. Of note, SPIEN must be 1 and SPISM must be 0 to enable the SPI peripheral in master mode. Then, the GPIO pins multiplexed with SCKx, MISOx, and MOSIx must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the GPIO pins attached to each slave's CS

pin need to be set to output a logic high voltage (note that these CS pins are *not* the same as this SPIx peripheral's own CSx pin, which is used when this device is in slave mode).

To initiate a SPI transfer, the master should first clear the SPI status register SPIxSR by writing 0 to it, and then must assert (pull low) the slave's CS pin. Then, the master may begin transferring as many frames as it wants in the packet. To send a frame, the master must first check if the SPIxTX register is empty (i.e. ready to accept a new word to queue for transfer) by ensuring that either SPITEIF is 1 or SPIBUSY is 0. Then, the master may write the next word to transmit to SPIxTX. Next, if SPITCIF is 1, the master should read the value of the SPIxRX register to read the value the slave transmitted on the prior frame. Then, the master must clear the status register and may either start another frame or end the packet by deasserting (pulling high) the CS wire.

A simplified procedure for producing master mode SPI transfers is shown in Listing 1 that does not use the TX queue for continuous transfers.

```

1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Declarations **/
5  uint32_t spi_transfer(uint32_t tx_data)
6
7  /** Main Function **/
8  int main()
9  {
10     // Initialize the SPI peripheral
11     SPIxCR = SPIEN;
12
13     // Configure the SCKx, MOSIx, and MISOx pins
14     SCKx_PxSEL |= SCKx_BIT;
15     MOSIx_PxSEL |= MOSIx_BIT;
16     MISOx_PxSEL |= MISOx_BIT;
17
18     // Configure the GPIO pin attached to the slave's CS pin as an output high
19     PxSEL &= ~(BITy);
20     PxOUT |= BITy;
21     PxDIR |= BITy;
22
23     // Assert the CS pin
24     PxOUT &= ~(BITy);
25
26     // Do a single SPI transfer
27     uint32_t tx_data = 57;
28     uint32_t rx_data;
29
30     rx_data = spi_transfer(tx_data);
31
32     // Deassert the CS pin
33     PxOUT |= BITy;
34
35     return 0;
36 }
37
38 /** Function Definitions **/
39 uint32_t spi_transfer(uint32_t tx_data)
40 {
41     // Begin a SPI frame by writing the transmit data to SPIxTX
42     SPIxTX = tx_data;
43
44     // Wait for the frame to finish transmitting
45     while (SPIxSR & SPIBUSY) {}
46
47     // Return the received data from the slave
48     return SPIxRX;
49 }

```

Listing 1: Example Program for SPI Master Transfers

11.8 Slave Mode Transfer Procedure

To use the SPI peripheral in slave mode, the user must first initialize the SPIx peripheral by configuring the SPIxCR register. Of note, SPIEN must be 1 and SPISM must be 1 to enable the SPI peripheral in slave mode. Then, the GPIO pins multiplexed with SCKx, MISOx, and MOSIx must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the CSx pin must be set as a high impedance input in GPIO mode to allow the master to assert this SPIx peripheral.

Immediately after initialization, a word should be written to the SPIxTX register, which will be transmitted to the master when the master asserts the CSx pin and begins sending SCK pulses. Whatever data is contained in SPIxTX will always be transmitted to the master when the master begins each frame regardless of if the slave has updated the value of SPIxTX, so care must be taken to always put valid data in the SPIxTX register before the master begins the next frame. Once the master begins sending a frame, the SPITEIF flag will become 1, indicating that the next word to transmit to the master should be written to SPIxTX. At the end of the frame, the SPITCIF flag will become 1, and the data in SPIxRX will contain the data received from the master. SPIxRX should then be read, and the status register SPIxSR must be cleared. Note that the SPIxRX register must be read before the end of the next frame, else the data will be overwritten and unrecoverable.

There is no dedicated interrupt indicating when the CSx pin is asserted or deasserted, but this functionality can be achieved using the ordinary GPIO pin interrupts.

11.9 External SPI Flash Native Memory Read Access

The SPI0 peripheral, which is attached to the external SPI flash memory containing the program, has a native memory access feature to quickly read data on the SPI flash as if it were addressable by the MCU memory bus. In other words, a pointer can be used in software to read the SPI flash memory as if it were a block of ROM on the address bus. This feature is intended to expand the maximum size of a program from the size of the internal RAM to the size of the SPI flash memory, or 8 MB. Note that using this feature will yield a significant reduction in program execution speed, as each instruction fetched from the SPI flash memory takes either 32 or 64 SCK pulses to read. This feature supports the AT45DB SPI flash memories with either 256- or 264-byte pages, up to the 64 megabit (8 MB) model.

To use the native SPI flash memory access feature, the SPI flash must first be initialized:

1. Set up the SPI0 peripheral using master mode, SPI MODE3, 8-bit transfers, an SCK frequency not exceeding 85 MHz, swapped RX bytes, not swapped TX bytes, and the native SPI flash memory access feature disabled (SPIFEN = 0)
2. Configure the SCK0, MOSI0, and MISO0 pins in secondary/peripheral mode with the appropriate PxSEL registers
3. Configure the CS_FLASH pin in GPIO mode, outputting high (deasserted)
4. Assert (pull low) the CS wire for the SPI flash
5. Send the 8-bit wake from deep sleep command, 0xAB, to the SPI flash
6. Deassert (pull high) the CS wire and delay for at least 10 ms
7. Assert (pull low) the CS wire and set the SPI0 peripheral to 32-bit transfers
8. Send the 32-bit command setting the page length of the SPI flash to 256 bytes: 0x3D2A7F9A
9. Deassert (pull high) the CS wire

10. Set the CS_FLASH pin to secondary/peripheral mode with the appropriate PxSEL register
11. Enable the native SPI flash memory access feature by setting SPIFEN to 1

After completing the initialization procedure, any 32-bit word stored in the SPI flash may be read by accessing a memory location between 0x01000000 and 0x01FFFFFF on the native MCU address bus. The SPI0 peripheral translates the address from the MCU address bus to the SPI flash using the SPI flash offset register SPI0FOS. The address accessed on the SPI flash is the 24-bit unsigned addition of the MCU address bus with SPI0FOS. Note that the addition wraps around 0 if the sum exceeds 0xFFFFFF, discarding any bits outside the range of bits 23 down to 0. SPI0FOS can be used to map a block of data in the SPI flash to a desired MCU address.

11.10 SPI0 Boot Behavior

The SPI0 peripheral is used at boot time to load the program from the SPI flash. When the chip is powered on or reset, the state of the BOOT pin determines if the chip will fetch and execute the program from the SPI flash or launch the interactive Forth interpreter. If it fetches the program, it uses the SPI0 peripheral to receive the program from the attached SPI flash. As a consequence, the user must guarantee that any other slave devices attached to the SPI0 peripheral will not vie for control over the MISO0 pin. This error can happen if one of the GPIO pins controls the CS pin of such a slave, as most of the GPIO pins become high impedance inputs upon chip reset. It is therefore possible that the slave's CS wire will settle to an asserted (low) state, causing the slave to attempt to seize control of the MISO0 pin. Care must be taken to prevent this possibility. One option is to use a start high pin (SHx), which resets as an output driving high, to control the slave's CS wire. Another option is to keep the SPI flash as the only slave attached to the SPI0 peripheral.

11.11 Register Description

11.11.1 SPICR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

SPI control register

31	30	29	28	27	26	25	24
Unused							
23	22	21	20	19	18	17	16
Unused				FEN	SM	TXSB	RXSB
				rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
BR							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
EN	MSB	TCIE	TEIE	DL		CPOL	CPHA
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bits 31-20	
FEN	Bit 19	SPI Flash enable. Allows native read-only memory access to the SPI flash memory attached to SPI0. (Only available on SPI0) 0 Disabled 1 Enabled
SM	Bit 18	SPI slave mode (Note that SPI0 is master-only, and this bit is unavailable) 0 Master mode 1 Slave mode
TXSB	Bit 17	SPI TX swap bytes. Swaps the byte order in 16- and 32-bit transmissions. In 32-bit transmissions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit transmissions, bytes 1 and 0 are swapped. Does not affect 8-bit transmissions. 0 Bytes not reversed 1 Bytes reversed
RXSB	Bit 16	SPI RX swap bytes. Swaps the byte order in 16- and 32-bit receptions. In 32-bit receptions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit receptions, bytes 1 and 0 are swapped. Does not affect 8-bit receptions. 0 Bytes not reversed 1 Bytes reversed
BR	Bits 15-8	SPI clock (SCK) baud control for master mode. Baud rate = $SMCLK / (2 * (1 + SPIBR))$
EN	Bit 7	SPI enable 0 Disabled

		1	Enabled
MSB	Bit 6	Endianness select	
		0	LSB-first
		1	MSB-first
TCIE	Bit 5	SPI transmit complete interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
TEIE	Bit 4	SPI transmit register empty interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
DL	Bits 3-2	SPI transmission data length select	
		00	SPIDL_8: 8-bit transfers
		01	SPIDL_16: 16-bit transfers
		10	SPIDL_32: 32-bit transfers
CPOL	Bit 1	SPI clock (SCK) polarity	
		0	SCK idles low, leading edge is a rising edge (for SPI_MODE0 or SPI_MODE1)
		1	SCK idles high, leading edge is a falling edge (for SPI_MODE2 or SPI_MODE3)
CPHA	Bit 0	SPI clock (SCK) phase. Note that in slave mode, only SPICPHA = 1 is supported, and this bit becomes a don't care.	
		0	Data is sampled on leading edge of SCK, data is updated on trailing edge of SCK (for SPI_MODE0 or SPI_MODE2)
		1	Data is updated on leading edge of SCK, data is sampled on trailing edge of SCK (for SPI_MODE1 or SPI_MODE3)

11.11.2 SPISR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

SPI status register

7	6	5	4	3	2	1	0
Unused					BUSY	TCIF	TEIF
					r-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
Unused	Bits 7-3	
BUSY	Bit 2	Indicates if a SPI transfer is occurring or not. When in slave mode, this bit is set whenever the chip select pin is asserted (driven low). 0 A SPI transfer is not presently occurring 1 A SPI transfer is presently occurring
TCIF	Bit 1	SPI transfer complete interrupt flag. This bit is set when a SPI transfer has completed, and must be cleared in software before it can be set again. To clear the bit, write a 1 to it or read the SPIxRX register. 0 A SPI transfer has not been completed 1 A SPI transfer has been completed (must be cleared before it can be set again)

TEIF	Bit 0	SPI transmit register empty interrupt flag. This bit is set when the SPI transmit register SPIxTX is empty, and indicates SPIxTX is ready to accept new data for the next transfer (though the new data will not be transmitted until a current transfer is complete). Must be cleared in software before it can be set again. To clear the bit, write a 1 to it.
	0	SPIxTX is not empty
	1	SPIxTX is empty and ready for new data (must be cleared before it can be set again)

11.11.3 SPITX Register

Register memory slot: 2, offset: 8 bytes, size: 4 bytes

SPI transmit buffer register. In master mode, write to this register to begin a SPI transfer. In slave mode, write to this register to queue the data to transmit during the next transfer, which will be initiated by the master.

31	30	29	28	27	26	25	24
xTX							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
xTX							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
xTX							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xTX							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

11.11.4 SPIRX Register

Register memory slot: 3, offset: 12 bytes, size: 4 bytes

SPI receive buffer register. The received SPI data will appear in this register after each SPI transfer is complete.

31	30	29	28	27	26	25	24
xRX							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
23	22	21	20	19	18	17	16
xRX							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
15	14	13	12	11	10	9	8
xRX							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
7	6	5	4	3	2	1	0
xRX							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

11.11.5 SPIFOS Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

SPI Flash memory address offset. This value is added to the 24-bit SPI Flash address, wrapping around back to 0 after 0x00FFFFFF. This can be used to create a virtual address for a file in the SPI Flash to make it appear at that address. (Only available in SPI0).

31	30	29	28	27	26	25	24
Unused							
23	22	21	20	19	18	17	16
xFOS							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
15	14	13	12	11	10	9	8
xFOS							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
7	6	5	4	3	2	1	0
xFOS							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

12 UARTx Peripheral

The Universal Asynchronous Receiver/Transmitter (UART) is an asynchronous, full-duplex serial interface. The main features of the UART are listed below:

- Full-duplex, independent receive and transmit operation
- Asynchronous (no serial clock wire)
- Two wires, one for TX and one for RX
- Communicates with one external device
- 8-bit data frames
- Adjustable baud generator
- Even or odd parity check flag
- Receive overflow flag
- Interrupts for receive complete, transmit complete, and TX register empty

12.1 UART Pins

The UARTx peripheral has two pins: RXx and TXx.

The TXx pin transmits each data bit from this UARTx peripheral to the external device. It must be attached to the RX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of TXx, making it an output.

The RXx pin receives each data bit from the external device. It must be attached to the TX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of RXx, making it a high impedance input.

The UART transmitter and receiver are independent of one another. Thus, transmissions and receptions need not occur at the same time or with the same external device. Indeed, both pins do not necessarily need to be used if one-way communications are all that is needed.

A connection diagram for the UART is displayed in Figure 7.

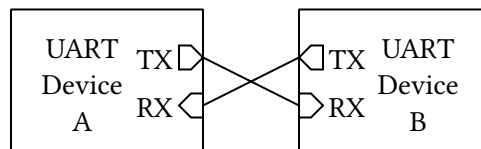


Figure 7: UART Connection Diagram

12.2 Generalized UART Transmission Process

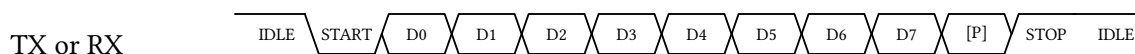


Figure 8: UART Data Frame Timing Diagram

Each UART wire is unidirectional, with the TX pin of the transmitting device driving the logic level of the wire and the RX pin of the receiving device reading the level of the pin. The only distinction between the two wires is the direction that data travels, and as such, this section will only depict one wire.

When no UART transmission is occurring, the UART wire is idle, and the TX pin drives the UART wire high. To start a transmission, the TX pin transmits a start bit by driving the wire low for $T_B = 1/\text{baud}$ seconds. It then begins transmitting each data bit, LSB-first, every T_B seconds until all 8 bits have been transmitted. Then, it optionally transmits a parity bit, which helps detect transmission errors. Finally, it transmits a stop bit by driving the wire high for T_B seconds. After that point, it may begin another transmission by sending another start bit, or it can let the wire be idle.

Figure 8 shows a timing diagram for the UART transmission process. Note that the parity bit [P] is optional.

12.3 UART Baud Generation

Both devices communicating over the UART must agree on the baud, or the number of bits transmitted per second. In the absence of a clock wire, the UART uses the baud to sample the RX wire every $T_B = 1/\text{baud}$ seconds, recording the logic levels in sequence. As such, the transmitter must send bits at that same rate. Any significant deviation of the baud can result in bit errors.

The baud for the UARTx peripheral is configured through the 12-bit UARTxBR register:

$$\text{UART baud} = \frac{f_{\text{SMCLK}}}{16 \times (\text{UARTxBR} + 1)}$$

The baud may therefore be anywhere from 16 to 65,536 times less than the SMCLK frequency.

12.4 Parity Bit

A UART frame has an optional parity bit after the last data bit and before the first stop bit. The purpose of the parity bit is to verify that the data received is indeed the data that was transmitted. If the received parity bit does not match the expected parity bit, then the receiver knows that the received data has been corrupted.

If the parity bit is enabled with UPEN, the transmitter calculates the parity of the byte to transmit by taking the exclusive OR of all 8 bits in the transmitted byte, the result of which is then put through a final exclusive OR gate with the parity polarity bit UPODD:

$$p = b_7 \oplus b_6 \oplus \dots \oplus b_1 \oplus b_0 \oplus \text{UPODD}$$

When UPODD is 0, the parity is said to be “even”, and when it is 1, “odd”. This operation is functionally counting how many bits are 1s in the transmitted byte. When even parity is selected, the parity bit is 0 when the number of 1s is even and 0 when the number of 1s is odd. That result is inverted when odd parity is selected.

On the receiver end, the parity bit is capable of detecting a single bit error in the transmission. That is, if one of the eight data bits (or the parity bit itself) is incorrectly received, then the parity error bit UPEF will detect a bit error. However, if two bits are incorrectly received, then the parity error bit will not detect the bit error. Note that if the number of bit errors is an odd number, the parity error bit will detect the error, but will not when the number of bit errors is even.

If a parity bit is enabled on the transmitter side, it must also be enabled on the receiver side with UPEN.

12.5 Transmit Buffer Register Queue

Writing to the UART transmit buffer register UARTxTX queues up the byte written to it as the next frame to transmit. If a transmission is not currently ongoing, the UARTx peripheral immediately begins transmitting the byte written to UARTxTX, and UTEIF is set immediately. If a transfer is currently ongoing, the UARTx peripheral waits for the current frame to finish transferring. As soon as it finishes, the UARTx peripheral begins transmitting the new frame written to UARTxTX and sets UTEIF.

The TX queue allows the UARTx peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the UARTxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the UTEIF flag to be 1 or URBIF to be 0 before writing to UARTxTX.

12.6 Flags and Interrupts

The UARTx peripheral has five non-interrupt flags/indicators and three interrupt flags in the UARTxSR register. If the corresponding interrupts are enabled in the UARTxCR register and the UARTx ISR is not masked, then the three interrupt flags will generate an interrupt whenever they are equal to 1. Any and all interrupt flags must be cleared before the UARTx ISR returns, else the ISR will mistakenly re-execute immediately upon the prior return.

The UART receiver busy indicator URBIF shows when the UARTx peripheral is currently receiving a byte of data when it is 1. Likewise, UTBF indicates that a transmission is ongoing when 1.

The UART framing error non-interrupt flag UFEF indicates that a framing error occurred on the last reception when it is a 1. A framing error typically occurs if the UART transmitter settings do not match the receiver settings, such as baud, number of bits to transmit, the presence/absence of a parity bit, or the presence/absence of a second stop bit. Specifically, a framing error is detected if the receiver does not get the start bit and stop bit(s) at the expected times. If UFEF is a 1 after a reception, that reception must be treated as unrecoverable, corrupted data. UFEF is cleared when the UARTxRX register is read.

The UART parity error non-interrupt flag UPEF indicates that a parity error occurred on the last reception when it is a 1. A parity error can only happen if the parity bit is enabled by setting UPEN to 1. A parity error occurs when the parity bit appended to the received data byte does not match the expected value. If UPEF is a 1 after a reception, that reception must be treated as corrupted data. UPEF is cleared when the UARTxRX register is read.

The UART receive data overflow non-interrupt flag UOVF indicates that the user did not read the UARTxRX register in time before another byte of data was received by the UARTx peripheral when it is a 1. In this case, the received data must be treated as incomplete. UOVF is cleared when the UARTxRX register is read.

The UART receive complete interrupt flag URCIF is set to 1 immediately when the UARTx peripheral finishes receiving a frame of data from the transmitter. The new data byte becomes available in the UARTxRX register as soon as URCIF is set. Also, this is the moment that UFEF, UPEF, and UOVF bits will be set, if the proper conditions are met. URCIF is cleared when the UARTxRX register is read.

The UART transmit register empty interrupt flag UTEIF is set the moment the UART TX buffer register UARTxTX becomes empty (i.e., is ready to queue up the next byte to transmit). If a frame is not currently being transmitted, the flag is set immediately when a new byte is written to UARTxTX. If a frame is currently being transmitted, it is set as soon as current transfer is completed, whereupon the value in the UARTxTX register begins to be transmitted. Read the UARTxSR register to clear UTEIF.

The UART transmit complete interrupt flag UTCIF is set the moment the UART transmission completes, and is cleared by reading UARTxSR.

12.7 Transmit Procedure

To transmit a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud with the UARTxBR register. Of note, the UART enable bit UEN must be a 1 to activate the UART. The GPIO pin multiplexed with TXx must be set to secondary/peripheral mode in the corresponding PxSEL register.

To transmit a byte of data over the UART, first clear the status register by reading it and discarding its value. Then, wait until the UART is not transmitting (when UTBF is 0) or the UART TX buffer is empty (when UTEIF). Next, write the byte to transmit to the UARTxTX register. The byte will be finished transmitting once UTBF is 0.

A simplified procedure for transmitting a byte over the UART shown in Listing 2.

```

1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Definitions **/
5  void uartx_putc(char tx_data)
6  {
7      // Wait for the transmitter to finish any previous transmissions
8      while (UARTxSR & UTBF) { }
9
10     // Send the new data byte
11     UARTxTX = c;
12 }
```

Listing 2: Example Function for UART Transmissions

12.8 Receive Procedure

To receive a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud with the UARTxBR register. Of note, the UART enable bit UEN must be a 1 to activate the UART. The GPIO pin multiplexed with RXx must be set to secondary/peripheral mode in the corresponding PxSEL register.

Once done setting up the UART, wait for the UART receive complete interrupt flag URCIF to be 1. Then, read the received byte contained in the UARTxRX register.

A simplified procedure for receiving a byte over the UART shown in Listing 3.

```

1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Definitions **/
5  char uartx_getc()
6  {
7      // Wait for the UART to receive a new character
8      while ((UARTxSR & URCIF) == 0) { }
9
10     // Return the new character that is in the receive buffer register
11     return UARTxRX;
12 }
```

Listing 3: Example Function for UART Receptions

12.9 UART0 Boot Behavior

When booting in Forth interpreter mode, the UART0 peripheral provides an interactive command processor with an external device, such as computer with a USB to UART device attached. When the MCU is powered

up or reset and the BOOT pin is low, the Forth interpreter will launch over UART0, which can be used to program or debug the MCU.

Section 4 elaborates on the power up and reset behavior of the MCU.

12.10 Register Description

12.10.1 UARTCR Register

Register memory slot: 0, offset: 0 bytes, size: 1 byte

UART control register

7	6	5	4	3	2	1	0
Unused	EN	PEN	PODD	RCIE	TEIE	TCIE	
	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	

Bitfield	Range	Description
Unused	Bits 7-6	
EN	Bit 5	UART enable 0 Disabled 1 Enabled
PEN	Bit 4	UART parity enable 0 Disabled 1 Enabled
PODD	Bit 3	UART parity select 0 Even parity 1 Odd parity
RCIE	Bit 2	UART receive complete interrupt enable 0 Interrupt disabled 1 Interrupt enabled
TEIE	Bit 1	UART transmit register empty interrupt enable 0 Interrupt disabled 1 Interrupt enabled
TCIE	Bit 0	UART transmit complete interrupt enable 0 Interrupt disabled 1 Interrupt enabled

12.10.2 UARTSR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

UART status register

7	6	5	4	3	2	1	0
RBF	TBF	FEF	PEF	OVF	RCIF	TEIF	TCIF
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
RBF	Bit 7	UART receiver busy flag 0 Receiver idle and ready for new reception 1 Receiver busy with ongoing reception

TBF	Bit 6	UART transmitter busy flag 0 Transmitter idle and ready for new transmission 1 Transmitter busy with ongoing transmission
FEF	Bit 5	UART framing error flag. Read UARTxRX to clear. 0 No framing error 1 Framing error on last reception
PEF	Bit 4	UART parity error flag. Read UARTxRX to clear. 0 No parity error 1 Parity error on last reception
OVF	Bit 3	UART receive data overflow flag. Read UARTxRX to clear. 0 No receive data overrun 1 Receive data overflow on last reception
RCIF	Bit 2	UART receive complete interrupt flag. Read UARTxRX to clear. 0 No new receive complete interrupt pending 1 New receive complete interrupt pending
TEIF	Bit 1	UART transmit register empty interrupt flag. Write a 1 to this bit to clear it. 0 No new transmit register empty interrupt pending 1 New transmit register empty interrupt pending
TCIF	Bit 0	UART transmit complete interrupt flag. Write a 1 to this bit to clear it. 0 No new transmit complete interrupt pending 1 New transmit complete interrupt pending

12.10.3 UARTBR Register

Register memory slot: 2, offset: 8 bytes, size: 2 bytes

UART baud rate register

15	14	13	12	11	10	9	8
Unused				BR			
				rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
BR							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bits 15-12	
BR	Bits 11-0	UART baud rate = SMCLK / (16 * (UBR + 1))

12.10.4 UARTRX Register

Register memory slot: 3, offset: 12 bytes, size: 1 byte

UART receive buffer register



12.10.5 UARTTX Register

Register memory slot: 4, offset: 16 bytes, size: 1 byte

UART transmit buffer register



13 TIMERx Peripheral

The TIMERx peripheral is a 32-bit general purpose timer/counter unit. Its main features are listed below:

- 32-bit timer values, from 0 to 4,294,967,295
- Memory mapped register to read and write current timer value
- Three output compare units, two of which can generate pulse width modulation (PWM) waveforms
- Two input capture units for precise timing measurements of external signals
- Multiple clock sources
- Clock divider
- Interrupts for input capture, output compare, and timer overflow events

13.1 Timer Pins

The TIMERx peripheral has four pins: TxCMP0, TxCMP1, TxCAP0, and TxCAP1.

The TxCMP0 pin outputs the PWM waveform generated by the output compare unit 0. When the corresponding PxSEL[y] bit is a 1, the TIMERx peripheral seizes the DIR, OUT, and REN controls of the pin, forcing it to be in output mode. Similarly, TxCMP1 outputs the PWM waveform of output compare unit 1.

The TxCAP0 pin is an input that waits for a pin state change and then notifies the input capture unit 0, allowing precise timing of pin changes. The corresponding PxSEL[y] bit has no effect on this pin, since its secondary/peripheral function only ever acts as an input. Similarly, TxCAP1 is attached to input capture unit 1.

13.2 Clock Sources and Divider

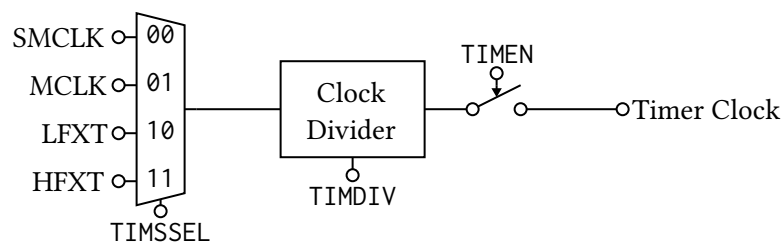


Figure 9: TIMERx Clock Diagram

The clock source for the TIMERx peripheral can be selected using TIMSSEL from four options: SMCLK, MCLK, LFXT (low frequency external clock), and HFXT (high frequency external clock). An anti-glitch filter built into the clock source multiplexer prevents spurious clock transitions on the timer clock, allowing the user to switch clock sources freely even while the timer is enabled. However, switching clocks will cause the timer clock to be inactive for several clock cycles of the clock that is being switched from or to, whichever has a smaller frequency.

After the clock multiplexer, the timer clock frequency may optionally be reduced with a clock divider. The timer value register TIMxVAL increments by 1 on every rising edge of the divided timer clock when TIMERx is enabled with TIMEN.

A clock diagram for the TIMERx peripheral is shown in Figure 9.

13.3 Starting, Halting, Setting, and Reading the Timer

To start the timer, set the TIMEN bit to 1, whereupon the timer will begin incrementing by 1 with each rising edge of the divided timer clock. To halt the timer at its present value, clear the TIMEN bit to 0. The timer will retain its value until it is enabled again, its value is set manually, or the MCU is reset. If the timer is reenabled, it will begin counting from the timer value just before it was reenabled.

The timer value may be manually set at any time by writing the new 32-bit timer value to the TIMxVAL register. The timer value is updated immediately, regardless of if the timer is enabled or disabled.

The timer value may be read at any time by reading its value from the TIMxVAL register, regardless of if the timer is enabled or disabled.

13.4 Overflow and Rollover Behavior

When the timer is enabled and its 32-bit value reaches its maximum of 4,294,967,295, the value will roll over back to zero on the next rising edge of the timer clock and the timer overflow interrupt flag TIMOVIF will be set. The timer will then continue incrementing as normal.

Optionally, the user may configure the timer to roll over before the 32-bit limit is reached by setting the new rollover value in the TIMxCMP2 register and setting the TIMCMP2RST bit to 1. In this mode, when the timer value becomes equivalent to the value of TIMxCMP2, the timer value will roll over back to zero on the next rising edge of the timer clock and then continue incrementing as normal, as depicted in Figure 10. This mechanism is used for setting the period of the PWM signals for the output compare units. Note: the timer rolls over back to zero when its value is *equal* to that of TIMxCMP2, but not if its value is greater than TIMxCMP2. This becomes important if the user manually sets the timer value above TIMxCMP2, or if the user sets TIMxCMP2 below the current timer value, whereupon the timer will continue counting all the way to the 32-bit maximum value.

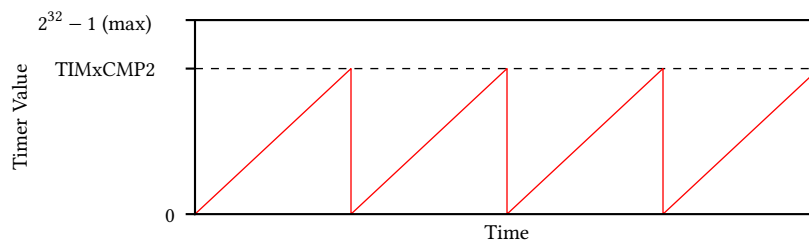


Figure 10: Timer Rollover Operation

13.5 Output Compare Units and Pulse Width Modulation

The TIMEx peripheral contains two output compare units that can be used to create two pulse width modulation (PWM) output signals on GPIO pins. Each output compare unit has an output compare register (TIMxCMP0 and TIMxCMP1), which is continually compared to the timer value TIMxVAL. The corresponding output compare pins (TxCMP0 and TxCMP1) toggle when either the timer value TIMxVAL equals the output compare register TIMxCMPy, or the timer rolls over. The TIMCMP0H and TIMCMP1H bits control the initial state of the TxCMP0 and TxCMP1 pins when the timer value is less than the output compare register. Figure 11 provides an example of PWM operation using an output compare unit.

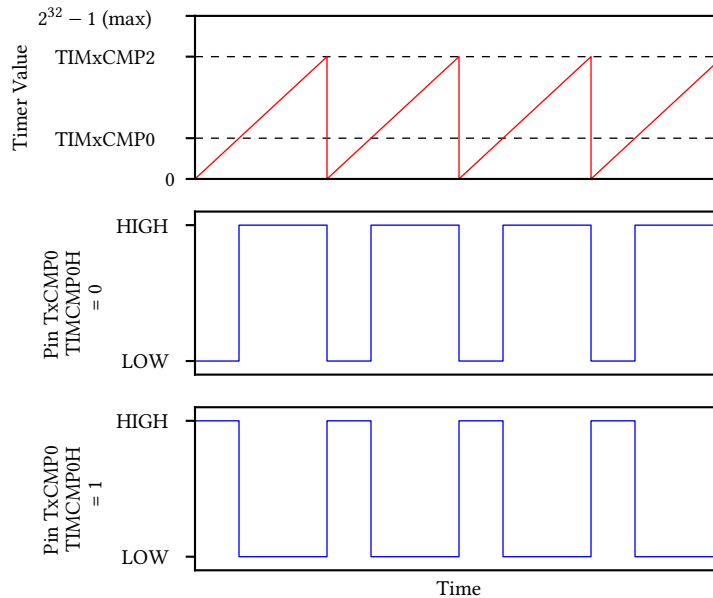


Figure 11: Timer Output Compare and PWM Operation

13.6 Input Capture Units

The TIMEx peripheral contains two input capture units which allow the precise timestamping of pin change events. When enabled with TIMCAPyEN, the input capture unit waits for the TxCAPy pin to toggle in the appropriate edge direction. The TIMCAPyFE bit selects either rising edge or falling edge sensitivity. When the pin toggles in the appropriate edge direction, the input capture unit stores the current timer value in the TIMxCAPy register and sets the TIMCAPy interrupt flag. The TIMCAPy bit must be cleared before the next input capture event by writing any value to the TIMxSR register.

Note: changing the value of TIMCAPyFE while TIMCAPyEN is enabled can inadvertently cause false input capture triggers. The user should not change TIMCAPyFE while TIMCAPyEN is enabled.

13.7 Flags and Interrupts

The TIMEx peripheral has two indicator bits and six interrupt flags in the timer status register TIMxSR.

The two indicator bits TCMP0 and TCMP1 show the current value of the output compare pins TxCMP0 and TxCMP1 respectively, regardless of if the corresponding GPIO pins are set to primary/GPIO mode or secondary/peripheral mode.

The timer value overflow interrupt flag TIMOVIF is set when the 32-bit timer value overflows from $2^{32} - 1$ back to 0. If TIMOVIE is set, the TIMEx ISR will be executed. TIMOVIF is cleared by writing any value to the status register TIMxSR.

The output compare interrupt flags TIMCMP0IF, TIMCMP1IF, and TIMCMP2IF are set when the timer value equals the respective output compare value (TIMxCMP0, TIMxCMP1, and TIMxCMP2). If TIMCMPyIE is set, the TIMEx ISR will be executed. TIMCMPyIF is cleared by writing any value to the status register TIMxSR.

The input capture interrupt flags TIMCAP0IF and TIMCAP1IF are set when a new input capture event

occurs on the TxCAP0 and TxCAP1 pins, respectively. The input capture register TIMxCAPy can be read as soon as TIMCAPyIF is set. If TIMCAPyIE is set, the TIMERx ISR will be executed. TIMCAPyIF is cleared by writing any value to the status register TIMxSR.

13.8 Register Description

13.8.1 TIMCR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

Timer/Counter control register

31	30	29	28	27	26	25	24
Unused							
23	22	21	20	19	18	17	16
Unused				DIV			
rw-(0)				rw-(0)			
15	14	13	12	11	10	9	8
CMP1IH	CMP0IH	CAP1FE	CAP0FE	CAP1EN	CAP0EN	SSEL	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
CMP2RST	EN	CAP1IE	CAP0IE	OVIE	CMP2IE	CMP1IE	CMP0IE
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bits 31-20	
DIV	Bits 19-16	Timer/Counter clock divider. Divides the clock source selected by TIMSSEL to produce each increment to TIMxVAL 0000 TIMDIV_1: /1 (no division) 0001 TIMDIV_2: /2 0010 TIMDIV_4: /4 0011 TIMDIV_8: /8 0100 TIMDIV_16: /16 0101 TIMDIV_32: /32 0110 TIMDIV_64: /64 0111 TIMDIV_128: /128 1000 TIMDIV_256: /256 1001 TIMDIV_512: /512 1010 TIMDIV_1024: /1,024 1011 TIMDIV_2048: /2,048 1100 TIMDIV_4096: /4,096 1101 TIMDIV_8192: /8,192 1110 TIMDIV_16384: /16,384 1111 TIMDIV_32768: /32,768
CMP1IH	Bit 15	Compare 1 TxCMP1 initial PWM logic state 0 Initial PWM logic state LOW 1 Initial PWM logic state HIGH
CMP0IH	Bit 14	Compare 0 TxCMP0 initial PWM logic state 0 Initial PWM logic state LOW

		1	Initial PWM logic state HIGH
CAP1FE	Bit 13	Capture 1 edge select	
		0	Rising edge
		1	Falling edge
CAP0FE	Bit 12	Capture 0 edge select	
		0	Rising edge
		1	Falling edge
CAP1EN	Bit 11	Capture 1 enable	
		0	Disabled
		1	Enabled
CAP0EN	Bit 10	Capture 0 enable	
		0	Disabled
		1	Enabled
SSEL	Bits 9-8	Timer/Counter clock source select	
		00	TIMSSSEL_SMCLK: SMCLK
		01	TIMSSSEL_MCLK: MCLK
		10	TIMSSSEL_LFXT: Low frequency crystal clock
		11	TIMSSSEL_HFXT: High frequency crystal clock
CMP2RST	Bit 7	Enable Timer/Counter reset on Compare 2 event	
		0	Does not reset Timer/Counter on Compare 2 event
		1	Resets Timer/Counter on Compare 2 event
EN	Bit 6	Timer/Counter enable	
		0	Disabled
		1	Enabled
CAP1IE	Bit 5	Capture 1 interrupt enable	
		0	Disabled
		1	Enabled
CAP0IE	Bit 4	Capture 0 interrupt enable	
		0	Disabled
		1	Enabled
OVIE	Bit 3	Timer/Counter overflow interrupt enable	
		0	Disabled
		1	Enabled
CMP2IE	Bit 2	Compare 2 interrupt enable	
		0	Disabled
		1	Enabled
CMP1IE	Bit 1	Compare 1 interrupt enable	
		0	Disabled
		1	Enabled
CMP0IE	Bit 0	Compare 0 interrupt enable	
		0	Disabled
		1	Enabled

13.8.2 TIMSR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

Timer/Counter status register.

7	6	5	4	3	2	1	0
TCMP1	TCMP0	CAP1IF	CAP0IF	OVIF	CMP2IF	CMP1IF	CMP0IF
r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
TCMP1	Bit 7	Current value of the Compare 1 pin TxCMP1 0 LOW 1 HIGH
TCMP0	Bit 6	Current value of the Compare 0 pin TxCMP0 0 LOW 1 HIGH
CAP1IF	Bit 5	Capture 1 interrupt flag. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Interrupt pending
CAP0IF	Bit 4	Capture 0 interrupt flag. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Interrupt pending
OVIF	Bit 3	Timer/Counter overflow interrupt flag. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Interrupt pending
CMP2IF	Bit 2	Compare 2 interrupt flag. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Interrupt pending
CMP1IF	Bit 1	Compare 1 interrupt flag. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Interrupt pending
CMP0IF	Bit 0	Compare 0 interrupt flag. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Interrupt pending

13.8.3 TIMVAL Register

Register memory slot: 2, offset: 8 bytes, size: 4 bytes

Timer/Counter value register. Read to get the number of cycles of the divided down source clock since the timer was reset or enabled. Write to set an arbitrary starting point for the Timer/Counter.

31	30	29	28	27	26	25	24
xVAL							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
xVAL							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
xVAL							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xVAL							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

13.8.4 TIMCMP0 Register

Register memory slot: 3, offset: 12 bytes, size: 4 bytes

Timer/Counter Compare 0 comparison register. When Compare 0 is enabled, a Compare 0 event will be triggered when TIMxVAL equals TIMxCMP0.

31	30	29	28	27	26	25	24
xCMP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
xCMP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
xCMP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xCMP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

13.8.5 TIMCMP1 Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

Timer/Counter Compare 1 comparison register. When Compare 1 is enabled, a Compare 1 event will be triggered when TIMxVAL equals TIMxCMP1.

31	30	29	28	27	26	25	24
xCMP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
xCMP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
xCMP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xCMP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

13.8.6 TIMCMP2 Register

Register memory slot: 5, offset: 20 bytes, size: 4 bytes

Timer/Counter Compare 2 comparison register. When Compare 2 is enabled, a Compare 2 event will be triggered when TIMxVAL equals TIMxCMP2.

31	30	29	28	27	26	25	24
xCMP2							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
xCMP2							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
xCMP2							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xCMP2							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

13.8.7 TIMCAP0 Register

Register memory slot: 6, offset: 24 bytes, size: 4 bytes

Timer/Counter Capture 0 value register. When Capture 0 is enabled and a Capture 0 event occurs, this register will latch the value of TIMxVAL.

31	30	29	28	27	26	25	24
xCAP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
xCAP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
xCAP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xCAP0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

13.8.8 TIMCAP1 Register

Register memory slot: 7, offset: 28 bytes, size: 4 bytes

Timer/Counter Capture 1 value register. When Capture 1 is enabled and a Capture 1 event occurs, this register will latch the value of TIMxVAL.

31	30	29	28	27	26	25	24
xCAP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
xCAP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
xCAP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xCAP1							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14 SYSTEM Peripheral

The System peripheral manages a potpourri of system-level blocks, including the MCU clocks, a CRC data verification module, analog bias levels within the global bias generator, and the bandgap voltage generator.

14.1 System Clocks and Clock Management

The MCU has three clock sources: HFXT, LFXT, and DCO0. HFXT and LFXT are external clock sources, which must be provided to the MCU via the CLKHFXT and CLKLFXT pins, respectively. DCO0 is an internal, programmable frequency clock source provided by a digitally controllable oscillator circuit.

HFXT must not exceed a maximum clock rate of about 100 MHz. LFXT is suggested to be exactly 32.768 kHz to provide an accurate method of measuring time. The frequency of DCO0 is configured with the DCO0FREQ register, and can be up to 112 MHz.

Two clocks, the main clock (MCLK) and the submain clock (SMCLK), form the roots of the MCU clock tree, shown in Figure 12. MCLK and SMCLK can each select one of the source clocks as their base and divide the frequency further with CLKDIVCR. MCLK drives the CPU clock, while SMCLK drives most of the peripherals. MCLK cannot be turned off, but all other clocks and source clocks (including the CPU clock) can be turned off with the SYSCLKCR register. If a source clock is being used by MCLK, that source clock will be turned on (even if it is turned off in SYSCLKCR) unless the CPU clock is also turned off and an interrupt service routine is not executing. If a source clock is being used by SMCLK, it cannot be turned off with the SYSCLKCR register.

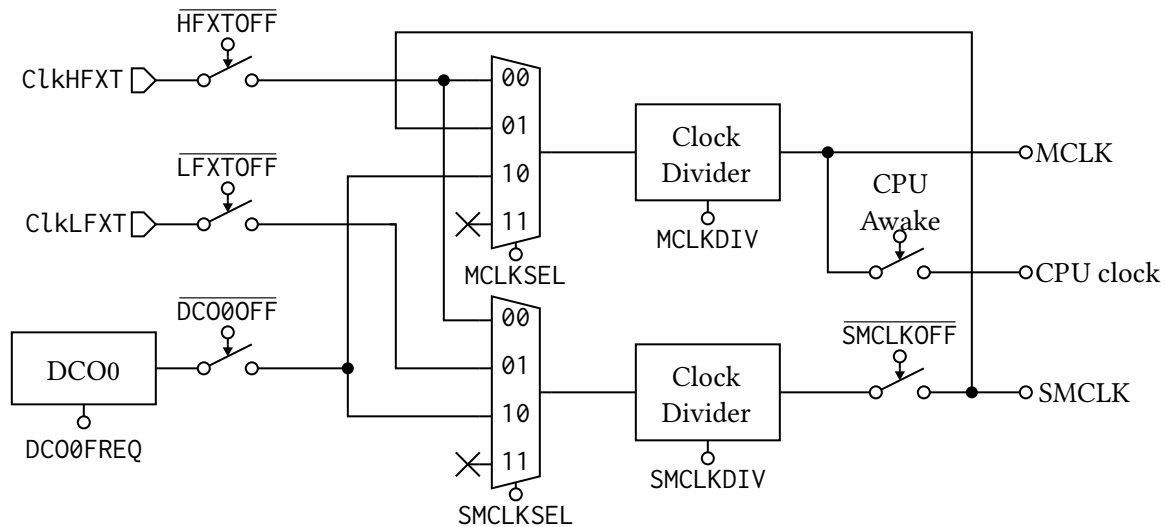


Figure 12: MCU Clock System

14.2 Power Saving Features

The MCU has several power-saving features.

The CPU is the most power-hungry part of the MCU, and its dynamic power consumption is proportional to the CPU clock frequency. Reducing the frequency of MCLK, the clock source for the CPU clock, is the main avenue to reduce the total dynamic power consumption. This can be done in one of two ways: either utilizing the MCLK clock divider, or directly reducing the frequency of the clock source. Reducing the clock source frequency is not always possible (such as when MCLK is sourced by an external clock

on the ClkHFXT pin), but it is always possible to switch the clock source to the internal DCO, which has a wide range of programmable frequencies to choose from.

Another method to save power is to turn off unused clocks, such as the internal DCO, when they are not being used. Remember, however, that the ClkHFXT pin is required to be a valid clock signals while the MCU boots. Failure to provide the clock may cause the chip to boot improperly, or not at all. For this reason, the start HIGH pin(s) (SHx) have been provided. On the PCB level, the start HIGH pins should connect to the enable pin of the clock generator chip for ClkHFXT so that they will always be driven HIGH when the MCU is reset. After the chip has booted, the user can switch the source clock for MCLK and then safely disable ClkHFXT.

The static power (or leakage power) can be reduced by switching off unused internal memories when they are not being used. Static power consumption, caused by the accumulated reverse bias currents of the many thousands of parasitic diodes in the digital circuits on the chip, is ever present as long as the chip is receiving power from the VDD, VDDPST, and AVDD pins. The on-chip memory units, however, include a power gating feature that switches off the circuit delivering them power. The ROM and each 16 KiB SRAM block can be powered down by setting ROMOFF and SRAMxOFF, respectively. Every additional memory that is powered down will reduce the static power consumption. Any read or write operations to a powered down will fail, producing undefined results. Furthermore, the entire contents of an SRAM block is lost when it is powered down, and is initially undefined when it is powered back on until it is filled again by the user. It is therefore important to keep track of which memories are or have been powered down to prevent undefined behavior.

When switching off SRAM blocks, it is recommended to power down the blocks at higher memory addresses first before powering down the lower blocks. The program occupies the lower memory addresses, which must not be corrupted by a negligent power down, but the stack, which is easily initialized in a custom location, starts at a high address and builds downward. To initialize the stack out of a powered down SRAM block and into a powered up block, add `-D RedefinedStackPointerInit=0x??` to the USER_DEFINES variable in the project makefile, replacing 0x?? with the desired memory address. The SRAM02 block should never be switched off because it contains the interrupt vector table.

The analog circuits on the chip can be powered down by disabling them in their control registers and disabling the global bias generator.

14.3 CPU Sleep Mode

Another way to conserve power is to only use the CPU when needed, and switch it off when not in use. As the CPU consumes the majority of the dynamic power on the chip, reducing the amount of time it is on will result in an overall power reduction. If properly configured, the peripherals can still operate even while the CPU is asleep, and can wake the CPU when certain events occur. Thus, the dynamic CPU power can be duty cycled such that the long term CPU power is equal to the percentage of the time that the CPU is on multiplied by the instantaneous power the CPU consumes when on: $P_{\text{CPU}} = P_{\text{CPU}, 100\%} \times t_{\text{on}} / (t_{\text{on}} + t_{\text{off}})$.

Sleep mode is activated with the custom sleep instruction, which reduces the dynamic power of the CPU to zero by switching off the CPU clock. Once it has entered sleep mode, the MCU can only awaken if an interrupt is triggered or if the MCU is reset. If an interrupt is triggered, the CPU clock is reactivated for the duration of the interrupt. Once all pending interrupts have been serviced, the MCU re-enters sleep mode. However, if one of the serviced ISRs executes the custom wake instruction, then the CPU will remain awake at the end of the master interrupt handler and jump back to the main part of the program where it left off when the CPU was put to sleep.

A code example to put the CPU to sleep is included in Listing 4.

```
1 /** Includes **/
2 #include <MemoryMap.h>
```

```
3  #include <irq.h>
4
5  /** Interrupt Service Routine Declaration Macros */
6  RVISR(IRQ_TIMER1_VECTOR, ISR_TIMER1)
7
8  /** Main Function */
9  int main()
10 {
11     // Set up TIMER1 here
12
13     // Enable interrupts by unmasking them
14     enable_all_interrupts();
15
16     // Go to sleep
17     cpu_sleep();
18
19     // Insert code to execute after ISR wakes the CPU
20
21     // Wait forever
22     while (1) {}
23 }
24
25 /** Interrupt Service Routines */
26 void ISR_TIMER1()
27 {
28     // Do ISR processing here
29
30     // Wake the CPU
31     cpu_wake();
32 }
```

Listing 4: Example program to use CPU sleep mode

14.4 CRC Module

The CRC module enables the computation of a CRC16 checksum on an array of data. The checksum can be used to ensure the data in a transmission or reception is free of any corruption in the communication channel.

The specifications of the CRC module uses the CRC16 CDMA2000 standard, which has the following specifications:

- CRC size: 16 bits
- Polynomial: 0xC857
- Initial CRC value: 0xFFFF
- No XOR operation on the output CRC value
- No input data reflection
- No output CRC value reflection

To use the CRC module on a byte array, first initialize the CRC value by writing 0xFFFF to CRCSTATE. Then, write each byte in the array to CRCDATA. Finally, read CRCSTATE to get the computed CRC16 value of the array.

14.5 Global Analog Bias Generator

The on-chip analog blocks receive bias voltages from the global bias generator circuit, which directly impacts the performance of all downstream analog blocks, such as the ADC, DACs, and opamps. Commercial chips usually go through rigorous calibration procedures to set the bias settings, which are then etched into the physical structure of the chip using laser engraving or one-time-programmable memories. This chip does not support one-time-programmable circuits, so the user needs to configure the global bias vector manually. Fortunately, this can be done using only the BIASCR register, which controls the bias levels for all analog circuits, and can disable the global bias generator and bandgap voltage generator as needed. Disabling the global bias generator also switches off all of the on-chip analog blocks, reducing their power consumption such that they only consume leakage power.

To adjust the global bias current setting, modify BIASADJ. Larger values of BIASADJ yield larger currents, and smaller values yield smaller currents, albeit in a somewhat nonlinear fashion. The on-chip analog circuits are designed to have a nominal value of decimal 37 for BIASADJ, but process variations, temperature changes, and power supply voltage deviations may require a different value of BIASADJ to achieve the best results. The user should calibrate the on-chip analog circuits by sweeping BIASADJ and measuring the analog performance at each test vector.

A schematic of the global bias generator is provided in Figure 13.

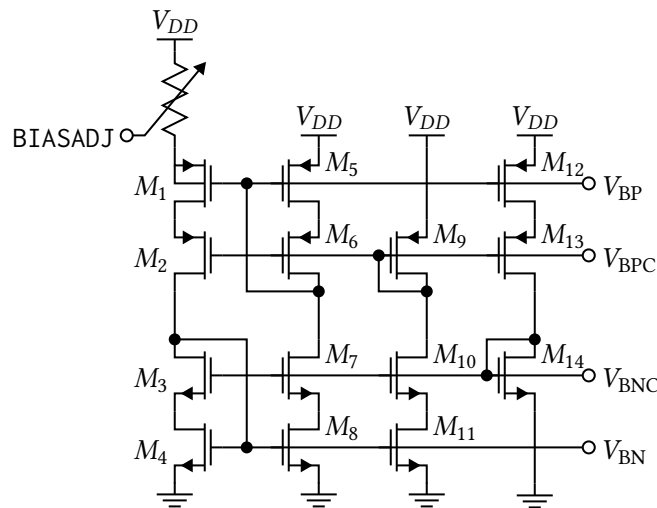


Figure 13: Global Analog Bias Generator Schematic

14.6 Global Bandgap Voltage Generator

The chip also contains a global bandgap voltage generator, which is used as a stable, accurate voltage reference for the ADC and DACs. The bandgap voltage is nominally 1.196 V. The bandgap reference is resistant to process variations and temperature swings, with its output swinging from 1.191 V to 1.203 V across many doping concentration variations and temperatures from -25°C to 75°C . The power supply rejection ratio for the voltage is 34.5 dB. A schematic of the bandgap generator is shown in Figure 14.

The bandgap reference is designed to output a DC voltage independent of the temperature. Unfortunately, most circuit components exhibit a nonzero temperature coefficient (TC), meaning that their voltages and currents are dependent on the temperature. The base-emitter junction voltage V_{BE} of the bipolar transistors Q_1 , Q_2 , and Q_3 each have a negative TC. That is, as temperature increases, V_{BE} decreases ($\partial V_{BE}/\partial T < 0$). The TC for base-emitter junctions is unfortunately nonlinear with respect to T . However,

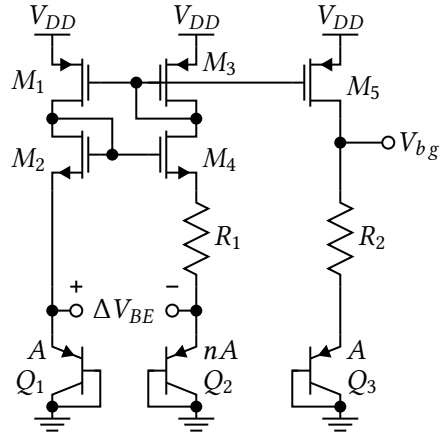


Figure 14: Global Bandgap Voltage Generator Schematic

the voltage *difference* between the base-emitter junctions of two bipolar transistors $\Delta V_{BE} = V_{BE1} - V_{BE2} = V_T \ln n$ is linear with respect to temperature, where $V_T = kT/q$ and $n = J_1/J_2$, the ratio of the current densities. The resulting temperature coefficient is therefore $\partial \Delta V_{BE} / \partial T = (k/q) \ln n$, which is a positive TC and not is dependent on the temperature.

The MOSFETs M_{1-4} enforce identical currents on Q_1 and Q_2 , despite their difference in emitter areas. M_5 mirrors that same current into Q_3 . Q_1 and Q_3 are identically sized, which makes their V_{BE} voltages and current densities equivalent. However, Q_2 has an emitter area n times larger than the other two. As all three emitter currents are identical, the current density of Q_2 is thus n times smaller than the others, giving rise to the equation $\Delta V_{BE} = V_T \ln n$, which has a positive TC. The MOSFETs also form an amplifier in feedback that ensures $V_{S2} = V_{S4}$, making the voltage across R_1 equal to ΔV_{BE} .

Thus, the output voltage is $V_{bg} = V_{BE3} + IR_2$. Note, however, that since the currents through R_1 and R_2 are equivalent and the voltage across R_1 is ΔV_{BE} , the voltage across R_2 can be rewritten as $(R_2/R_1)\Delta V_{BE}$. Hence, the output voltage of the bandgap circuit is $V_{bg} = V_{BE3} + (R_2/R_1)\Delta V_{BE}$, which is nominally about 1.196 V.

To force the TC of the output voltage V_{bg} to zero, the negative TC ($\partial V_{BE3} / \partial T$) needs to cancel the positive TC ($\partial \Delta V_{BE} / \partial T$). As the magnitude of $\partial V_{BE3} / \partial T$ is much larger than $\partial \Delta V_{BE} / \partial T$, R_2/R_1 is sized such that $\partial \Delta V_{BE} / \partial T$ is amplified to be equal to the negative of $\partial V_{BE3} / \partial T$, yielding a zero TC for the output. Finally, it is important to note that while $\partial^2 \Delta V_{BE} / \partial T^2$ is equal to zero, $\partial^2 V_{BE3} / \partial T^2$ is nonzero, which makes the output TC equal to zero at only one temperature. However, this is not a strong influence on the output voltage, so the TC is about equal to zero for a wide range of temperatures centered on room temperature.

14.7 Register Description

14.7.1 SYCLKCR Register

Register memory slot: 0, offset: 0 bytes, size: 2 bytes

System clock control register

15	14	13	12	11	10	9	8
Unused				DCO1OFF	DCO0OFF	HFXTOFF	LFXTOFF
				rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
Unused	SMCLKOFF	Unused	SMCLKSEL		Unused	MCLKSEL	
	rw-(0)		rw-(0)	rw-(0)		rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bits 15-12	
DCO1OFF	Bit 11	Enables/disables the digitally controlled oscillator 1 (DCO1) clock. Cannot be disabled if it is being used as a source to the CPU clock or SMCLK, even if this bit is set to disable mode. 0 Enabled and Powered On 1 Disabled and Powered Down
DCO0OFF	Bit 10	Enables/disables the digitally controlled oscillator 0 (DCO0) clock. Cannot be disabled if it is being used as a source to the CPU clock or SMCLK, even if this bit is set to disable mode. 0 Enabled and Powered On 1 Disabled and Powered Down
HFXTOFF	Bit 9	Enables/disables the high frequency crystal clock. Cannot be disabled if it is being used as a source to the CPU clock or SMCLK, even if this bit is set to disable mode. 0 Enabled 1 Disabled
LFXTOFF	Bit 8	Enables/disables the low frequency crystal clock. Cannot be disabled if it is being used as a source to SMCLK, even if this bit is set to disable mode. 0 Enabled 1 Disabled
Unused	Bit 7	
SMCLKOFF	Bit 6	Enables/disables the submain clock. This is the main vehicle for putting the peripherals to sleep. The MCU is not aware of which peripherals are currently using SMCLK, and in consequence, setting this bit to disable mode will globally and absolutely shut down SMCLK. 0 Enabled 1 Disabled
Unused	Bit 5	
SMCLKSEL	Bits 4-3	Submain clock source select 00 SMCLKSEL_HFXT: High Frequency Crystal Clock 01 SMCLKSEL_LFXT: Low Frequency Crystal Clock

		10	SMCLKSEL_DC00: Digitally Controlled Oscillator 0
		11	SMCLKSEL_DC01: Digitally Controlled Oscillator 1
<i>Unused</i>	Bit 2		
MCLKSEL	Bits 1-0		Main clock source select (also CPU clock source select)
		00	MCLKSEL_HFXT: High Frequency Crystal Clock
		01	MCLKSEL_SMCLK: Submain Clock
		10	MCLKSEL_DC00: Digitally Controlled Oscillator 0
		11	MCLKSEL_DC01: Digitally Controlled Oscillator 1

14.7.2 CLKDIVCR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

MCLK and SMCLK clock divider control register

7	6	5	4	3	2	1	0
<i>Unused</i>		SMCLKDIV			MCLKDIV		
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
<i>Unused</i>	Bits 7-6	
SMCLKDIV	Bits 5-3	SMCLK clock division selection
		000 SMCLKDIV_1: /1 (no division)
		001 SMCLKDIV_2: /2
		010 SMCLKDIV_4: /4
		011 SMCLKDIV_8: /8
		100 SMCLKDIV_16: /16
		101 SMCLKDIV_32: /32
		110 SMCLKDIV_64: /64
		111 SMCLKDIV_128: /128
MCLKDIV	Bits 2-0	MCLK clock division selection
		000 MCLKDIV_1: /1 (no division)
		001 MCLKDIV_2: /2
		010 MCLKDIV_4: /4
		011 MCLKDIV_8: /8
		100 MCLKDIV_16: /16
		101 MCLKDIV_32: /32
		110 MCLKDIV_64: /64
		111 MCLKDIV_128: /128

14.7.3 MEMPWRCR Register

Register memory slot: 2, offset: 8 bytes, size: 2 bytes

Memory power control register

15	14	13	12	11	10	9	8
SRAM15OFF	SRAM14OFF	SRAM13OFF	SRAM12OFF	SRAM11OFF	SRAM10OFF	SRAM09OFF	SRAM08OFF
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
SRAM07OFF	SRAM06OFF	SRAM05OFF	SRAM04OFF	SRAM03OFF	SRAM02OFF	Unused	ROMOFF
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)		rw-(0)

Bitfield	Range	Description
SRAM15OFF	Bit 15	Controls the power state of SRAM15. When off, all memory in SRAM15 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM14OFF	Bit 14	Controls the power state of SRAM14. When off, all memory in SRAM14 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM13OFF	Bit 13	Controls the power state of SRAM13. When off, all memory in SRAM13 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM12OFF	Bit 12	Controls the power state of SRAM12. When off, all memory in SRAM12 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM11OFF	Bit 11	Controls the power state of SRAM11. When off, all memory in SRAM11 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM10OFF	Bit 10	Controls the power state of SRAM10. When off, all memory in SRAM10 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM09OFF	Bit 9	Controls the power state of SRAM09. When off, all memory in SRAM09 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off

SRAM08OFF	Bit 8	Controls the power state of SRAM08. When off, all memory in SRAM08 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM07OFF	Bit 7	Controls the power state of SRAM07. When off, all memory in SRAM07 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM06OFF	Bit 6	Controls the power state of SRAM06. When off, all memory in SRAM06 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM05OFF	Bit 5	Controls the power state of SRAM05. When off, all memory in SRAM05 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM04OFF	Bit 4	Controls the power state of SRAM04. When off, all memory in SRAM04 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM03OFF	Bit 3	Controls the power state of SRAM03. When off, all memory in SRAM03 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
SRAM02OFF	Bit 2	Controls the power state of SRAM02. When off, all memory in SRAM02 becomes undefined, it no longer responds to read or write access, and leakage power consumption decreases. 0 Power on 1 Power off
<i>Unused</i>	Bit 1	
ROMOFF	Bit 0	Controls the power state of the boot ROM. When off, it no longer responds to read access, and leakage power consumption decreases. 0 Power on 1 Power off

14.7.4 CRCDATA Register

Register memory slot: 3, offset: 12 bytes, size: 1 byte

CRC input data register. Write the next byte of the data array to this register to continue calculating the CRC value of the data array.

7	6	5	4	3	2	1	0
CRCDATA							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.5 CRCSTATE Register

Register memory slot: 4, offset: 16 bytes, size: 2 bytes

CRC state register. Read this register to get the output of the CRC calculation. Write to this register to set the initial value of the CRC. Note that the value of this register is undefined during the time between when it is written to until when CRCDATA is written to.

15	14	13	12	11	10	9	8
CRCSTATE							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
CRCSTATE							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.6 IRQEN Register

Register memory slot: 5, offset: 20 bytes, size: 4 bytes

IRQ enable register. Each bit enables the associated IRQ when 1, and disables it when 0. The bit number corresponds to the IRQ vector number.

31	30	29	28	27	26	25	24
IRQEN							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
IRQEN							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
IRQEN							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
IRQEN							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.7 IRQPRI Register

Register memory slot: 6, offset: 24 bytes, size: 4 bytes

IRQ priority register. Each bit sets the associated IRQ to the higher priority tier when 1, and sets it to the lower priority tier when 0. The bit number corresponds to the IRQ vector number. IRQ priority is determined first by the value of IRQPRI and then by the IRQ vector order. An IRQ with an associated

IRQPRI bit of 0 will always have higher priority than one with an IRQPRI bit of 1. If two IRQs have the same value of IRQPRI bits, then the one with the smaller IRQ vector number has greater priority.

31	30	29	28	27	26	25	24
IRQPRI							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
IRQPRI							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
IRQPRI							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
IRQPRI							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.8 WDTPASS Register

Register memory slot: 7, offset: 28 bytes, size: 4 bytes

Watchdog timer password register. To unlock the WDTCSR register for writing, first write 0x3FB0AD1C to WDTPASS. After that, the WDTCSR register will be unlocked for 64 MCLK cycles during which it can be written to. Outside this time, any attempts to write to WDTCSR will have no effect. To reset the watchdog timer counter, write 0xD6F402BC to WDTPASS. If the watchdog timer is enabled and reaches its maximum value determined by WDTCDIV, it will reset the system. Reading this register always yields 0.

31	30	29	28	27	26	25	24
WDTPASS							
w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)
23	22	21	20	19	18	17	16
WDTPASS							
w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)
15	14	13	12	11	10	9	8
WDTPASS							
w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)
7	6	5	4	3	2	1	0
WDTPASS							
w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)	w-(0)

14.7.9 WDTCSR Register

Register memory slot: 8, offset: 32 bytes, size: 1 byte

Watchdog timer control register. This register is normally read-only, and must first be unlocked to write to it. To unlock the register, write 0x3FB0AD1C to the WDTPASS register. After that, the WDTCR register will be unlocked for write access for 64 MCLK cycles, and then return to being read-only.

7	6	5	4	3	2	1	0
Unused	HWRST	WDTCDIV				WDTIE	WDTREN
	w-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bit 7	
HWRST	Bit 6	Hardware reset. Write a 1 to this bit to perform a hardware reset of the system. 0 Does nothing 1 Systemwide hardware reset
WDTCDIV	Bits 5-2	Watchdog timer clock divider. Determines the number of MCLK cycles before the watchdog timer resets the system when enabled as $2^{(WDTCDIV + 16)}$ 0000 WDTCDIV_65536: /65,536 0001 WDTCDIV_131072: /131,072 0010 WDTCDIV_262144: /262,144 0011 WDTCDIV_524288: /524,288 0100 WDTCDIV_1048576: /1,048,576 0101 WDTCDIV_2097152: /2,097,152 0110 WDTCDIV_4194304: /4,194,304 0111 WDTCDIV_8388608: /8,388,608 1000 WDTCDIV_16777216: /16,777,216 1001 WDTCDIV_33554432: /33,554,432 1010 WDTCDIV_67108864: /67,108,864 1011 WDTCDIV_134217728: /134,217,728 1100 WDTCDIV_268435456: /268,435,456 1101 WDTCDIV_536870912: /536,870,912 1110 WDTCDIV_1073741824: /1,073,741,824 1111 WDTCDIV_2147483648: /2,147,483,648
WDTIE	Bit 1	Watchdog timer interrupt enable. The watchdog timer begins counting up from 0 immediately after being enabled, which is achieved by either setting WDTREN or WDTIE. When the watchdog timer interrupt is enabled, if the watchdog timer reaches its maximum value determined by WDTCDIV, then the SYSTEM interrupt will execute. Upon its return, the MCU will be reset if WDTREN is enabled. The SYSTEM IRQ must also be enabled in IRQEN. 0 Disabled 1 Enabled

WDTREN	Bit 0	Watchdog timer reset enable. The watchdog timer begins counting up from 0 immediately after being enabled, which is achieved by either setting WDTREN or WDTIE. If the watchdog timer is enabled and reaches its maximum value determined by WDTCDIV, it will reset the system. Disabling the watchdog timer also resets its internal counter.
	0	Disabled
	1	Enabled

14.7.10 WDTSR Register

Register memory slot: 9, offset: 36 bytes, size: 1 byte

Watchdog timer status register.

7	6	5	4	3	2	1	0
Unused						WDTIF	WDTRF
						rw1-(0)	rw1-(0)

Bitfield	Range	Description
Unused	Bits 7-2	
WDTIF	Bit 1	Watchdog timer interrupt flag. Indicates that the watchdog timer has reached its maximum value, but the system has not yet been reset. To clear the bit, write a 1 to it.
	0	No pending interrupt
	1	Interrupt pending
WDTRF	Bit 0	Watchdog timer reset flag. Indicates that the previous systemwide reset was generated by the watchdog timer. To clear the bit, write a 1 to it.
	0	Previous reset was not generated by the watchdog timer
	1	Previous reset was generated by the watchdog timer

14.7.11 WDTVAl Register

Register memory slot: 10, offset: 40 bytes, size: 4 bytes

Watchdog timer value register. Contains the current value of the watchdog timer counter value. Reads as 0 if the watchdog timer is disabled.

31	30	29	28	27	26	25	24
WDTVAL							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
23	22	21	20	19	18	17	16
WDTVAL							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
15	14	13	12	11	10	9	8
WDTVAL							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
7	6	5	4	3	2	1	0
WDTVAL							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

14.7.12 DCO0FREQ Register

Register memory slot: 11, offset: 44 bytes, size: 2 bytes

DCO0 non-stabilized frequency register (manual frequency adjustment)

15	14	13	12	11	10	9	8
Unused				DCO0MFREQ			
				rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
DCO0MFREQ							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bits 15-12	
DCO0MFREQ	Bits 11-0	12-bit manual frequency adjustment

14.7.13 DCO1FREQ Register

Register memory slot: 12, offset: 48 bytes, size: 2 bytes

DCO1 non-stabilized frequency register (manual frequency adjustment)

15	14	13	12	11	10	9	8
Unused				DCO1MFREQ			
				rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
DCO1MFREQ							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
<i>Unused</i>	Bits 15-12	
DC01MFREQ	Bits 11-0	12-bit manual frequency adjustment

14.7.14 TPMR Register

Register memory slot: 13, offset: 52 bytes, size: 1 byte

Analog and digital test port multiplexer register

7	6	5	4	3	2	1	0
<i>Unused</i>				TPMR			
				rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.15 BIASCR Register

Register memory slot: 14, offset: 56 bytes, size: 2 bytes

Global bias and bandgap generator control register

15	14	13	12	11	10	9	8
<i>Unused</i>					EnBG	UseExtBias	UseBiasDac
					rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
EnBiasBuf	EnBiasGen	BiasAdj					
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
<i>Unused</i>	Bits 15-11	
EnBG	Bit 10	Enables/disables the internal bandgap generator at 1.196 V 0 Disabled 1 Enabled
UseExtBias	Bit 9	Enables/disables external bias voltages. Note that this bit does not enable or disable the internal bias generator or DACs. Those must be toggled separately. 0 Disabled 1 Enabled
UseBiasDac	Bit 8	Switches between using the bias generator voltages or bias DACs for the global bias voltages. 0 Uses bias generator 1 Uses DACs
EnBiasBuf	Bit 7	Enables/disables buffers on the internal global bias voltages. 0 Disabled 1 Enabled
EnBiasGen	Bit 6	Enables/disables the internal bias generator. 0 Disabled

1	Enabled
---	---------

BiasAdj	Bits 5-0	Internal bias generator adjustment vector. Higher vector codes produce smaller currents. The nominal vector from simulation is decimal 37.
---------	----------	--

14.7.16 BIASDBP Register

Register memory slot: 15, offset: 60 bytes, size: 2 bytes

Global bias P DAC register

15	14	13	12	11	10	9	8
Unused		BIASDBP					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
BIASDBP							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.17 BIASDBPC Register

Register memory slot: 16, offset: 64 bytes, size: 2 bytes

Global bias P cascode DAC register

15	14	13	12	11	10	9	8
Unused		BIASDBPC					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
BIASDBPC							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.18 BIASDBNC Register

Register memory slot: 17, offset: 68 bytes, size: 2 bytes

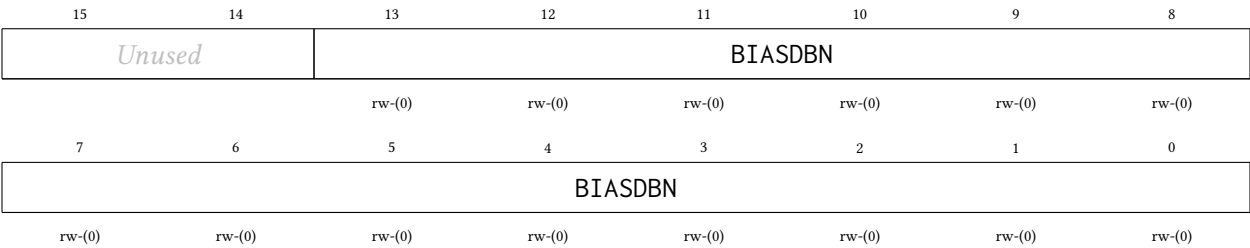
Global bias N cascode DAC register

15	14	13	12	11	10	9	8
Unused		BIASDBNC					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
BIASDBNC							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

14.7.19 BIASDBN Register

Register memory slot: 18, offset: 72 bytes, size: 2 bytes

Global bias N DAC register



15 NNx Peripheral

Fixed point artificial neural network hardware accelerator. The peripheral calculates a single layer of a multilayer perceptron neural network. The input is a vector of signed Q0.15 integers (signed int16), and the output is a vector of signed Q0.15 integers (signed int16), and the weights are 24-bit signed Q8.15 numbers (signed int24). It can be configured to use a linear activation function or a logistic sigmoid approximation activation function. The input vector, output vector, and weights must all fit inside a single 16 KiB multiplexed SRAM.

15.1 Register Description

15.1.1 NNCR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

Neural network control register

31	30	29	28	27	26	25	24
Unused							
23	22	21	20	19	18	17	16
Unused	CIE	CIF	LSIS	CS	BIAS	AFS	RUN
	rw-(0)	rw0-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw1-(0)
15	14	13	12	11	10	9	8
0							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
I							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

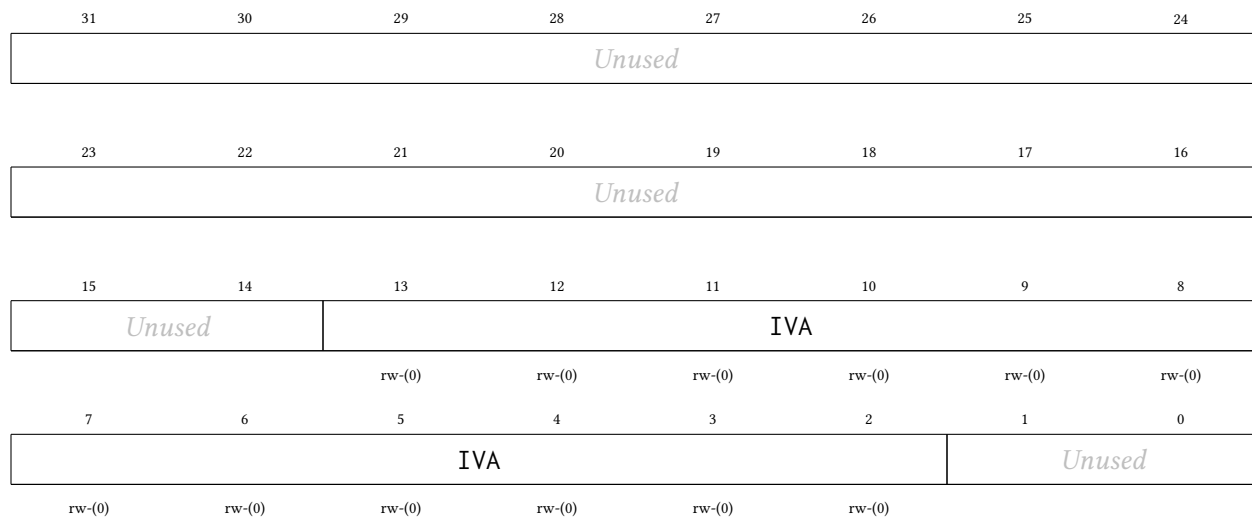
Bitfield	Range	Description
Unused	Bits 31-23	
CIE	Bit 22	Neural network layer complete interrupt enable 0 Disabled 1 Enabled
CIF	Bit 21	Neural network layer complete interrupt flag. Write a 0 to clear the flag. 0 No interrupt pending 1 Interrupt pending
LSIS	Bit 20	Logistic sigmoid input select. Used for testing the hardware logistic sigmoid approximation activation function. 0 Internal input (required for neural network usage) 1 Input is NNxLSI (used for testing)
CS	Bit 19	Neural network clock select 0 SMCLK 1 MCLK

BIAS	Bit 18	Neural network layer bias enable, used to insert a bias from the synaptic weights into the input of the activation function 0 Disabled 1 Enabled
AFS	Bit 17	Activation function select 0 Logistic sigmoid approximation 1 Linear
RUN	Bit 16	Neural network run. Set to start the neural network accelerator process. Read to determine if the neural network accelerator is running or not. 0 Not running 1 Running (write 1 to start)
O	Bits 15-8	Number of neural network outputs in output vector minus 1. The actual number of outputs is NNO + 1
I	Bits 7-0	Number of neural network inputs in input vector minus 1. The actual number of inputs is NNI + 1.

15.1.2 NNIVA Register

Register memory slot: 1, offset: 4 bytes, size: 4 bytes

Input vector start address. Must be contained within the appropriate multiplexed SRAM, and must be on a 4-byte boundary. Each input in the input vector is a signed int16 (signed Q0.15) number. Inputs in the input vector must be stored as follows. The input at index 0 must be stored with its LSbyte at NNxIVA + 0 (in bytes) and its MSbyte at NNxIVA + 1 (in bytes). The input at index 1 must be stored with its LSbyte at NNxIVA + 2 (in bytes) and its MSbyte at NNxIVA + 3 (in bytes). The rest of the inputs follow the same sequence.

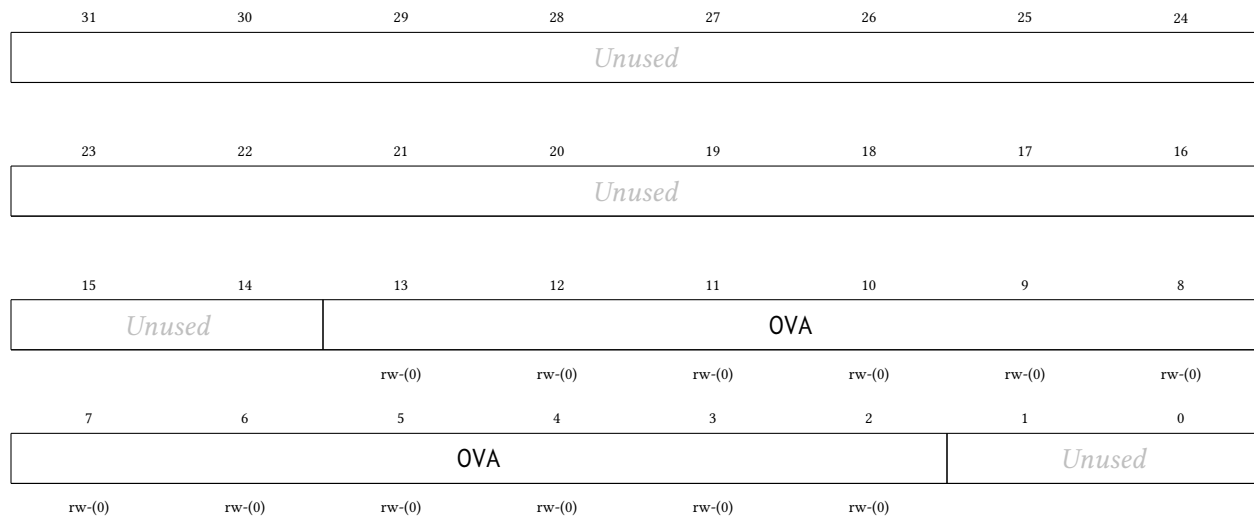


Bitfield	Range	Description
Unused	Bits 31-14	
IVA	Bits 13-2	Input vector start address (divided by 4)
Unused	Bits 1-0	

15.1.3 NNOVA Register

Register memory slot: 2, offset: 8 bytes, size: 4 bytes

Output vector start address. Must be contained within the appropriate multiplexed SRAM, and must be on a 4-byte boundary. Each output in the output vector is a signed int16 (signed Q0.15) number. Outputs in the output vector are stored as follows. The output at index 0 is stored with its LSbyte at $NNxOVA + 0$ (in bytes) and its MSbyte at $NNxOVA + 1$ (in bytes). The output at index 1 is stored with its LSbyte at $NNxOVA + 2$ (in bytes) and its MSbyte at $NNxOVA + 3$ (in bytes). The rest of the outputs follow the same sequence.

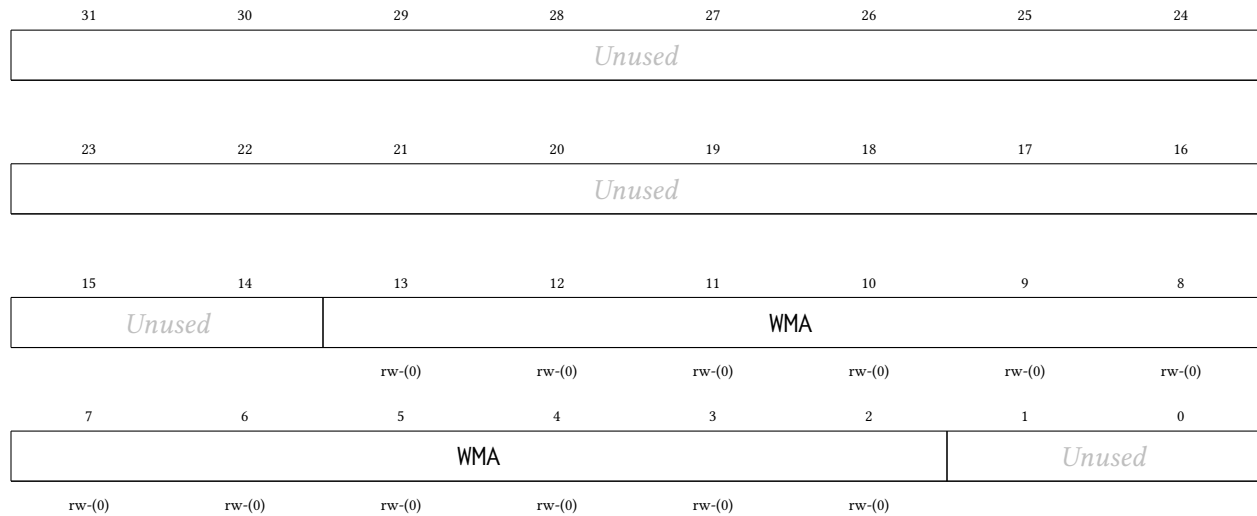


Bitfield	Range	Description
<i>Unused</i>	Bits 31-14	
OVA	Bits 13-2	Output vector start address (divided by 4)
<i>Unused</i>	Bits 1-0	

15.1.4 NNWMA Register

Register memory slot: 3, offset: 12 bytes, size: 4 bytes

Synaptic weights matrix start address. Must be contained within the appropriate multiplexed SRAM, and must be on a 4-byte boundary. Each synaptic weight in the synaptic weights matrix is a signed int24 (signed Q8.15) number. Note that bit 23 of each weight is the sign bit. Weights in the weights matrix are stored as follows. The weight at index 0 is stored with its LSbyte at $NNxWMA + 0$ (in bytes), its center byte at $NNxWMA + 1$ (in bytes), and its MSbyte at $NNxWMA + 2$ (in bytes). The weight at index 1 is stored with its LSbyte at $NNxWMA + 3$ (in bytes), its center byte at $NNxWMA + 4$ (in bytes), and its MSbyte at $NNxWMA + 5$ (in bytes). The weight at index 2 is stored with its LSbyte at $NNxWMA + 6$ (in bytes), its center byte at $NNxWMA + 7$ (in bytes), and its MSbyte at $NNxWMA + 8$ (in bytes). The weight at index 3 is stored with its LSbyte at $NNxWMA + 9$ (in bytes), its center byte at $NNxWMA + 10$ (in bytes), and its MSbyte at $NNxWMA + 11$ (in bytes). The rest of the weights follow the same sequence.

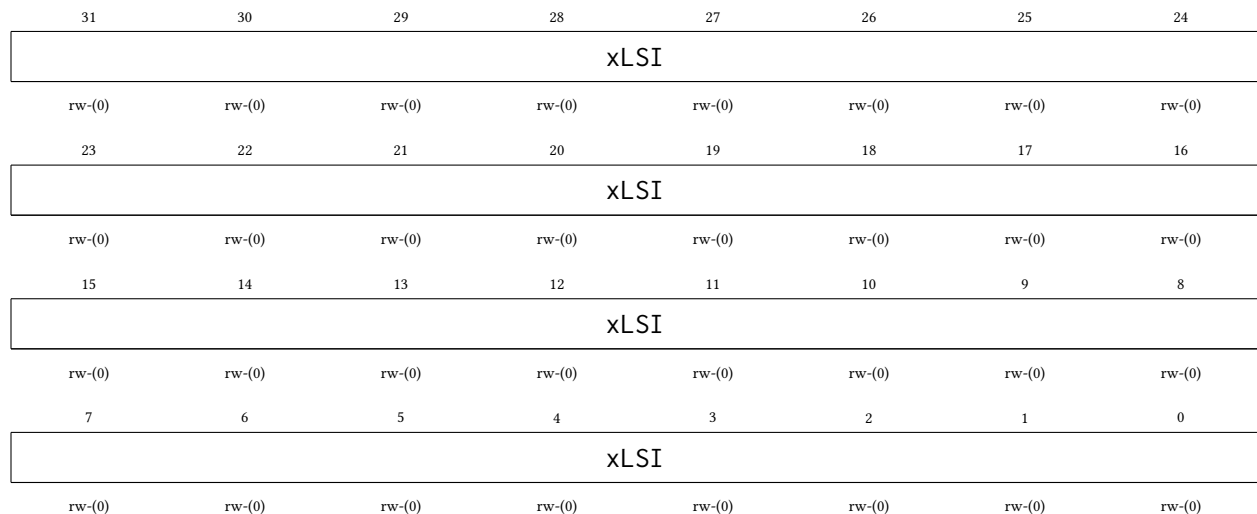


Bitfield	Range	Description
<i>Unused</i>	Bits 31-14	
WMA	Bits 13-2	Synaptic weights matrix start address (divided by 4)
<i>Unused</i>	Bits 1-0	

15.1.5 NNLSI Register

Register memory slot: 4, offset: 16 bytes, size: 4 bytes

Logistic sigmoid input, which is a signed int32 (signed Q8.15) number. Used for testing the hardware logistic sigmoid approximation activation function. When NNLSIS is set, NNxLSO will contain the output of the hardware logistic sigmoid approximation with NNxLSI as its input.



15.1.6 NNLSO Register

Register memory slot: 5, offset: 20 bytes, size: 2 bytes

Logistic sigmoid output, which is a signed int16 (signed Q0.15) number. Used for testing the hardware logistic sigmoid approximation activation function. When NNLSIS is set, NNxLSO will contain the output of the hardware logistic sigmoid approximation with NNxLSI as its input.

15	14	13	12	11	10	9	8
xLSO							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
xLSO							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

16 SARADCx Peripheral

Successive Approximation Register Analog-to-Digital Converter with 4 channels

16.1 Register Description

16.1.1 SARADCCR Register

Register memory slot: 0, offset: 0 bytes, size: 2 bytes

SAR ADC control register

15	14	13	12	11	10	9	8
<i>Unused</i>							CONTMEAS
							rw-(0)
7	6	5	4	3	2	1	0
DATAIE	DEBUG	EN	SAMPLESTEP			RESET	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
<i>Unused</i>	Bits 15-9	
CONTMEAS	Bit 8	Continuous measurement mode enable 0 Disabled 1 Enabled
DATAIE	Bit 7	Data valid interrupt enable 0 Interrupt disabled 1 Interrupt enabled
DEBUG	Bit 6	Debug mode 0 Normal operation 1 Debug mode
EN	Bit 5	ADC enable 0 Disabled 1 Enabled
SAMPLESTEP	Bits 4-1	Sampling step/pulse width
RESET	Bit 0	ADC reset 0 Normal operation 1 Reset

16.1.2 SARADCCDIV Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

SAR ADC clock divider register

7	6	5	4	3	2	1	0
CDIV							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
CDIV	Bits 7-0	Clock divider value

16.1.3 SARADCSR Register

Register memory slot: 2, offset: 8 bytes, size: 1 byte

SAR ADC status register

7	6	5	4	3	2	1	0
Unused				RDY	OVF	DATAVALID	BUSY
				r-(0)	rw1-(0)	r-(0)	r-(0)

Bitfield	Range	Description
Unused	Bits 7-4	
RDY	Bit 3	ADC ready 0 ADC not ready 1 ADC ready
OVF	Bit 2	ADC overflow interrupt flag 0 No overflow 1 Overflow occurred
DATAVALID	Bit 1	Data valid flag 0 Data not valid 1 Data valid
BUSY	Bit 0	Conversion busy 0 Idle 1 Busy

16.1.4 SARADCDATA Register

Register memory slot: 3, offset: 12 bytes, size: 2 bytes

SAR ADC data register

15	14	13	12	11	10	9	8
Unused						DATA	
						r-(0)	r-(0)
7	6	5	4	3	2	1	0
DATA							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

Bitfield	Range	Description
Unused	Bits 15-10	
DATA	Bits 9-0	ADC conversion result (10-bit)

16.1.5 SARADCTPR Register

Register memory slot: 4, offset: 16 bytes, size: 1 byte
SAR ADC test port register



Bitfield	Range	Description
DTP1SEL	Bits 7-4	Digital test port 1 select
DTP0SEL	Bits 3-0	Digital test port 0 select

17 AFEx Peripheral

Analog front-end interface. Capable of pulse height analysis (PHA) and pulse shape discrimination (PSD). Features a DMA to place PHA and PSD data directly into RAM without CPU intervention.

17.1 Register Description

17.1.1 AFECR0 Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

AFE control register 0

31	30	29	28	27	26	25	24
<i>Unused</i>		AdcExtIn	AdcMidSel	AdcMuxTest	BufCMMid	CsaBiasSel	CsaForceRst
rw-(0)		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
CsaForceUnRst	CsaRstMode	EnAfe	EnCM	EnCsa	EnAdc	EnThresh	EnDma
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
EnPUREj	EnPsd	EnBLLT	ForceThresh	OpenInput	PsdOrder	PUPara	RamOff
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
RejectMode	SHPwrMode	ThreshSel		PulseDoneWal	PulseDoneIE	AdcConvDoneIF	PsdFullIE
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
<i>Unused</i>	Bits 31-30	
AdcExtIn	Bit 29	
AdcMidSel	Bit 28	
AdcMuxTest	Bit 27	
BufCMMid	Bit 26	
CsaBiasSel	Bit 25	
CsaForceRst	Bit 24	
CsaForceUnRst	Bit 23	
CsaRstMode	Bit 22	
EnAfe	Bit 21	
EnCM	Bit 20	
EnCsa	Bit 19	
EnAdc	Bit 18	
EnThresh	Bit 17	
EnDma	Bit 16	
EnPUREj	Bit 15	
EnPsd	Bit 14	
EnBLLT	Bit 13	

ForceThresh	Bit 12
OpenInput	Bit 11
PsdOrder	Bit 10
PUPara	Bit 9
RamOff	Bit 8
RejectMode	Bit 7
SHPwrMode	Bit 6
ThreshSel	Bits 5-4
PulseDoneWait	Bit 3
PulseDoneIE	Bit 2
AdcConvDoneIE	Bit 1
PsdFullIE	Bit 0

17.1.2 AFECR1 Register

Register memory slot: 1, offset: 4 bytes, size: 4 bytes

AFE control register 1

31	30	29	28	27	26	25	24
AdcClkDivN			AdcClkDivM			AdcCp	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
AdcCp						AdcSampT	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
AdcSampT	ClkSel		CMShClkDiv			CsaCmAdj	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
CsaCmAdj		ISink					
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
AdcClkDivN	Bits 31-29	
AdcClkDivM	Bits 28-25	
AdcCp	Bits 24-18	
AdcSampT	Bits 17-15	
ClkSel	Bits 14-13	
CMShClkDiv	Bits 12-9	
CsaCmAdj	Bits 8-6	
ISink	Bits 5-0	

17.1.3 AFECFB Register

Register memory slot: 2, offset: 8 bytes, size: 1 byte

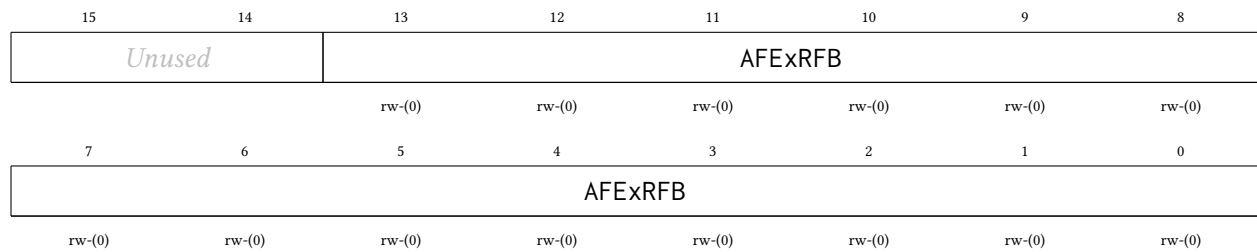
CSA feedback capacitance register. Increment is 250 fF. A value of 0 gives 0 fF (Cfb disconnected).



17.1.4 AFERFB Register

Register memory slot: 3, offset: 12 bytes, size: 2 bytes

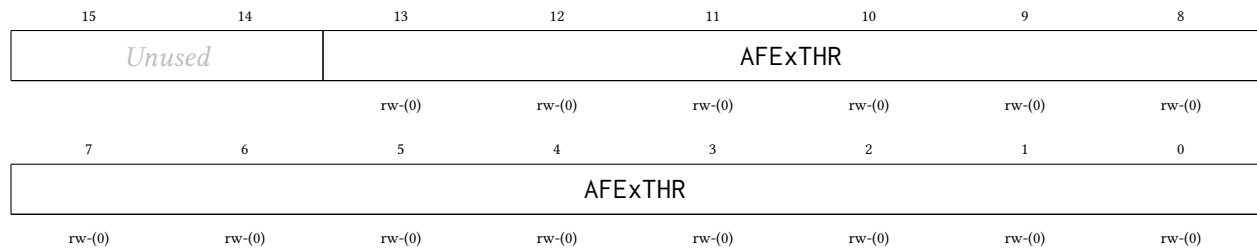
CSA feedback resistance DAC register. Increasing values yield smaller resistances.



17.1.5 AFETHR Register

Register memory slot: 4, offset: 16 bytes, size: 2 bytes

Threshold DAC register



17.1.6 AFETPR Register

Register memory slot: 5, offset: 20 bytes, size: 4 bytes

AFE test port register

31	30	29	28	27	26	25	24
<i>Unused</i>		Atp1BufEn	Atp1Sel			Atp0BufEn	
rw-(0)		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
23	22	21	20	19	18	17	16
Atp0Sel				Dtp3Sel			
rw-(0)		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
Dtp3Sel	Dtp2Sel					Dtp1Sel	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
Dtp1Sel			Dtp0Sel				
rw-(0)		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
<i>Unused</i>	Bits 31-30	
Atp1BufEn	Bit 29	
Atp1Sel	Bits 28-25	
Atp0BufEn	Bit 24	
Atp0Sel	Bits 23-20	
Dtp3Sel	Bits 19-15	
Dtp2Sel	Bits 14-10	
Dtp1Sel	Bits 9-5	
Dtp0Sel	Bits 4-0	

17.1.7 AFESPT Register

Register memory slot: 6, offset: 24 bytes, size: 1 byte

Smallest pulse time register

7	6	5	4	3	2	1	0
AFExSPT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.8 AFEPIT Register

Register memory slot: 7, offset: 28 bytes, size: 2 bytes

PHA integration time register. When this timer elapses, the PHA peak sample and hold latches the output of the CSA and allows the ADC to begin converting it. If AfeEnPsd is 0, the timer starts immediately after a valid new pulse is detected. If AfeEnPsd is 1 and AfePsdOrder is 0, the timer starts after the early PSD integration timer elapses. If AfeEnPsd is 1 and AfePsdOrder is 1, it starts after the late PSD integration timer elapses.

15	14	13	12	11	10	9	8
AFExPIT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExPIT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.9 AFEEIT Register

Register memory slot: 8, offset: 32 bytes, size: 2 bytes

PSD early integration time register. When this timer elapses, the early PSD peak sample and hold latches the output of the CSA and allows the ADC to begin converting it. If AfeEnPsd is 0, the timer is disabled. If 1, the timer starts immediately after a valid new pulse is detected.

15	14	13	12	11	10	9	8
AFExEIT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExEIT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.10 AFELIT Register

Register memory slot: 9, offset: 36 bytes, size: 2 bytes

PSD late integration time register. When this timer elapses, the late PSD peak sample and hold latches the output of the CSA and allows the ADC to begin converting it. If AfeEnPsd is 0, the timer is disabled. If AfeEnPsd is 1 and AfePsdOrder is 1, the timer starts after the early PSD integration timer elapses. If AfeEnPsd is 1 and AfePsdOrder is 0, it starts after the PHA integration timer elapses.

15	14	13	12	11	10	9	8
AFExLIT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExLIT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.11 AFERJT Register

Register memory slot: 10, offset: 40 bytes, size: 2 bytes

Reject time register.

15	14	13	12	11	10	9	8
AFExRJT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExRJT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.12 AFERST Register

Register memory slot: 11, offset: 44 bytes, size: 1 byte

CSA reset time register. Determines the amount of time the CSA is held in reset after the reject timer has elapsed.

7	6	5	4	3	2	1	0
AFExRST							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.13 AFExAOFST Register

Register memory slot: 12, offset: 48 bytes, size: 2 bytes

ADC offset DAC register. Centers the CSA swing on the common mode voltage of the ADC by setting the ADC reference voltage Vref. The ADC buffer converts the differential inputs VInP and VInM to a single-ended signal referenced to VSS with the equation $V_{buf} = (V_{InP} - V_{InM}) + V_{ofst}$

15	14	13	12	11	10	9	8
Unused		AFExAOFST					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExAOFST							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.14 AFExBLLT Register

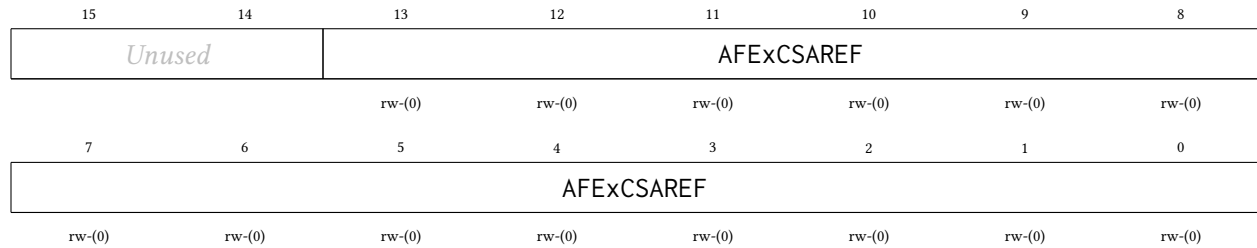
Register memory slot: 13, offset: 52 bytes, size: 2 bytes

Baseline lower threshold DAC register. The baseline lower threshold is used to reset the CSA if its output dips below the threshold, acting as a form of baseline restoration.

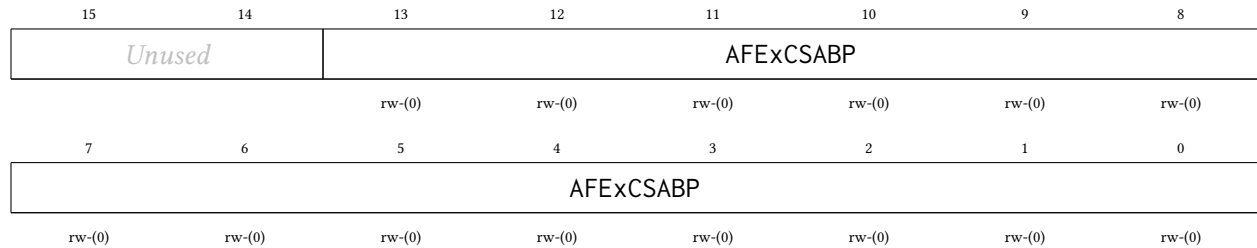
15	14	13	12	11	10	9	8
Unused		AFExBLLT					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExBLLT							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.15 AFECsAREF Register*Register memory slot: 14, offset: 56 bytes, size: 2 bytes*

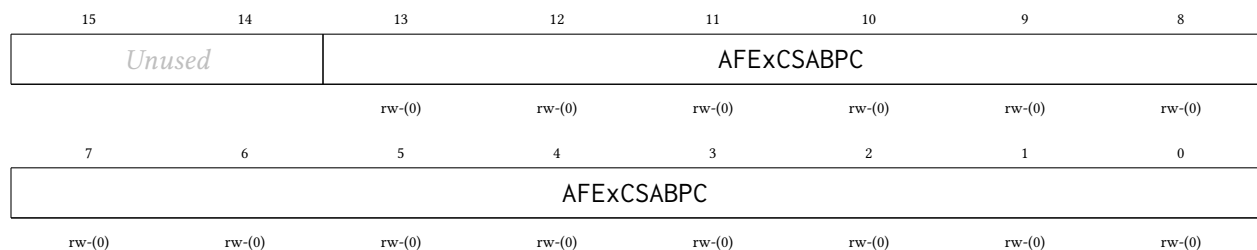
CSA reference DAC register

**17.1.16 AFECsABP Register***Register memory slot: 15, offset: 60 bytes, size: 2 bytes*

CSA bias P DAC register

**17.1.17 AFECsABPC Register***Register memory slot: 16, offset: 64 bytes, size: 2 bytes*

CSA bias P cascode DAC register

**17.1.18 AFECsABNC Register***Register memory slot: 17, offset: 68 bytes, size: 2 bytes*

CSA bias N cascode DAC register

15	14	13	12	11	10	9	8
<i>Unused</i>		AFExCSABNC					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExCSABNC							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.19 AFExSABN Register

Register memory slot: 18, offset: 72 bytes, size: 2 bytes

CSA bias N DAC register

15	14	13	12	11	10	9	8
<i>Unused</i>		AFExCSABN					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExCSABN							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.20 AFExCMSHR Register

Register memory slot: 19, offset: 76 bytes, size: 2 bytes

Current mirror sample and hold reference DAC register

15	14	13	12	11	10	9	8
<i>Unused</i>		AFExCMSHR					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
AFExCMSHR							
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.21 AFExCLPF Register

Register memory slot: 20, offset: 80 bytes, size: 1 byte

Current mirror low pass filter capacitance adjust register

7	6	5	4	3	2	1	0
<i>Unused</i>		AFExCLPF					
		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

17.1.22 AFESR Register

Register memory slot: 21, offset: 84 bytes, size: 1 byte

AFE status register

7	6	5	4	3	2	1	0
DTP1VAL	DTP0VAL	AdcActive	AdcDataReady	DmaEnabled	PulseDone	AdcConvDone	PsdFull
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
DTP1VAL	Bit 7	The current value of digital test port 1
DTP0VAL	Bit 6	The current value of digital test port 0
AdcActive	Bit 5	Indicates that the ADC is currently active either sampling or converting 0 ADC inactive 1 ADC active
AdcDataReady	Bit 4	
DmaEnabled	Bit 3	
PulseDone	Bit 2	
AdcConvDone	Bit 1	
PsdFull	Bit 0	PSD memory full interrupt flag. Write a 1 to this bit to clear the flag and allow the AFE finite state machine to overwrite the memory with new PSD data.

17.1.23 AFEADCVAL Register

Register memory slot: 22, offset: 88 bytes, size: 2 bytes

ADC value register. Shows the value of the most recent ADC conversion.

15	14	13	12	11	10	9	8
Unused						AFExADCVAL	
						r-(0)	r-(0)
7	6	5	4	3	2	1	0
AFExADCVAL							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

17.1.24 AFEVPC Register

Register memory slot: 23, offset: 92 bytes, size: 4 bytes

Valid pulse count register. Write anything to this register to clear both AFExVPC and AFExTPC.

31	30	29	28	27	26	25	24
AFExVPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
23	22	21	20	19	18	17	16
AFExVPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
15	14	13	12	11	10	9	8
AFExVPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
7	6	5	4	3	2	1	0
AFExVPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

17.1.25 AFETPC Register

Register memory slot: 24, offset: 96 bytes, size: 4 bytes

Total pulse count register. Write anything to this register to clear both AFExVPC and AFExTPC.

31	30	29	28	27	26	25	24
AFExTPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
23	22	21	20	19	18	17	16
AFExTPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
15	14	13	12	11	10	9	8
AFExTPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)
7	6	5	4	3	2	1	0
AFExTPC							
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

18 I2Cx Peripheral

I2C serial port interface. The master and slave I2C interfaces are split between two sets of registers.

To use master transmitter mode, first configure the I2C peripheral by setting I2CMEN, clearing I2CSEN, and configuring I2CMDIV with the appropriate clock division factor, noting that the I2C clock source is SMCLK. To send a start condition, set I2CMST, wait for the I2CMSTS flag to be set (if the bus is busy, the I2C peripheral will wait for it to become idle and then send a start condition), and then clear the status register. I2CMCB will now indicate that the I2C peripheral now has control of the bus as its master. Next, write to I2CxMTX the desired slave address in the most significant 7 bits followed by the desired read/write bit (0 for write/master transmitter) in the least significant bit. Then, wait for I2CMXC or I2CMARB to be set. If I2CMARB is set, then the I2C peripheral has lost the bus arbitration contest and has released control of the bus. If I2CMNR is set, then the desired slave has not acknowledged itself. Clear the status register. Next, send the slave a byte of data by writing the desired data to I2CxMTX. When the I2C peripheral is ready for another byte of data to be queued for transmission, the I2CMTXE flag will be set. Again, wait for I2CMXC or I2CMARB, then check I2CMARB and I2CMNR, and finally clear the status register. Once finished sending all of the desired bytes, either send a stop condition to release control of the bus by setting I2CMSP, or send a repeated start condition to retain control of the bus with a new transmission (and possibly a new slave and read/write mode) by setting I2CMST. Once a stop condition is sent, wait for I2CMSTS to be set, indicating that a stop condition has been sent. Clear the status register.

To use master receiver mode, first configure the I2C peripheral by setting I2CMEN, clearing I2CSEN, and configuring I2CMDIV with the appropriate clock division factor, noting that the I2C clock source is SMCLK. To send a start condition, set I2CMST, wait for the I2CMSTS flag to be set (if the bus is busy, the I2C peripheral will wait for it to become idle and then send a start condition), and then clear the status register. I2CMCB will now indicate that the I2C peripheral now has control of the bus as its master. Next, write to I2CxMTX the desired slave address in the most significant 7 bits followed by the desired read/write bit (1 for read/master receiver) in the least significant bit. Then, wait for I2CMXC or I2CMARB to be set. If I2CMARB is set, then the I2C peripheral has lost the bus arbitration contest and has released control of the bus. If I2CMNR is set, then the desired slave has not acknowledged itself. Clear the status register. Next, begin to receive a byte of data from the slave by setting I2CMRB. Wait for I2CMXC to be set. Clear the status register. Read I2CxMRX to get the byte of data received from the slave. To send the slave an ACK and begin to read another byte from the slave, set I2CMRB. Or, to send the slave a NACK and send a stop condition, set I2CMSP. Or, to send the slave a NACK and send a repeated start condition, set I2CMST. Wait for the appropriate flag, then clear the status register.

To use slave receiver mode, first configure the I2C peripheral by setting I2CSEN, clearing I2CSEN, clearing I2CSN, and configuring I2CSCS and I2CGCE to the desired values. Note that if clock stretching is enabled with I2CSCS, the I2C peripheral will seize control of the bus by driving SCL low during every ACK/NACK bit transfer (if the I2C peripheral was addressed) until I2CSC is set, which requires user intervention to prevent indefinite hold-ups of the I2C bus. But, if clock stretching is not enabled, the master will be allowed full control of the rate data is sent over the bus, which opens the possibility that the software running on this MCU does not notice that a byte has been transferred in time before the next is transferred. Note that if I2CGCE is set, the I2C peripheral will be addressed if either its address is received or if the general call is received. Wait for the I2C peripheral to be addressed when I2CSA is set. Check I2CSTM to see if the master has requested slave receiver or slave transmitter mode (0 indicates slave receiver). Clear the status register. If clock stretching is enabled, send an ACK or NACK by clearing or setting I2CSN, and then set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Next, wait for the slave to receive a data byte from the master when I2CSXC is set. If I2CSOVF is set, then the MCU has failed to read one of the bytes sent by the master in the past. Clear the status register, and then read I2CSRX to get

the data byte sent from the master. If clock stretching is enabled, send an ACK or NACK by clearing or setting I2CSN, and then set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Next, wait for I2CSXC, I2CSPR, or I2CSTR to be set, indicating the I2C peripheral has received a new byte of data, a stop condition, or a repeated start condition. If a stop or start condition has been received, clear the status register.

To use slave transmitter mode, first configure the I2C peripheral by setting I2CSEN, clearing I2CSEN, clearing I2CSN, and configuring I2CSCS and I2CGCE to the desired values. Note that if clock stretching is enabled with I2CSCS, the I2C peripheral will seize control of the bus by driving SCL low during every ACK/NACK bit transfer (if the I2C peripheral was addressed) until I2CSC is set, which requires user intervention to prevent indefinite hold-ups of the I2C bus. But, if clock stretching is not enabled, the master will be allowed full control of the rate data is sent over the bus, which opens the possibility that the software running on this MCU does not notice that a byte has been transferred in time before the next is transferred. Check I2CSTM to see if the master has requested slave receiver or slave transmitter mode (1 indicates slave transmitter). Clear the status register. Queue the byte of data to transmit to the master by writing the byte to I2CSTX. If clock stretching is enabled, set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Wait for I2CSTXE to be set, indicating that the I2C peripheral is ready to queue the next byte to send to the master. Clear the status register, and write the next byte to send to the master to I2CxSTX. If clock stretching is enabled, wait for I2CSXC to be set, clear the status register, and set I2CSC. Wait for I2CSTXE, I2CSPR, or I2CSTR to be set, then clear the status register.

18.1 Register Description

18.1.1 I2CCR Register

Register memory slot: 0, offset: 0 bytes, size: 4 bytes

I2C control register

31	30	29	28	27	26	25	24
Unused							
23	22	21	20	19	18	17	16
Unused		MEN	SEN	SN	SCS	GCE	MDIV
rw-(0)		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
15	14	13	12	11	10	9	8
MDIV		SAIE	STXEIE	SOVFIE	SNRIE	SXCIE	
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)
7	6	5	4	3	2	1	0
MSTSIE	MSPSIE	MARBIE	MTXEIE	MNRIE	MXCIE	STRIE	SPRIE
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
Unused	Bits 31-22	

MEN	Bit 21	I2C master enable. When enabled, this device awaits a command to send a start condition with I2CMST, and then begins acting as a master until commanded to send a stop condition with I2CMSP. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave. 0 Disabled 1 Enabled
SEN	Bit 20	I2C slave enable. When enabled, this device behaves as an I2C slave and begins listening for its address. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave. 0 Disabled 1 Enabled
SN	Bit 19	I2C slave NACK next byte received. When enabled, this slave will reply with a NACK whenever it receives its address or whenever it receives a byte from a master. When disabled, it will send an ACK in those situations. If clock stretching is enabled, this bit can be changed in accordance with the desired ACK/NACK reply before allowing a rising edge of SCL. 0 ACK 1 NACK
SCS	Bit 18	I2C slave clock stretching enable. When enabled, this slave will hold the SCL line low during the ACK phase of the transmission to allow this slave more time. Note that the master will be left waiting for the ACK/NACK as long as this bit is set to 1. 0 Disabled 1 Enabled
GCE	Bit 17	I2C slave general call enable. When enabled, this slave will be addressed if a global call is issued on the bus. 0 Disabled 1 Enabled
MDIV	Bits 16-13	I2C master mode clock divider. The master mode finite state machine clock source is SMCLK, which is divided by a factor of $4 * 2^{**}I2CMDIV$. 0000 I2CMDIV_1: /1 (no division) 0001 I2CMDIV_2: /2 0010 I2CMDIV_4: /4 0011 I2CMDIV_8: /8 0100 I2CMDIV_16: /16 0101 I2CMDIV_32: /32 0110 I2CMDIV_64: /64 0111 I2CMDIV_128: /128 1000 I2CMDIV_256: /256 1001 I2CMDIV_512: /512 1010 I2CMDIV_1024: /1,024 1011 I2CMDIV_2048: /2,048

		1100	I2CMDIV_4096: /4,096
		1101	I2CMDIV_8192: /8,192
		1110	I2CMDIV_16384: /16,384
		1111	I2CMDIV_32768: /32,768
SAIE	Bit 12	I2C slave mode addressed interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
STXEIE	Bit 11	I2C slave transmit register empty interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
SOVFIE	Bit 10	I2C slave receive register overflow interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
SNRIE	Bit 9	I2C slave mode NACK received from master interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
SXCIE	Bit 8	I2C slave mode transfer complete interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
MSTSIE	Bit 7	I2C master mode start condition sent interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
MSPSIE	Bit 6	I2C master mode stop condition sent interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
MARBIE	Bit 5	I2C master mode arbitration error interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
MTXEIE	Bit 4	I2C master transmit register empty interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
MNRIE	Bit 3	I2C master mode NACK received interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
MXCIE	Bit 2	I2C master transfer complete interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
STRIE	Bit 1	I2C start received interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled
SPRIE	Bit 0	I2C stop received interrupt enable	
		0	Interrupt disabled
		1	Interrupt enabled

18.1.2 I2CFR Register

Register memory slot: 1, offset: 4 bytes, size: 1 byte

I2C flow control register. Writing a 1 to a bit in this register initiates or queues the associated command. Writing a 0 to a bit does nothing. Reading this register always returns the value 0.

7	6	5	4	3	2	1	0
<i>Unused</i>				SC	MST	MSP	MRB
				w1-(0)	w1-(0)	w1-(0)	w1-(0)

Bitfield	Range	Description
<i>Unused</i>	Bits 7-4	
SC	Bit 3	I2C slave continue command. When clock stretching is enabled in slave mode, set this bit to tell the slave to continue with the ACK/NACK phase of the current byte by releasing SCL. This may only be set if clock stretching is enabled, slave mode is enabled, and the slave transfer complete flag has just been set. 0 No effect 1 Send a start condition
MST	Bit 2	I2C master send start condition command. Set this bit to 1 to send a start condition or a repeated start condition. Once this bit is set, this master is required to send at least one address frame before initiating this command again. If another master has control of the bus when this bit is set, this master will wait until the bus is idle before seizing control of it and sending the start condition. If this master is busy with a transaction when this bit is set, then it will send a restart condition immediately after it finishes the transaction. 0 No effect 1 Send a start condition
MSP	Bit 1	I2C master send stop condition command. Set this bit to 1 while this master is in control of the bus to send a stop condition. This master is required to have already sent a start condition and at least one address frame before initiating this command. If this master is busy with a transaction when this bit is set, then it will send the stop condition immediately after it finishes the transaction. 0 No effect 1 Send a stop condition
MRB	Bit 0	I2C master read byte command. Set this bit to 1 while in master receiver mode to read one byte from the slave. This master is required to have already sent the slave address and received an ACK from the slave before initiating this command. 0 No effect 1 Read next byte

18.1.3 I2CSR Register

Register memory slot: 2, offset: 8 bytes, size: 2 bytes

I2C status register

15	14	13	12	11	10	9	8
BS	MCB	STM	SA	STXE	SOVF	SNR	SXC
r-(0)	r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)
7	6	5	4	3	2	1	0
MSTS	MSPS	MARB	MTXE	MNR	MXC	STR	SPR
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
BS	Bit 15	I2C bus state indicator. This bit cannot be cleared by writing to the status register. 0 The I2C bus is idle 1 The I2C bus is active
MCB	Bit 14	I2C master controls bus indicator. This bit cannot be cleared by writing to the status register. 0 This master does not control the bus 1 This master controls the bus
STM	Bit 13	I2C slave transmitter mode indicator. Indicates for which mode this slave has been addressed. Only valid if I2CSA is 1. This bit cannot be cleared by writing to the status register. 0 Slave receiver mode 1 Slave transmitter mode
SA	Bit 12	I2C slave mode addressed interrupt flag. Indicates that this slave has been addressed by another master. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
STXE	Bit 11	I2C slave transmit register empty interrupt flag. This bit is set when this slave latches the data stored in the masslaveter transmit register to indicate that the slave transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SOVF	Bit 10	I2C slave receive register overflow interrupt flag. Indicates that this slave has failed to read one or more bytes from the I2CxSRX register before they were overwritten by another transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SNR	Bit 9	I2C slave mode NACK received from master interrupt flag. This bit is set in slave transmitter mode if the master responds with a NACK after this slave sends it a byte of data. This bit is not set if the master responds with an ACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SXC	Bit 8	I2C slave mode transfer complete interrupt flag. This bit is set after this slave receives a byte of data from a master, but before this slave sends an ACK/NACK. Write a 1 to this bit to clear it.

		0 No pending interrupt
		1 Pending interrupt
MSTS	Bit 7	I2C master mode start condition sent interrupt flag. This flag is set after this master sends a start condition or repeated start condition. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt
MSPS	Bit 6	I2C master mode stop condition sent interrupt flag. This flag is set after this master sends a stop condition. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt
MARB	Bit 5	I2C master mode arbitration loss interrupt flag. This bit is set when this master detects that the value it tried to write to SDA is being overridden by another master. After it detects the arbitration loss, this master releases control of the bus. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt
MTXE	Bit 4	I2C master transmit register empty interrupt flag. This bit is set when this master latches the data stored in the master transmit register to indicate that the master transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt
MNR	Bit 3	I2C master mode NACK received interrupt flag. This flag is set in master transmitter mode after ACK/NACK bit is sent if the slave sends a NACK. It is not set if the slave sends an ACK. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt
MXC	Bit 2	I2C master transfer complete interrupt flag. In master transmitter mode, this flag is set after this master has sent the data byte and the slave has sent an ACK/NACK. In master receiver mode, this flag is set after the slave has sent the data byte and before this master sends an ACK/NACK. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt
STR	Bit 1	I2C start condition received interrupt flag. This flag is set whenever a start condition or repeated start condition is detected on the bus, regardless of if this master or another sent it. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt
SPR	Bit 0	I2C stop condition received interrupt flag. This flag is set whenever a stop condition condition is detected on the bus, regardless of which device sent it. Write a 1 to this bit to clear it.
		0 No pending interrupt
		1 Pending interrupt

18.1.4 I2CMTX Register

Register memory slot: 3, offset: 12 bytes, size: 1 byte

I2C master transmit register. Write the desired slave address and read/write bit to this register after sending a start condition to begin a transmission with a slave. Note that the desired slave address must occupy the upper seven bits and the read/write bit must occupy the least significant bit. If the read bit is 0, the master enters master transmitter mode. If the read bit is 1, the master enters master receiver mode. Write a byte of data to this register after sending the address frame or a data frame to send that byte of data to the slave. If this master is busy with a transmission when this register is written, it will send the byte after it finishes the transmission.



18.1.5 I2CMRX Register

Register memory slot: 4, offset: 16 bytes, size: 1 byte

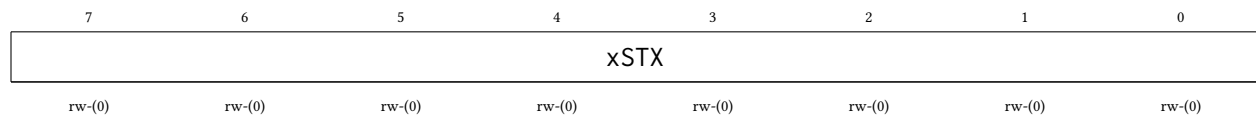
I2C master receive register. When in master receiver mode, read this register after the master transfer complete interrupt flag (I2CMXC) is set to get the received data byte.



18.1.6 I2CSTX Register

Register memory slot: 5, offset: 20 bytes, size: 1 byte

I2C slave transmit register. When in slave transmitter mode, write to this register after the slave addressed flag (I2CSA) or the slave transaction complete flag (I2CSXC) has been set to queue the next byte to send to the master.



18.1.7 I2CSRX Register

Register memory slot: 6, offset: 24 bytes, size: 1 byte

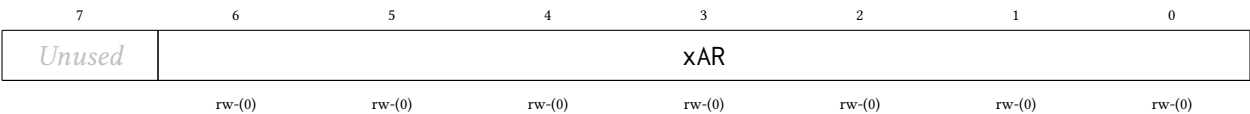
I2C slave receive register. When in slave receiver mode, read this register after the slave transaction complete flag (I2CSXC) has been set to get the data byte sent from the master. Note that if this slave fails to clear the status register before another byte is received, the slave receive overflow flag (I2CSOVF) will be set.



18.1.8 I2CAR Register

Register memory slot: 7, offset: 28 bytes, size: 1 byte

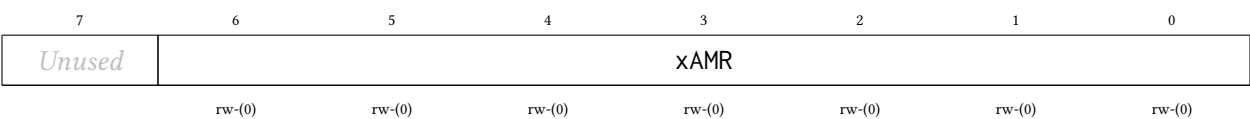
I2C this slave address register. When in slave mode, any master that sends an address frame containing this address will activate this slave.



18.1.9 I2CAMR Register

Register memory slot: 8, offset: 32 bytes, size: 1 byte

I2C this slave address mask register. Any bit set to 1 in this register indicates that the corresponding bit in the slave address register is a wildcard. Only the slave address register bits that correspond to 0s in this register will be compared to the received slave address.



19 Register and Peripheral Memory Map

19.1 GPI00 Peripheral

Base address: 0x4000, peripheral memory slot: 0

Register	Address	Slot	Offset (bytes)	Size (bytes)
P0IN	0x4000	0	0	1
P0OUT	0x4004	1	4	1
P0OUTS	0x4008	2	8	1
P0OUTC	0x400C	3	12	1
P0OUTT	0x4010	4	16	4
P0DIR	0x4014	5	20	1
P0IFG	0x4018	6	24	1
P0IES	0x401C	7	28	1
P0IE	0x4020	8	32	1
P0SEL	0x4024	9	36	1
P0REN	0x4028	10	40	1

19.2 GPI01 Peripheral

Base address: 0x4100, peripheral memory slot: 1

Register	Address	Slot	Offset (bytes)	Size (bytes)
P1IN	0x4100	0	0	1
P1OUT	0x4104	1	4	1
P1OUTS	0x4108	2	8	1
P1OUTC	0x410C	3	12	1
P1OUTT	0x4110	4	16	4
P1DIR	0x4114	5	20	1
P1IFG	0x4118	6	24	1
P1IES	0x411C	7	28	1
P1IE	0x4120	8	32	1
P1SEL	0x4124	9	36	1
P1REN	0x4128	10	40	1

19.3 SPI0 Peripheral

Base address: 0x4200, peripheral memory slot: 2

Register	Address	Slot	Offset (bytes)	Size (bytes)
SPI0CR	0x4200	0	0	4
SPI0SR	0x4204	1	4	1
SPI0TX	0x4208	2	8	4
SPI0RX	0x420C	3	12	4
SPI0FOS	0x4210	4	16	4

19.4 SPI1 Peripheral

Base address: 0x4300, peripheral memory slot: 3

Register	Address	Slot	Offset (bytes)	Size (bytes)
SPI1CR	0x4300	0	0	4
SPI1SR	0x4304	1	4	1
SPI1TX	0x4308	2	8	4
SPI1RX	0x430C	3	12	4
SPI1FOS	0x4310	4	16	4

19.5 UART0 Peripheral

Base address: 0x4400, peripheral memory slot: 4

Register	Address	Slot	Offset (bytes)	Size (bytes)
UART0CR	0x4400	0	0	1
UART0SR	0x4404	1	4	1
UART0BR	0x4408	2	8	2
UART0RX	0x440C	3	12	1
UART0TX	0x4410	4	16	1

19.6 UART1 Peripheral

Base address: 0x4500, peripheral memory slot: 5

Register	Address	Slot	Offset (bytes)	Size (bytes)
UART1CR	0x4500	0	0	1
UART1SR	0x4504	1	4	1
UART1BR	0x4508	2	8	2
UART1RX	0x450C	3	12	1
UART1TX	0x4510	4	16	1

19.7 TIMER0 Peripheral

Base address: 0x4600, peripheral memory slot: 6

Register	Address	Slot	Offset (bytes)	Size (bytes)
TIM0CR	0x4600	0	0	4
TIM0SR	0x4604	1	4	1
TIM0VAL	0x4608	2	8	4
TIM0CMP0	0x460C	3	12	4
TIM0CMP1	0x4610	4	16	4
TIM0CMP2	0x4614	5	20	4
TIM0CAP0	0x4618	6	24	4
TIM0CAP1	0x461C	7	28	4

19.8 TIMER1 Peripheral

Base address: 0x4700, peripheral memory slot: 7

Register	Address	Slot	Offset (bytes)	Size (bytes)
----------	---------	------	----------------	--------------

TIM1CR	0x4700	0	0	4
TIM1SR	0x4704	1	4	1
TIM1VAL	0x4708	2	8	4
TIM1CMP0	0x470C	3	12	4
TIM1CMP1	0x4710	4	16	4
TIM1CMP2	0x4714	5	20	4
TIM1CAP0	0x4718	6	24	4
TIM1CAP1	0x471C	7	28	4

19.9 GPIO2 Peripheral

Base address: 0x4800, peripheral memory slot: 8

Register	Address	Slot	Offset (bytes)	Size (bytes)
P2IN	0x4800	0	0	1
P2OUT	0x4804	1	4	1
P2OUTS	0x4808	2	8	1
P2OUTC	0x480C	3	12	1
P2OUTT	0x4810	4	16	4
P2DIR	0x4814	5	20	1
P2IFG	0x4818	6	24	1
P2IES	0x481C	7	28	1
P2IE	0x4820	8	32	1
P2SEL	0x4824	9	36	1
P2REN	0x4828	10	40	1

19.10 SYSTEM Peripheral

Base address: 0x4900, peripheral memory slot: 9

Register	Address	Slot	Offset (bytes)	Size (bytes)
SYSCLKCR	0x4900	0	0	2
CLKDIVCR	0x4904	1	4	1
MEMPWRCR	0x4908	2	8	2
CRCDATA	0x490C	3	12	1
CRCSTATE	0x4910	4	16	2
IRQEN	0x4914	5	20	4
IRQPRI	0x4918	6	24	4
WDTPASS	0x491C	7	28	4
WDTCR	0x4920	8	32	1
WDTSR	0x4924	9	36	1
WDTVAL	0x4928	10	40	4
DC00FREQ	0x492C	11	44	2
DC01FREQ	0x4930	12	48	2
TPMR	0x4934	13	52	1
BIASCR	0x4938	14	56	2
BIASDBP	0x493C	15	60	2
BIASDBPC	0x4940	16	64	2

BIASDBNC	0x4944	17	68	2
BIASDBN	0x4948	18	72	2

19.11 NN0 Peripheral

Base address: 0x4A00, peripheral memory slot: 10

Register	Address	Slot	Offset (bytes)	Size (bytes)
NN0CR	0x4A00	0	0	4
NN0IVA	0x4A04	1	4	4
NN0OVA	0x4A08	2	8	4
NN0WMA	0x4A0C	3	12	4
NN0LSI	0x4A10	4	16	4
NN0LSO	0x4A14	5	20	2

19.12 SARADC0 Peripheral

Base address: 0x4B00, peripheral memory slot: 11

Register	Address	Slot	Offset (bytes)	Size (bytes)
SARADC0CR	0x4B00	0	0	2
SARADC0CDIV	0x4B04	1	4	1
SARADC0SR	0x4B08	2	8	1
SARADC0DATA	0x4B0C	3	12	2
SARADC0TPR	0x4B10	4	16	1

19.13 AFE0 Peripheral

Base address: 0x4C00, peripheral memory slot: 12

Register	Address	Slot	Offset (bytes)	Size (bytes)
AFE0CR0	0x4C00	0	0	4
AFE0CR1	0x4C04	1	4	4
AFE0CFB	0x4C08	2	8	1
AFE0RFB	0x4C0C	3	12	2
AFE0THR	0x4C10	4	16	2
AFE0TPR	0x4C14	5	20	4
AFE0SPT	0x4C18	6	24	1
AFE0PIT	0x4C1C	7	28	2
AFE0EIT	0x4C20	8	32	2
AFE0LIT	0x4C24	9	36	2
AFE0RJT	0x4C28	10	40	2
AFE0RST	0x4C2C	11	44	1
AFE0AOFST	0x4C30	12	48	2
AFE0BLLT	0x4C34	13	52	2
AFE0CSAREF	0x4C38	14	56	2
AFE0CSABP	0x4C3C	15	60	2
AFE0CSABPC	0x4C40	16	64	2
AFE0CSABNC	0x4C44	17	68	2

AFE0CSABN	0x4C48	18	72	2
AFE0CMSHR	0x4C4C	19	76	2
AFE0CLPF	0x4C50	20	80	1
AFE0SR	0x4C54	21	84	1
AFE0ADCVAL	0x4C58	22	88	2
AFE0VPC	0x4C5C	23	92	4
AFE0TPC	0x4C60	24	96	4

19.14 GPIO3 Peripheral

Base address: 0x4D00, peripheral memory slot: 13

Register	Address	Slot	Offset (bytes)	Size (bytes)
P3IN	0x4D00	0	0	1
P3OUT	0x4D04	1	4	1
P3OUTS	0x4D08	2	8	1
P3OUTC	0x4D0C	3	12	1
P3OUTT	0x4D10	4	16	4
P3DIR	0x4D14	5	20	1
P3IFG	0x4D18	6	24	1
P3IES	0x4D1C	7	28	1
P3IE	0x4D20	8	32	1
P3SEL	0x4D24	9	36	1
P3REN	0x4D28	10	40	1

19.15 I2C0 Peripheral

Base address: 0x4E00, peripheral memory slot: 14

Register	Address	Slot	Offset (bytes)	Size (bytes)
I2C0CR	0x4E00	0	0	4
I2C0FCR	0x4E04	1	4	1
I2C0SR	0x4E08	2	8	2
I2C0MTX	0x4E0C	3	12	1
I2C0MRX	0x4E10	4	16	1
I2C0STX	0x4E14	5	20	1
I2C0SRX	0x4E18	6	24	1
I2C0AR	0x4E1C	7	28	1
I2C0AMR	0x4E20	8	32	1

19.16 I2C1 Peripheral

Base address: 0x4F00, peripheral memory slot: 15

Register	Address	Slot	Offset (bytes)	Size (bytes)
I2C1CR	0x4F00	0	0	4
I2C1FCR	0x4F04	1	4	1
I2C1SR	0x4F08	2	8	2
I2C1MTX	0x4F0C	3	12	1

I2C1MRX	0x4F10	4	16	1
I2C1STX	0x4F14	5	20	1
I2C1SRX	0x4F18	6	24	1
I2C1AR	0x4F1C	7	28	1
I2C1AMR	0x4F20	8	32	1

20 Errata

Listed are the known issues with the Myshkin MCU:

-